

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



NGUYỄN TRỌNG NHÂN

**MẠNG ĐỐI NGHỊCH TẠO SINH TRONG CHUYỂN ĐỔI
ẢNH CHÂN DUNG SANG PHONG CÁCH ANIME**

LUẬN VĂN THẠC SĨ KHOA HỌC MÁY TÍNH

HÀ NỘI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



NGUYỄN TRỌNG NHÂN

**MẠNG ĐỐI NGHỊCH TẠO SINH TRONG CHUYỂN ĐỔI
ẢNH CHÂN DUNG SANG PHONG CÁCH ANIME**

Ngành: Khoa học máy tính
Chuyên ngành: Khoa học máy tính
Mã số: 7480101

LUẬN VĂN THẠC SĨ KHOA HỌC MÁY TÍNH

NGƯỜI HƯỚNG DẪN KHOA HỌC: TS. MA THỊ CHÂU

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



NGUYEN TRONG NHAN

**GENERATIVE ADVERSARIAL NETWORKS FOR
CONVERTING PORTRAITS TO ANIME STYLE**

Major: Computer Science
Specialization: Computer Science
Code: 7480101

MASTER'S THESIS IN COMPUTER SCIENCE

SUPERVISOR: DR. MA THI CHAU

HANOI - 2026

LỜI CẢM ƠN

Trước hết, tôi xin chân thành cảm ơn TS. Ma Thị Châu, người hướng dẫn đề tài nghiên cứu lần này, đã tận tình hỗ trợ, chỉ bảo và động viên tôi trong suốt quá trình nghiên cứu và hoàn thành. Nhờ có sự hướng dẫn của cô, tôi đã có được những kiến thức quý báu và kinh nghiệm thực tiễn trong lĩnh vực nghiên cứu của mình.

Tôi cũng xin gửi lời cảm ơn đến các thầy cô giáo của Trường Đại học Công Nghệ, đã truyền đạt cho tôi những kiến thức cơ bản và nâng cao về ngành Khoa học Máy Tính. Tôi xin chúc các thầy cô luôn mạnh khỏe và thành công trong công tác giảng dạy và nghiên cứu khoa học.

Tôi xin gửi lời cảm ơn đặc biệt đến tác giả của bài báo DTGAN (AnimeGANv3) – anh Asher Chan (asher_chan@foxmail.com) – đã nhiệt tình hỗ trợ tôi trong việc lý giải mã nguồn và kiến trúc mô hình trong suốt quá trình nghiên cứu. Sự giúp đỡ của anh đã giúp tôi hiểu sâu hơn về các chi tiết kỹ thuật và hoàn thành luận văn một cách trọn vẹn.

Sau cùng, tôi muốn bày tỏ lòng biết ơn sâu sắc đến gia đình, người thân và bạn bè, những người luôn ở bên cạnh tôi trong những thời điểm khó khăn nhất, luôn động viên và khích lệ tôi trong cuộc sống và công việc. Thành tựu này của tôi không thể có được nếu thiếu sự giúp đỡ của họ.

Mặc dù tôi đã cố gắng hoàn thành báo cáo nhưng không thể tránh khỏi những sai sót, tôi rất mong nhận được nhận xét và sự hướng dẫn từ phía giáo viên và hội đồng.

Hà Nội, ngày ... tháng ... năm 2026

Học viên

Nguyễn Trọng Nhân

LỜI CAM ĐOAN

Tôi xin cam đoan rằng tất cả các nội dung trong tài liệu này đều là kết quả của công trình nghiên cứu cá nhân của tôi dưới sự hướng dẫn của TS.Ma Thị Châu. Tôi không sao chép bất kỳ tài liệu hay công trình nghiên cứu nào của người khác mà không chỉ rõ nguồn gốc trong phần tài liệu tham khảo. Tôi hiểu rằng việc sao chép trái phép là một hành vi vi phạm đạo đức học thuật và tôi sẽ chịu trách nhiệm về những hành vi này.

Tôi cam kết rằng, bản báo cáo của tôi không vi phạm bản quyền của bất kỳ ai và không vi phạm bất kỳ quyền sở hữu nào, cũng như bất kỳ ý tưởng, kỹ thuật, trích dẫn hoặc bất kỳ tài liệu nào từ công trình nghiên cứu của người khác. Tôi xin nhận đầy đủ trách nhiệm và sẵn sàng chấp nhận mọi biện pháp kỷ luật nếu vi phạm cam kết.

Hà Nội, ngày ... tháng ... năm 2026

Học viên

Nguyễn Trọng Nhân

TÓM TẮT

Tóm tắt: Sự phát triển mạnh mẽ của thị trường Anime/Manga toàn cầu cùng với nhu cầu ngày càng lớn về các công cụ sáng tạo nội dung dựa trên trí tuệ nhân tạo đã thúc đẩy nghiên cứu bài toán chuyển đổi ảnh chân dung người thật sang phong cách anime 2D (photo-to-anime). Đây là bài toán giàu tiềm năng ứng dụng nhưng đặt ra nhiều thách thức về chất lượng hình ảnh, khả năng bảo toàn đặc điểm nhận dạng khuôn mặt và tính ổn định khi huấn luyện. Các phương pháp trước đó như Neural Style Transfer hay CycleGAN thường gặp hạn chế về nhiễu, tạo tác (artifacts) và tốc độ xử lý.

Luận văn này nghiên cứu, tái hiện và mở rộng mô hình AnimeGANv3 dựa trên kiến trúc Double-Tail GAN (DTGAN) – một kiến trúc Mạng đối nghịch tạo sinh (GAN) thế hệ mới sử dụng Generator hai nhánh (Support Tail và Main Tail), kỹ thuật chuẩn hóa Linearly Adaptive Denormalization (LADE), cơ chế External Attention v3 và hệ thống hàm mất mát chuyên biệt kết hợp mất mát đối nghịch, mất mát nội dung (dựa trên VGG19), mất mát phong cách, mất mát màu sắc và mất mát làm mượt vùng. Đóng góp chính của luận văn là xây dựng và tích hợp module Face Mode sử dụng RetinaFace MobileNet 0.25 kết hợp thuật toán Margin Expansion, tạo thành pipeline hoàn chỉnh: phát hiện khuôn mặt → cắt và mở rộng vùng mặt → chuyển đổi anime chất lượng cao.

Mô hình được huấn luyện trên nền tảng điện toán đám mây vast.ai với hai tập dữ liệu không ghép cặp gồm ảnh chân dung người thật và ảnh anime phong cách Hayao Miyazaki, Makoto Shinkai. Quá trình huấn luyện qua 2 giai đoạn (5 epoch pre-training và 69 epoch adversarial) cho thấy hội tụ ổn định. Kết quả thực nghiệm đạt FID = 58,6 và LPIPS = 0,384 ở chế độ Face Mode – vượt trội so với CycleGAN, CartoonGAN và AnimeGANv2. Module Face Mode cải thiện khoảng 8,7% FID so với chế độ Landscape Mode. Model ONNX chỉ 9,1 MB đạt tốc độ 42 FPS tại 256×256 trên GPU. Hệ thống được triển khai thành ứng dụng web hoàn chỉnh (React frontend và FastAPI backend), hỗ trợ chuyển đổi cả ảnh và video. Luận văn cũng đề xuất các hướng phát triển tương lai bao gồm face alignment, face blending, temporal consistency cho video và mở rộng đa phong cách.

Từ khóa: GAN, AnimeGANv3, DTGAN, chuyển phong cách ảnh, ảnh chân dung, phong cách anime, LADE, RetinaFace, Face Mode, ONNX.

ABSTRACT

Abstract: The rapid growth of the global Anime/Manga market, coupled with the increasing demand for AI-powered content creation tools, has driven research into the problem of converting real-person portraits to 2D anime style (photo-to-anime). This task offers significant application potential but poses many challenges regarding image quality, facial identity preservation, and training stability. Previous approaches such as Neural Style Transfer and CycleGAN often suffer from noise, artifacts, and slow processing speed.

This thesis investigates, reproduces, and extends the AnimeGANv3 model based on the Double-Tail GAN (DTGAN) architecture – a next-generation Generative Adversarial Network (GAN) employing a dual-branch Generator (Support Tail and Main Tail), Linearly Adaptive Denormalization (LADE), External Attention v3, and a specialized loss system combining adversarial loss, content loss (based on VGG19), style loss, color loss, and region smoothing loss. The main contribution of this thesis is the design and integration of a Face Mode module using RetinaFace MobileNet 0.25 combined with a Margin Expansion algorithm, forming a complete pipeline: face detection → face region cropping and expansion → high-quality anime conversion.

The model was trained on the vast.ai cloud computing platform using two unpaired datasets consisting of real-person portraits and anime images in the styles of Hayao Miyazaki and Makoto Shinkai. The two-phase training process (5 epochs of pre-training and 69 epochs of adversarial training) demonstrated stable convergence. Experimental results achieved $FID = 58.6$ and $LPIPS = 0.384$ in Face Mode – outperforming CycleGAN, CartoonGAN, and AnimeGANv2. The Face Mode module improved FID by approximately 8.7% compared to Landscape Mode. The ONNX model is only 9.1 MB and achieves 42 FPS at 256×256 on GPU. The system was deployed as a complete web application (React frontend and FastAPI backend), supporting both image and video conversion. The thesis also proposes future development directions including face alignment, face blending, temporal consistency for video, and multi-style expansion.

Keywords: *GAN, AnimeGANv3, DTGAN, image style transfer, portrait, anime style, LADE, RetinaFace, Face Mode, ONNX.*

MỤC LỤC

Lời cảm ơn	
Lời cam đoan	
Tóm tắt	
Abstract	
Mục lục	i
Danh mục hình vẽ	v
Danh mục bảng biểu	vii
MỞ ĐẦU	1
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN NGHIÊN CỨU	5
1.1. Bài toán Chuyển ảnh sang Phong cách Anime	5
1.1.1. Định nghĩa bài toán	5
1.1.2. Đặc trưng thị giác của phong cách Anime	5
1.1.3. Thách thức kỹ thuật	5
1.2. Nền tảng Mạng Nơ-ron Tích chập	6
1.2.1. Lớp Tích chập, Pooling và Hàm kích hoạt	6
1.2.2. Các Kỹ thuật Chuẩn hóa	7
1.2.3. Kiến trúc Encoder-Decoder	8
1.3. Kiến trúc Mạng Đối nghịch Tạo sinh	9
1.3.1. Nguyên lý Hoạt động và Hàm Mục tiêu	9
1.3.2. Thách thức trong Huấn luyện GAN	11
1.3.3. Các Giải pháp về Hàm Mục tiêu	15
1.3.4. Đánh giá Chất lượng GAN	18
1.4. Các mô hình GAN tiêu biểu cho bài toán Anime	19
1.4.1. CycleGAN và dịch ảnh không ghép cặp	19
1.4.2. StyleGAN và GAN Inversion	20
1.4.3. AnimeGANv3 (DTGAN) – Sơ bộ kiến trúc	21
1.4.4. So sánh tổng quan và Định hướng	22
1.5. Mô hình Khuếch tán và So sánh với GAN	23
1.5.1. Nguyên lý Hoạt động của Mô hình Khuếch tán	24
1.5.2. Mô hình Khuếch tán cho Chuyển Phong cách	24
1.5.3. So sánh GAN và Mô hình Khuếch tán cho Bài toán Anime	25
1.5.4. Biện luận Lựa chọn GAN	25
1.6. Phát hiện Khuôn mặt – Sơ bộ	26
1.6.1. Tổng quan các Phương pháp	26

1.6.2. RetinaFace – Sơ bộ Kiến trúc	27
CHƯƠNG 2. KIẾN TRÚC MÔ HÌNH AnimeGANv3 (DTGAN)	31
2.1. Tổng quan kiến trúc DTGAN	31
2.1.1. Động lực thiết kế: vì sao cần kiến trúc hai nhánh	31
2.1.2. Ba thành phần của Generator	32
2.2. Mạng Generator (G_net)	35
2.2.1. Base Encoder (Chia sẻ)	35
2.2.2. Support Tail (Decoder nhánh hỗ trợ)	36
2.2.3. Main Tail (Decoder nhánh chính)	36
2.3. Kỹ thuật chuẩn hóa LADE	37
2.3.1. Động lực và ý tưởng	37
2.3.2. Công thức toán học của LADE	38
2.3.3. Diễn giải bổ sung và phân tích chi phí	40
2.3.4. So sánh LADE với các kỹ thuật chuẩn hóa khác	41
2.3.5. Biểu thức LADE_D trong Discriminator	41
2.4. Cơ chế External Attention v3	42
2.4.1. Hạn chế của Self-Attention và Giải pháp	42
2.4.2. Kiến trúc External Attention v3	43
2.4.3. Cơ chế Double Normalization	44
2.4.4. Phân tích độ phức tạp	45
2.5. Discriminator (D_net)	45
2.6. Các hàm mất mát được sử dụng trong kiến trúc	47
2.6.1. Tổng quan hệ thống hàm mất mát	48
2.6.2. Hàm mất mát Giai đoạn Pre-training	49
2.6.3. Content Loss (Perceptual Loss)	49
2.6.4. Style Loss (Decentralized Gram Matrix)	50
2.6.5. Region Smoothing Loss	51
2.6.6. Lab Color Loss	52
2.6.7. Total Variation Loss	52
2.6.8. Adversarial Loss (LSGAN)	53
2.6.9. Fine-grained Revision Loss (Main Tail)	54
2.6.10. Tổng hợp Loss	54
2.7. Xử lý Hậu kỳ: Guided Filter	55
2.8. Quy trình Huấn luyện	56
2.8.1. Hai giai đoạn huấn luyện	56
2.8.2. Siêu tham số huấn luyện	57
2.8.3. Tiền xử lý Dữ liệu Huấn luyện	58
2.9. Phân tích và Điểm nổi bật của DTGAN	58

2.9.1. Ưu điểm so với AnimeGAN và AnimeGANv2	58
2.9.2. Hạn chế của kiến trúc	59
CHƯƠNG 3. LUỒNG ANIME TÍCH HỢP PHÁT HIỆN KHUÔN MẶT	62
3.1. Tổng quan Pipeline Tích hợp	62
3.1.1. Nguyên tắc thiết kế: can thiệp ở tầng pipeline	62
3.1.2. Ba chế độ vận hành	63
3.2. Phát hiện Khuôn mặt với RetinaFace	64
3.2.1. Tổng quan RetinaFace	64
3.2.2. Kiến trúc Mạng Phát hiện	65
3.2.3. Tiền xử lý Ảnh Đầu vào	66
3.2.4. Hậu xử lý và Prior Box	67
3.3. Kỹ thuật Mở rộng Vùng Khuôn mặt (Margin Expansion)	70
3.3.1. Động lực và Mục tiêu	70
3.3.2. Thuật toán Margin Expansion	72
3.3.3. Tính chất của thuật toán	73
3.3.4. Xử lý Trường hợp Đặc biệt (Edge Cases)	74
3.3.5. Ý nghĩa của Tham số margin = 0.5	74
3.4. Phân tích Độ phân giải Hiệu dụng của Khuôn mặt	75
3.4.1. Định nghĩa và công thức	76
3.4.2. Diễn giải và điểm hòa vốn	76
3.5. Luồng xử lý Ảnh trong Pipeline	77
3.5.1. Đọc và Giới hạn Kích thước Ảnh	77
3.5.2. Xử lý Phân nhánh: Face / Landscape / Hybrid	78
3.5.3. Chuẩn hóa và Suy luận ONNX	79
3.5.4. Tóm tắt Luồng Dữ liệu	79
3.6. Chế độ Hybrid Mode	80
3.6.1. Nguyên lý Hoạt động	80
3.6.2. Ánh xạ Tọa độ giữa Ảnh Giới hạn và Ảnh Gốc	82
3.6.3. Kỹ thuật Feathered Blending	83
3.6.4. Phân tích Độ phức tạp	86
3.7. Phân tích Kỹ thuật và Đánh giá Thiết kế	86
3.7.1. Lý do Chọn RetinaFace MobileNet 0.25	86
3.7.2. Đánh giá Pipeline Tổng thể	87
3.7.3. Hạn chế của Thiết kế	88
3.7.4. Cân nhắc về Quyền riêng tư	88
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ	91
4.1. Môi trường Thực nghiệm	91
4.1.1. Cấu hình Phần cứng	91

4.1.2. Cấu hình Phần mềm	92
4.2. Bộ Dữ liệu	92
4.2.1. Dữ liệu Ảnh Thực (Photo)	92
4.2.2. Dữ liệu Ảnh Anime (Style)	92
4.3. Kết quả Huấn luyện	93
4.3.1. Giai đoạn Pre-training Generator	93
4.3.2. Giai đoạn Huấn luyện Đầy đủ	94
4.4. Đánh giá Định lượng	96
4.4.1. Các Chỉ số Đánh giá	96
4.4.2. Kết quả So sánh Định lượng	97
4.4.3. Tốc độ Suy luận theo Độ Phân giải	97
4.5. Đánh giá Định tính	98
4.5.1. So sánh ba Chế độ: Hybrid, Landscape và Face Mode	98
4.5.2. Phân tích Ảnh hưởng của Module Face Mode	100
4.6. Đánh giá Module Face Mode	100
4.6.1. Độ chính xác Phát hiện Khuôn mặt	100
4.6.2. Phân tích Tham số margin	100
4.6.3. Xử lý Trường hợp Đặc biệt	101
4.7. Đánh giá Người dùng (User Study)	101
4.8. Thảo luận và Phân tích	102
4.8.1. Điểm mạnh của Hệ thống	102
4.8.2. Hạn chế và Định hướng Cải thiện	103
4.8.3. So sánh với Mô hình Khuếch tán	103
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	105
5.1. Kết luận	105
5.1.1. Mức độ hoàn thành mục tiêu	106
5.2. Hạn chế	106
5.3. Hướng phát triển	106
5.3.1. Cải thiện chất lượng xử lý khuôn mặt	106
5.3.2. Xử lý video với temporal consistency	107
5.3.3. Mở rộng đa phong cách và triển khai mobile	107
5.4. Lời kết	107
TÀI LIỆU THAM KHẢO	109

DANH MỤC HÌNH VẼ

Hình 1.1	Kiến trúc cơ bản của Mạng Nơ-ron Tích chập	6
Hình 1.2	So sánh các hàm kích hoạt: Sigmoid, Tanh, ReLU và Leaky ReLU	7
Hình 1.3	So sánh bốn kỹ thuật chuẩn hóa trên tensor ($N \times C \times H \times W$)	8
Hình 1.4	Kiến trúc U-Net với skip connections	9
Hình 1.5	Kiến trúc tổng quát của Mạng Đối nghịch Tạo sinh (GAN)	10
Hình 1.6	Minh họa hiện tượng Mode Collapse trong GAN	13
Hình 1.7	Động lực học huấn luyện GAN: quỹ đạo xoay tròn trong không gian tham số và vấn đề Không Hội tụ	14
Hình 1.8	So sánh gradient flow giữa GAN gốc (JS divergence) và WGAN (Wasserstein distance) khi hai phân phối không giao nhau	16
Hình 1.9	Quy trình tính FID và IS: cả hai đều trích xuất đặc trưng qua mạng Inception-v3, nhưng IS so sánh với phân phối nhãn còn FID so sánh với phân phối đặc trưng của ảnh thật	19
Hình 1.10	Kiến trúc CycleGAN: hai Generator G, F và hai Discriminator D_X, D_Y huấn luyện đồng thời với ràng buộc cycle consistency	20
Hình 1.11	Sơ đồ tổng quát của Double-Tail GAN (DTGAN): Base Encoder chia sẻ đặc trưng cho Support Tail (học đường nét) và Main Tail (học màu sắc)	22
Hình 1.12	Biểu đồ radar so sánh đa chiều các hướng tiếp cận GAN cho bài toán anime theo 6 tiêu chí	23
Hình 1.13	Kiến trúc RetinaFace với backbone MobileNet-0.25, FPN đa tỉ lệ và ba nhánh detection head đa nhiệm	28
Hình 2.1	Tổng quan kiến trúc DTGAN (AnimeGANv3)	33
Hình 2.2	Cơ chế hoạt động của LADE	40
Hình 2.3	Kiến trúc External Attention v3 sử dụng trong DTGAN tại vị trí bottleneck 32×32	43
Hình 2.4	Pipeline huấn luyện hai giai đoạn và hệ thống hàm mất mát của DTGAN	48
Hình 3.1	Pipeline tích hợp Face Extraction vào AnimeGANv3	63
Hình 3.2	Kiến trúc RetinaFace với backbone MobileNet 0.25 và Feature Pyramid Network (FPN)	65
Hình 3.3	Minh họa kỹ thuật Margin Expansion	71

Hình 3.4	Pipeline xử lý Hybrid Mode: kết hợp Landscape Inference cho nền và Face Inference cho từng khuôn mặt	81
Hình 3.5	Kỹ thuật Feathered Blending: mặt nạ alpha với viền mờ Gaussian tạo vùng chuyển tiếp liền mạch	84
Hình 4.1	Đường cong hội tụ Pre-train G_loss trong giai đoạn pre-training (Epoch 0–4)	93
Hình 4.2	Đường cong G_loss (xanh lam) và D_loss (cam) qua 28.632 steps trong giai đoạn huấn luyện đầy đủ (Epoch 5–73)	94
Hình 4.3	Đường cong chi tiết bốn thành phần loss phụ trong giai đoạn adversarial training	95
Hình 4.4	Giá trị loss trung bình theo epoch: (trái) G_loss và D_loss, (phải) các thành phần loss phụ	96
Hình 4.5	So sánh kết quả chuyển đổi anime giữa Hybrid, Landscape và Face Mode trên ảnh chụp thẳng mặt	98
Hình 4.6	So sánh ba chế độ trên ảnh nhóm người	99
Hình 4.7	So sánh ba chế độ trên ảnh chân dung góc nghiêng	99

DANH MỤC BẢNG BIỂU

Bảng 1.1	Tóm tắt các chế độ thất bại của GAN và giải pháp tương ứng	14
Bảng 1.1	Tóm tắt các chế độ thất bại của GAN và giải pháp tương ứng	15
Bảng 1.2	So sánh định lượng các mô hình chuyển ảnh sang anime (FID đánh giá trên tập ảnh chân dung, inference trên GPU RTX A5000)	22
Bảng 1.3	So sánh AnimeGANv3 (GAN) và Stable Diffusion + ControlNet (Diffusion) cho bài toán chuyển phong cách anime – đánh giá theo tiêu chí triển khai thực tế	25
Bảng 1.4	So sánh các phương pháp phát hiện khuôn mặt theo tiêu chí triển khai thực tế	27
Bảng 2.1	Cấu trúc chi tiết Base Encoder của DTGAN	35
Bảng 2.2	So sánh Support Tail và Main Tail trong DTGAN	37
Bảng 2.3	So sánh các kỹ thuật chuẩn hóa trong mạng chuyển đổi phong cách ảnh	41
Bảng 2.4	So sánh độ phức tạp giữa Self-Attention và External Attention tại bottleneck 32×32 ($N = 1024, C = 128, k = 128$)	45
Bảng 2.5	Tổng hợp hệ thống hàm mất mát của DTGAN	49
Bảng 2.6	Siêu tham số huấn luyện của DTGAN trong thực nghiệm của luận văn	57
Bảng 2.7	So sánh DTGAN với các mô hình chuyển đổi ảnh anime tiền nhiệm	59
Bảng 3.1	Cấu hình Prior Box của RetinaFace với backbone MobileNet 0.25	67
Bảng 3.2	Độ phân giải hiệu dụng của khuôn mặt theo từng kích bản ảnh	76
Bảng 3.3	So sánh ba chế độ xử lý trong pipeline	78
Bảng 3.4	Phân tích độ phức tạp thời gian giữa ba chế độ	86
Bảng 3.5	So sánh các phương pháp phát hiện khuôn mặt cho ứng dụng thực tế	87
Bảng 4.1	Cấu hình phần cứng sử dụng trong thực nghiệm	91
Bảng 4.2	Cấu hình phần mềm và thư viện sử dụng	92
Bảng 4.3	Tóm tắt các chỉ số đánh giá định lượng sử dụng trong thực nghiệm	96
Bảng 4.4	Kết quả đánh giá định lượng trên tập ảnh chân dung (FID đo trên 1.000 ảnh, ONNX inference trên GPU RTX A5000)	97
Bảng 4.5	Tốc độ suy luận ONNX theo độ phân giải ảnh đầu vào	97
Bảng 4.6	So sánh chất lượng chuyển đổi có và không sử dụng module Face Mode	100

Bảng 4.7	Kết quả phát hiện khuôn mặt theo kịch bản khác nhau (ngưỡng confidence = 0.8)	100
Bảng 4.8	Ảnh hưởng của tham số margin đến chất lượng ảnh đầu ra và mức độ bao phủ ngữ cảnh	101
Bảng 4.9	Kết quả xử lý các trường hợp đặc biệt trong module Face Mode . .	101
Bảng 4.10	Kết quả đánh giá cảm nhận người dùng: Hybrid Mode so với Landscape Mode (thang Likert 5 điểm)	102
Bảng 4.11	So sánh hiệu năng thực tế giữa hệ thống AnimeGANv3 và phương pháp dựa trên Diffusion cho bài toán chuyển phong cách anime	103
Bảng 5.1	Đánh giá mức độ hoàn thành các mục tiêu nghiên cứu	106

MỞ ĐẦU

Tính cấp thiết và ý nghĩa của đề tài

Sự phát triển mạnh mẽ của nghệ thuật kỹ thuật số cùng với thị trường Anime/Manga toàn cầu đã tạo động lực lớn cho các nghiên cứu về chuyển đổi phong cách hình ảnh. Theo thống kê năm 2024, quy mô thị trường anime toàn cầu đạt khoảng 81,96 tỷ USD và dự kiến vượt 200 tỷ USD vào năm 2034. Công nghệ AI hiện nay cũng đang dần xâm nhập vào quy trình sản xuất anime – ví dụ, các công cụ Generative AI đã có thể tự động hoá việc vẽ phông nền và tô màu, giúp giảm bớt công việc lặp lại cho các họa sĩ. Trong bối cảnh đó, nhu cầu tự động hoá chuyển đổi ảnh thực sang phong cách anime trở nên cấp thiết, nhằm phục vụ cộng đồng người hâm mộ khổng lồ và ngành công nghiệp sáng tạo nội dung đang bùng nổ.

Tuy nhiên, việc chuyển một ảnh chân dung người thật sang tranh anime là thách thức không nhỏ. Phong cách anime có đặc trưng rất khác biệt (đường nét đơn giản, mắt to, màu sắc phẳng, v.v.), trong khi ảnh chụp chứa nhiều chi tiết thực tế phức tạp. Các phương pháp truyền thống dựa trên Neural Style Transfer thường gặp khó khăn trong việc giữ được nội dung gốc và dễ sinh ra nhiều và lỗi đồ họa (artifacts) khi khác biệt phong cách quá lớn. Thêm vào đó, các phương pháp như CycleGAN tuy có khả năng học dạng chuyển đổi không ghép cặp nhưng còn chậm và chất lượng đầu ra chưa ổn định. Vẽ tay thủ công bởi các họa sĩ tuy chất lượng cao nhưng tốn kém thời gian và công sức. Do đó, bài toán đặt ra là làm thế nào để tự động hoá quá trình này mà vẫn đảm bảo chất lượng cao và bảo toàn được đặc điểm nhận dạng của đối tượng trong ảnh gốc.

Mạng Đối nghịch Tạo sinh (GAN) nổi lên như một công nghệ đột phá có thể giải quyết các nhiệm vụ tổng hợp hình ảnh phức tạp. Được Ian Goodfellow giới thiệu năm 2014 [11] và được Yann LeCun ca ngợi là “ý tưởng thú vị nhất của ML trong 10 năm”, GAN đã chứng tỏ hiệu quả vượt trội trong việc sinh ảnh chân thực từ dữ liệu huấn luyện. Đặc biệt trong bài toán dịch chuyển phong cách (style transfer), GANs cung cấp một khuôn khổ linh hoạt để huấn luyện mô hình tạo ảnh anime từ ảnh thật mà không đòi hỏi dữ liệu ảnh cặp một-một. Trong số các mô hình GAN ứng dụng cho anime hóa ảnh, AnimeGANv3 (DTGAN) [56] nổi bật với kiến trúc Double-Tail Generator sáng tạo, kỹ thuật chuẩn hóa LADE và bộ hàm mất mát chuyên biệt – đạt được đồng thời tốc độ nhanh và chất lượng ảnh cao. Luận văn này nghiên cứu, tái hiện và mở rộng hệ thống đó bằng cách tích hợp thêm module **trích xuất khuôn mặt (Face Extraction)** sử dụng RetinaFace, tạo thành một pipeline hoàn chỉnh từ đầu đến cuối: từ ảnh chân dung thô đến ảnh anime chất lượng cao.

Mục tiêu nghiên cứu

Mục tiêu tổng quát của luận văn là nghiên cứu, tái hiện và mở rộng mô hình AnimeGANv3 (DTGAN) để xây dựng một hệ thống chuyển đổi ảnh chân dung người thật sang phong cách anime chất lượng cao, có tích hợp module trích xuất khuôn mặt tự động. Cụ thể, luận văn hướng đến các mục tiêu sau:

1. Nghiên cứu và trình bày nền tảng lý thuyết về mạng nơ-ron tích chập (CNN), các biến thể GAN ứng dụng trong chuyển đổi phong cách ảnh, cùng với các kỹ thuật phát hiện khuôn mặt và tiền xử lý ảnh liên quan.
2. Phân tích chi tiết kiến trúc AnimeGANv3 (DTGAN): cấu trúc Generator hai nhánh (Double-Tail), kỹ thuật chuẩn hóa LADE, cơ chế External Attention v3 và hệ thống hàm mất mát đặc thù.
3. Xây dựng và tích hợp module trích xuất khuôn mặt (Face Extraction) sử dụng RetinaFace kết hợp thuật toán Mở rộng Vùng mặt Đối xứng (Symmetric Margin Expansion) 6 bước, sau đó ghép kết quả trở lại nền bằng kỹ thuật Feathered Blending (alpha mask + Gaussian blur), tạo thành pipeline hoàn chỉnh: Ảnh đầu vào → Phát hiện khuôn mặt → Cắt & mở rộng vùng mặt → Chuyển đổi anime (DTGAN) → Feathered Blending → Kết quả Hybrid.
4. Huấn luyện và đánh giá toàn diện mô hình trên ba bộ dữ liệu phong cách Hayao Miyazaki, Makoto Shinkai và Ghibli_o1 bằng các chỉ số định lượng (FID, LPIPS, FPS), nghiên cứu người dùng (user study với 42 người tham gia) và đánh giá định tính.
5. Xây dựng ứng dụng web demo (React + FastAPI) triển khai suy luận qua ONNX Runtime để minh chứng khả năng ứng dụng thực tiễn của hệ thống trên thiết bị cấu hình hạn chế.

Đóng góp của luận văn

Luận văn sử dụng mô hình AnimeGANv3 (DTGAN) [56] làm nền tảng đã có sẵn (pretrained backbone) cho bài toán chuyển đổi phong cách ảnh. Trên nền tảng đó, luận văn đề xuất và triển khai ba đóng góp chính mang tính nguyên bản:

1. **Tách khuôn mặt tự động (Face Detection):** Tích hợp mạng RetinaFace với backbone MobileNet 0.25 (~500KB) để tự động phát hiện và tách khuôn mặt từ ảnh đầu vào. Module này cho phép hệ thống xử lý khuôn mặt ở độ phân giải 256×256 thay vì phụ thuộc vào kích thước khuôn mặt trong ảnh gốc, giải quyết hạn chế về bảo toàn chi tiết khuôn mặt của DTGAN khi khuôn mặt chiếm tỉ lệ nhỏ.

2. **Xử lý riêng khuôn mặt (Face Processing):** Thiết kế thuật toán mở rộng vùng mặt (Margin Expansion) 6 bước, tận dụng 5 điểm đặc trưng khuôn mặt (2 mắt, mũi, 2 đầu môi) từ RetinaFace để xác định và mở rộng vùng cắt khuôn mặt một cách thông minh, đảm bảo bao gồm đủ bối cảnh (tóc, tai, cổ) trước khi đưa vào mô hình chuyển đổi.
3. **Ghép và vá vùng biên (Face Blending):** Phát triển kỹ thuật feathered blending sử dụng alpha mask kết hợp Gaussian blur để ghép khuôn mặt đã chuyển đổi anime trở lại nền (Hybrid Mode), đảm bảo đồng bộ về ánh sáng, đường nét và màu sắc giữa vùng khuôn mặt và background. Chế độ Hybrid Mode kết hợp cả hai ưu điểm: nền được anime hóa bằng Landscape Mode trong khi từng khuôn mặt được xử lý riêng ở chất lượng cao.

Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: Kiến trúc mạng Double-Tail Generative Adversarial Network (DTGAN) áp dụng cho bài toán chuyển đổi phong cách ảnh chân dung sang anime. Cụ thể, đề tài tập trung vào: (1) các thành phần cấu trúc của Generator và Discriminator trong AnimeGANv3, (2) kỹ thuật chuẩn hóa LADE và cơ chế External Attention v3, (3) hệ thống hàm mất mát đặc thù cho bài toán anime hóa, và (4) module phát hiện – trích xuất khuôn mặt sử dụng RetinaFace.

Phạm vi nghiên cứu:

- *Dữ liệu:* Ảnh chân dung người thật (portrait photos) làm đầu vào; ba bộ dữ liệu anime làm tập phong cách mục tiêu: **Hayao Miyazaki** (1.792 ảnh, không ghép cặp), **Makoto Shinkai** (1.650 ảnh, không ghép cặp) và **Ghibli_o1** (4.991 cặp ảnh thực-anime, ghép cặp có giám sát). Dữ liệu nguồn gồm tập *train_photo* (6.656 ảnh) dùng chung cho Hayao và Shinkai, và tập *train/images* của Ghibli_o1 cho bộ dữ liệu chân dung có giám sát.
- *Mô hình:* Sử dụng mô hình AnimeGANv3 (DTGAN) [56] làm nền tảng, có mở rộng thêm module Face Extraction dựa trên RetinaFace [42] với backbone MobileNet 0.25.
- *Triển khai:* Mô hình được chuyển đổi và triển khai suy luận qua ONNX Runtime, hỗ trợ cả ảnh đơn và xử lý theo lô (batch processing).
- *Ngoài phạm vi:* Chuyển đổi phong cách cho video thời gian thực và các phong cách hoạt hình khác (cartoon phương Tây, 3D). Mặc dù các kiến trúc mới hơn như mô hình khuếch tán (Diffusion Models) đạt chất lượng sinh ảnh ấn tượng, chúng yêu cầu dung lượng mô hình lớn gấp hàng trăm lần và tốc độ chậm hơn 20–80 lần

so với GAN, không phù hợp cho mục tiêu triển khai thời gian thực trên thiết bị cấu hình hạn chế của luận văn (phân tích chi tiết tại Mục 1.5).

Cấu trúc luận văn

Luận văn được tổ chức theo cấu trúc như sau:

- **Mở đầu:** Trình bày lý do chọn đề tài, sự cần thiết, mục tiêu, đối tượng, phạm vi nghiên cứu và giới thiệu cấu trúc của luận văn.
- **Chương 1: Cơ sở lý thuyết và tổng quan nghiên cứu.** Giới thiệu bài toán chuyển ảnh sang phong cách anime, nền tảng mạng nơ-ron tích chập (CNN), lý thuyết mạng đối nghịch tạo sinh (GAN) và các biến thể (WGAN, LSGAN, CycleGAN), các mô hình GAN tiêu biểu cho bài toán anime, và sơ bộ về phương pháp phát hiện khuôn mặt RetinaFace.
- **Chương 2: Kiến trúc mô hình AnimeGANv3 (DTGAN).** Phân tích chi tiết kiến trúc mạng Double-Tail GAN: Base Encoder, Support Tail và Main Tail của Generator; hai Discriminator; kỹ thuật chuẩn hóa LADE; cơ chế External Attention v3; hệ thống hàm mất mát; pipeline tiền xử lý dữ liệu huấn luyện và quy trình huấn luyện.
- **Chương 3: Luồng anime tích hợp phát hiện khuôn mặt.** Trình bày pipeline end-to-end tích hợp phát hiện khuôn mặt (RetinaFace) và chuyển đổi phong cách anime (AnimeGANv3), kỹ thuật mở rộng vùng mặt (Margin Expansion), và phân tích hai chế độ xử lý Face Mode/Landscape Mode.
- **Chương 4: Thử nghiệm và đánh giá.** Trình bày môi trường thực nghiệm, bộ dữ liệu, kết quả huấn luyện, đánh giá định lượng (FID, LPIPS, FPS) và định tính, triển khai ứng dụng web (React + FastAPI) và demo hệ thống.
- **Kết luận:** Tóm tắt những kết quả nổi bật, đóng góp của đề tài, thảo luận các hạn chế còn tồn tại và đề xuất phương hướng mở rộng trong tương lai.

CHƯƠNG 1

CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN NGHIÊN CỨU

1.1. Bài toán Chuyển ảnh sang Phong cách Anime

1.1.1. Định nghĩa bài toán

Bài toán chuyển ảnh sang phong cách anime thuộc lớp bài toán *dịch ảnh giữa các miền* (image-to-image translation): cho một ảnh chụp thực tế $x \in \mathcal{X}$ thuộc miền ảnh thật, tìm ánh xạ $G : \mathcal{X} \rightarrow \mathcal{Y}$ sao cho ảnh đầu ra $G(x) \in \mathcal{Y}$ mang phong cách anime nhưng vẫn bảo toàn nội dung ngữ nghĩa của ảnh gốc [25, 29]. Đặc thù của bài toán này là dữ liệu huấn luyện ở dạng **không ghép cặp** (unpaired): ta chỉ có tập ảnh thật $\{x_i\}$ và tập ảnh anime $\{y_j\}$ độc lập, không có sự tương ứng một-một giữa hai tập [29].

1.1.2. Đặc trưng thị giác của phong cách Anime

Phong cách anime có các đặc trưng thị giác khác biệt rõ ràng so với ảnh chụp thực tế, bao gồm [40, 56]:

- **Đường nét viền rõ ràng:** Các đối tượng được phân tách bởi các đường viền đen hoặc tối, tạo hiệu ứng “ink outline” đặc trưng.
- **Vùng màu phẳng (cel-shading):** Thay vì gradient mịn như ảnh thật, anime sử dụng các vùng màu đồng nhất với ranh giới rõ ràng giữa vùng sáng và vùng tối.
- **Đơn giản hóa chi tiết:** Kết cấu bề mặt (texture) như da, tóc, vải được tối giản hoá, loại bỏ phần lớn chi tiết thực tế.
- **Phân phối màu sắc đặc thù:** Màu sắc anime thường tươi sáng hơn, bão hoà hơn và có phân phối brightness khác biệt đáng kể so với ảnh chụp.

1.1.3. Thách thức kỹ thuật

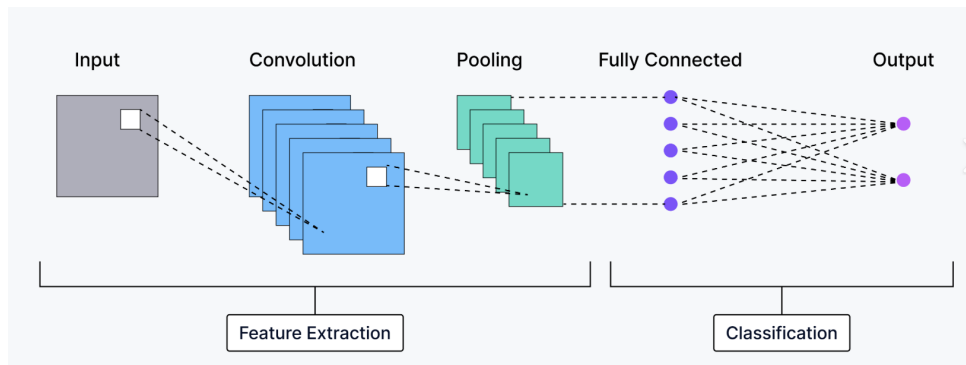
Việc chuyển đổi tự động ảnh thật sang anime đặt ra ba thách thức chính:

1. **Cân bằng nội dung và phong cách:** Mô hình phải đồng thời bảo toàn cấu trúc ngữ nghĩa của ảnh gốc (bố cục, hình dáng đối tượng, danh tính khuôn mặt) và chuyển đổi phong cách thị giác sang anime. Hai mục tiêu này thường mâu thuẫn: biến đổi phong cách quá mạnh sẽ phá hủy nội dung, trong khi bảo toàn quá chặt sẽ không đạt được phong cách anime đích [15, 29].
2. **Dữ liệu không ghép cặp:** Không thể thu thập bộ dữ liệu gồm các cặp (ảnh thật, ảnh anime tương ứng) vì mỗi ảnh anime được vẽ theo phong cách chủ quan của họa sĩ. Do đó, mô hình phải học ánh xạ giữa hai miền từ dữ liệu unpaired [29, 27].

3. **Bảo toàn danh tính khuôn mặt:** Khi xử lý ảnh chân dung, các đặc điểm nhận dạng (ánh mắt, nụ cười, tỉ lệ khuôn mặt) cần được giữ nguyên trong phiên bản anime. Nhiều mô hình hiện tại gặp khó khăn với khuôn mặt có tỉ lệ nhỏ trong ảnh hoặc bị biến dạng chi tiết khuôn mặt [44, 56].

1.2. Nền tảng Mạng Nơ-ron Tích chập

Mạng Nơ-ron Tích chập (Convolutional Neural Networks – CNNs) là kiến trúc nền tảng trong thị giác máy tính, có khả năng tự động học các biểu diễn đặc trưng phân cấp từ dữ liệu ảnh thô [39]. Mục này tóm tắt các thành phần CNN thiết yếu cho việc hiểu kiến trúc Generator và Discriminator trong các mô hình GAN ở các chương sau.



Hình 1.1. Kiến trúc cơ bản của Mạng Nơ-ron Tích chập

1.2.1. Lớp Tích chập, Pooling và Hàm kích hoạt

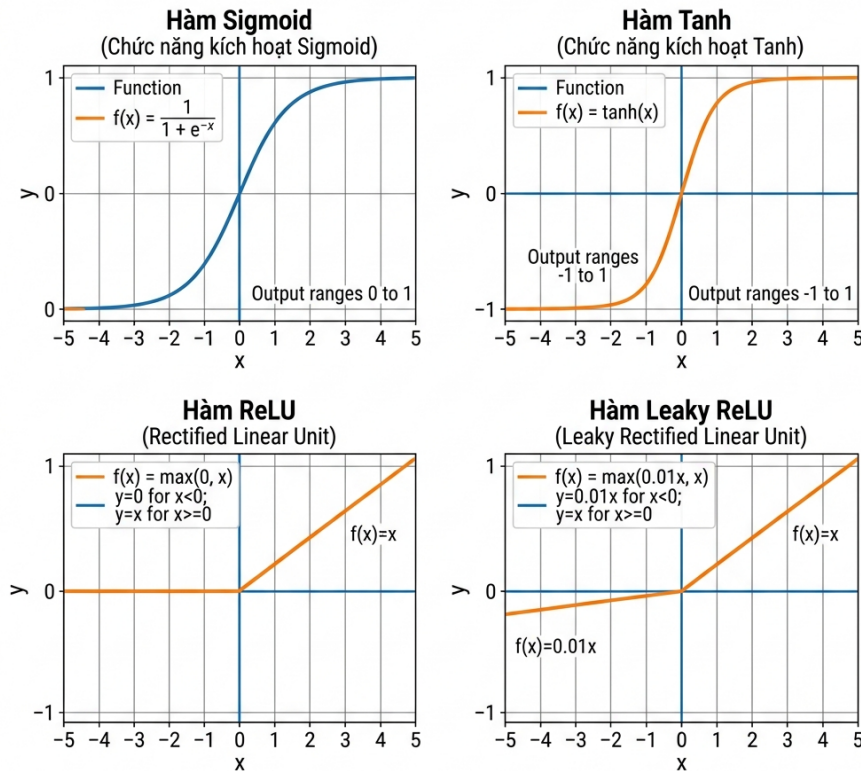
Lớp tích chập sử dụng các kernel nhỏ (thường 3×3) trượt trên ảnh đầu vào, thực hiện phép nhân tích chập cục bộ để trích xuất các đặc trưng hình học như cạnh, đường cong và kết cấu. Mỗi kernel được học tự động trong quá trình huấn luyện, thay thế hoàn toàn các đặc trưng thủ công (SIFT, HOG) của thị giác máy tính truyền thống [39, 10].

Lớp Pooling (thường là Max Pooling) giảm độ phân giải không gian của bản đồ đặc trưng, giúp giảm chi phí tính toán và tạo tính bất biến dịch chuyển cục bộ. Tuy nhiên, pooling gây mất thông tin vị trí chính xác – một hạn chế cần được xử lý trong các tác vụ yêu cầu độ chính xác pixel như sinh ảnh [39].

Hàm kích hoạt đưa tính phi tuyến vào mạng. Trong các kiến trúc GAN hiện đại, hai hàm được sử dụng phổ biến nhất là [6, 9]:

- **ReLU** ($\max(0, x)$): Tiêu chuẩn cho hầu hết CNN, giảm vanishing gradient nhưng có thể gây “Dying ReLU” khi nơ-ron nhận giá trị âm liên tục.
- **Leaky ReLU** ($\max(\alpha x, x)$ với $\alpha = 0.01$): Khắc phục Dying ReLU bằng cách giữ gradient nhỏ cho giá trị âm. Được sử dụng rộng rãi trong Generator và Discriminator của AnimeGANv3 [56].

CÁC CHỨC NĂNG KÍCH HOẠT MẠNG NEURAL PHỔ BIẾN

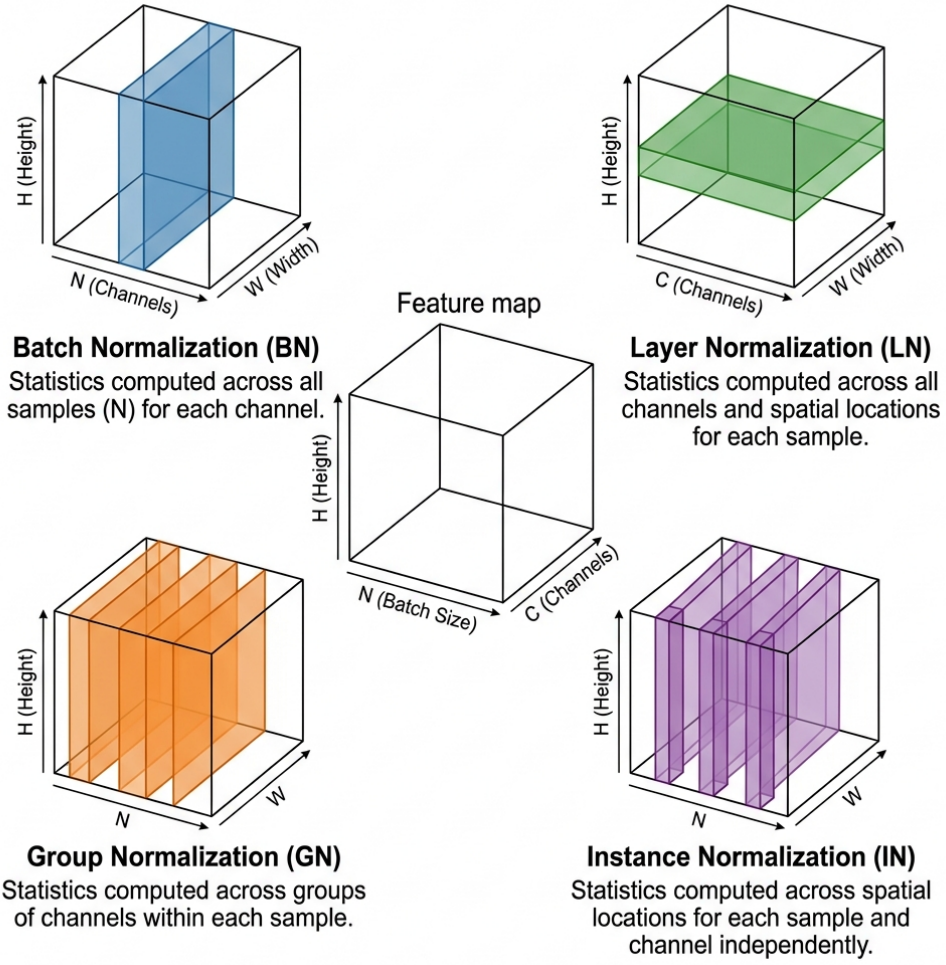


Hình 1.2. So sánh các hàm kích hoạt: Sigmoid, Tanh, ReLU và Leaky ReLU

1.2.2. Các Kỹ thuật Chuẩn hóa

Chuẩn hóa (Normalization) là cơ chế thiết yếu để ổn định và tăng tốc quá trình huấn luyện mạng sâu. Bốn kỹ thuật chính được phân biệt theo chiều tính toán thống kê trên tensor đặc trưng ($N \times C \times H \times W$):

- **Batch Normalization (BN)** [12]: Chuẩn hóa theo chiều batch N , giảm Internal Covariate Shift. Nhạy cảm với kích thước batch nhỏ.
- **Layer Normalization (LN)** [14]: Chuẩn hóa toàn bộ kênh C của một mẫu. Phổ biến trong Transformers và RNNs.
- **Group Normalization (GN)** [33]: Chia kênh thành nhóm nhỏ, cân bằng giữa BN và LN. Không phụ thuộc batch size.
- **Instance Normalization (IN)** [19]: Chuẩn hóa từng kênh của từng mẫu độc lập. **Đặc biệt quan trọng** cho bài toán chuyển phong cách vì IN loại bỏ thông tin phong cách toàn cục, cho phép mô hình học phong cách mới. IN là nền tảng trực tiếp của kỹ thuật LADE trong AnimeGANv3 (trình bày ở Chương 2).



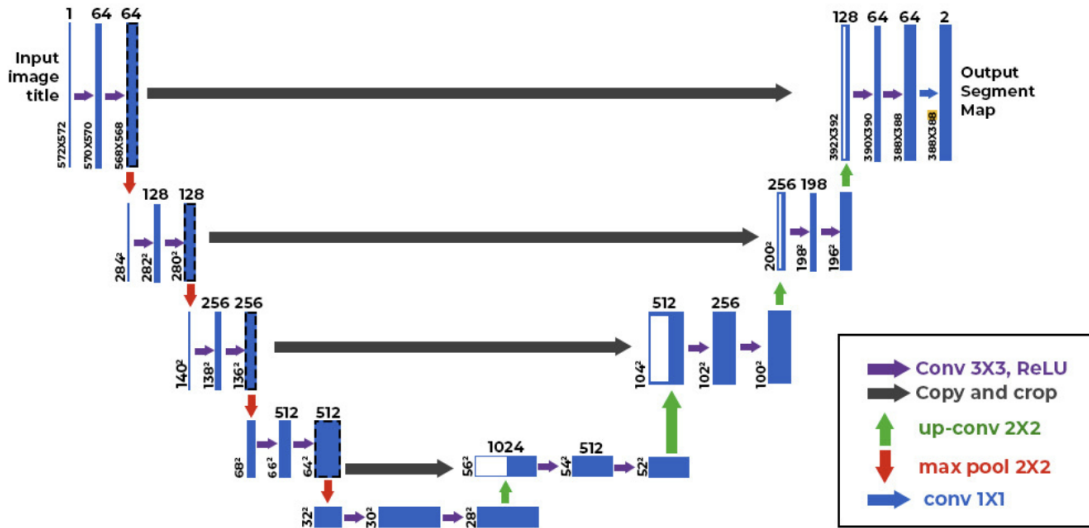
Hình 1.3. So sánh bốn kỹ thuật chuẩn hóa trên tensor ($N \times C \times H \times W$)

Chú thích: BN chuẩn hóa theo chiều batch N ; LN theo toàn bộ kênh C của một mẫu; GN theo nhóm kênh; IN theo từng kênh của từng mẫu độc lập [33, 19]

Ngoài ra, **Spectral Normalization (SN)** [32] chuẩn hóa ma trận trọng số W bằng cách chia cho giá trị kỳ dị lớn nhất $\sigma(W)$, đảm bảo ràng buộc Lipschitz cho Discriminator. SN đã trở thành tiêu chuẩn trong hầu hết kiến trúc GAN hiện đại [38, 30].

1.2.3. Kiến trúc Encoder-Decoder

Kiến trúc Encoder-Decoder là nền tảng cho các bài toán dịch ảnh (image-to-image translation). Phần **Encoder** sử dụng tích chập và downsampling để nén ảnh gốc thành vector đặc trưng ẩn, trong khi phần **Decoder** tái tạo vector ẩn thành ảnh đầu ra mong muốn [35].



Hình 1.4. Kiến trúc U-Net với skip connections

Biến thể U-Net bổ sung **skip connections** kết nối trực tiếp giữa các tầng encoder và decoder cùng độ phân giải. Cơ chế này cho phép decoder truy cập đặc trưng chi tiết cục bộ từ encoder, khắc phục hiện tượng mất thông tin không gian qua bottleneck [35]. Generator trong AnimeGANv3 sử dụng kiến trúc U-Net cho cả hai nhánh decoder (Chương 2).

Trong phần decoder, việc tăng độ phân giải (upsampling) có hai lựa chọn chính:

- **Transposed Convolution:** Lớp học được, nhưng dễ gây lỗi đồ họa dạng checkerboard do chồng lớp không đều [35].
- **Bilinear/Nearest-Neighbor Interpolation + Conv:** Không học được phần upsampling, nhưng loại bỏ lỗi đồ họa dạng checkerboard. AnimeGANv3 chọn phương pháp này để đảm bảo ảnh đầu ra mịn [56].

1.3. Kiến trúc Mạng Đối nghịch Tạo sinh

Mạng Đối nghịch Tạo sinh (Generative Adversarial Networks – GANs) do Ian Goodfellow và cộng sự đề xuất năm 2014 [11] là nền tảng cho hầu hết các phương pháp chuyển đổi phong cách ảnh hiện đại. Khác với các mô hình tạo sinh trước đây dựa trên phương pháp xác suất phức tạp, GAN sử dụng trò chơi đối kháng giữa hai mạng nơ-ron để tạo ra dữ liệu có độ trung thực cao.

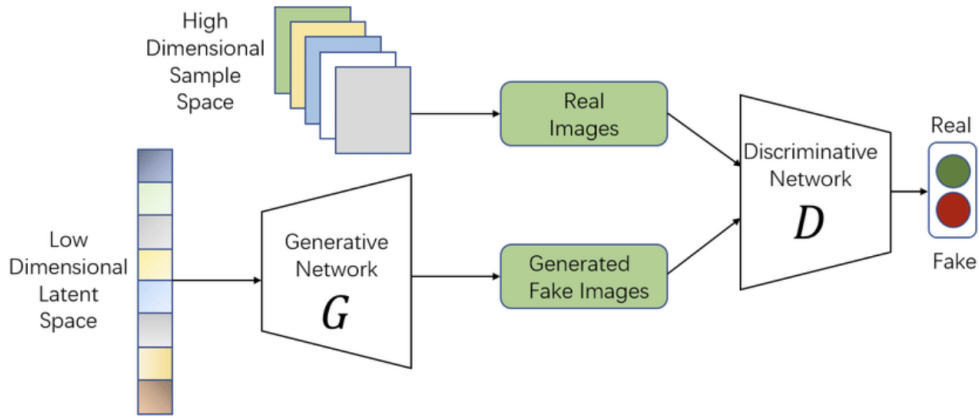
1.3.1. Nguyên lý Hoạt động và Hàm Mục tiêu

1.3.1.1. Khung Làm việc Đối kháng

GAN gồm hai mạng nơ-ron được huấn luyện đồng thời qua trò chơi có tổng bằng không [11]:

- **Generator (G):** Nhận vector nhiễu ngẫu nhiên $z \sim p_z(z)$ từ không gian tiềm ẩn và tạo ra mẫu giả $G(z)$. Mục tiêu: đánh lừa Discriminator.

- **Discriminator (D):** Bộ phân loại nhị phân, nhận dữ liệu x và xuất xác suất $D(x)$ dữ liệu đó là thật. Mục tiêu: phân biệt dữ liệu thật và giả.



Hình 1.5. Kiến trúc tổng quát của Mạng Đối nghịch Tạo sinh (GAN)

Chú thích: Generator G nhận vector nhiễu $z \sim p_z(z)$ để tạo mẫu giả $G(z)$, Discriminator D phân biệt dữ liệu thật x với dữ liệu giả. Hai mạng huấn luyện đồng thời theo trò chơi minimax [11]

1.3.1.2. Trò chơi Minimax

Quá trình huấn luyện được mô hình hóa dưới dạng bài toán tối ưu Minimax [11]:

$$\min_G \max_D V(D, G) = \underbrace{\mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)]}_{(1) \text{ Discriminator nhận dạng ảnh thật}} + \underbrace{\mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]}_{(2) \text{ Discriminator bác bỏ ảnh giả}} \quad (1.1)$$

trong đó:

- $G : \mathcal{Z} \rightarrow \mathcal{X}$ – Generator ánh xạ vector nhiễu z từ không gian tiềm ẩn $\mathcal{Z}$ sang không gian ảnh $\mathcal{X}$.
- $D : \mathcal{X} \rightarrow [0, 1]$ – Discriminator xuất xác suất mẫu đầu vào là ảnh thật.
- $p_{\text{data}}(x)$ – phân phối dữ liệu thật; $p_z(z)$ – phân phối nhiễu đầu vào (thường là $\mathcal{N}(0, I)$).
- $\mathbb{E}[\cdot]$ – kỳ vọng thống kê trên phân phối tương ứng.

Ý nghĩa từng số hạng:

- **Số hạng (1)** $\mathbb{E}[\log D(x)]$: Với mẫu thật $x \sim p_{\text{data}}$, $D(x)$ càng gần 1 thì $\log D(x)$ càng gần 0 (lớn nhất). Discriminator muốn *tối đa hóa* số hạng này – tức là gán xác suất cao cho ảnh thật.

- **Số hạng (2)** $\mathbb{E}[\log(1 - D(G(z)))]$: Với mẫu giả $G(z)$, nếu Discriminator nhận ra giả thì $D(G(z)) \approx 0$, khiến $\log(1 - D(G(z))) \approx 0$ (lớn nhất). Generator muốn *tối thiểu hóa* số hạng này – tức là đánh lừa D bằng cách làm $D(G(z)) \rightarrow 1$.

1.3.1.3. Điểm Cân bằng Nash và Discriminator Tối ưu

Mục tiêu cuối cùng là đạt *Điểm cân bằng Nash*, nơi D không thể phân biệt ảnh thật và giả ($D(x) = 0.5$ cho mọi x), và phân phối sinh p_g trùng khớp với p_{data} [47]. Với G cố định, Discriminator tối ưu theo $V(D, G)$ là:

$$D_G^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \quad (1.2)$$

trong đó $p_g(x)$ là phân phối của mẫu giả $G(z)$ khi $z \sim p_z$. Công thức này có thể giải tích bằng cách lấy đạo hàm $\frac{\partial V}{\partial D(x)} = 0$: tại mỗi điểm x , D^* cân bằng tỷ lệ giữa mật độ thật và tổng mật độ. Khi $p_g = p_{\text{data}}$, ta có $D^*(x) = \frac{1}{2}$ – Discriminator không thể làm tốt hơn đoán ngẫu nhiên.

Thay D_G^* vào Phương trình (1.1), hàm mục tiêu của Generator rút gọn thành:

$$C(G) = -\log 4 + 2 \cdot D_{\text{JS}}(p_{\text{data}} \| p_g) \quad (1.3)$$

trong đó $D_{\text{JS}}(p \| q) = \frac{1}{2} D_{\text{KL}}\left(p \parallel \frac{p+q}{2}\right) + \frac{1}{2} D_{\text{KL}}\left(q \parallel \frac{p+q}{2}\right)$ là khoảng cách Jensen-Shannon – phiên bản đối xứng của phân kỳ KL. Vì $D_{\text{JS}} \geq 0$ và bằng 0 khi và chỉ khi $p_{\text{data}} = p_g$, hàm $C(G)$ đạt cực tiểu toàn cục tại $C^* = -\log 4$ đúng khi Generator tái tạo hoàn hảo phân phối dữ liệu thật.

1.3.2. Thách thức trong Huấn luyện GAN

Mặc dù lý thuyết Minimax chặt chẽ, GAN trong thực tế gặp ba vấn đề nghiêm trọng [18, 23]. Các vấn đề này bắt nguồn từ sự không tương thích giữa giả định toán học (phân phối liên tục, tập dữ liệu vô hạn) và thực tế huấn luyện (mini-batch hữu hạn, không gian ảnh chiều cao).

1.3.2.1. Biến mất Gradient (Vanishing Gradients)

Trong không gian ảnh chiều cao, p_{data} và p_g thường tập trung trên các đa tạp (manifold) chiều thấp không giao nhau. Khi đó, tồn tại siêu phẳng hoàn toàn tách hai phân phối và Discriminator tối ưu đạt độ chính xác 100% [21].

Phân tích toán học: Khi p_{data} và p_g tách rời hoàn toàn (disjoint support), Discriminator tối ưu $D^*(x) = 1$ với $x \sim p_{\text{data}}$ và $D^*(x) = 0$ với $x \sim p_g$. Thay vào hàm $C(G)$ (Phương trình 1.3):

$$D_{\text{JS}}(p_{\text{data}} \| p_g) = \log 2 \quad \Rightarrow \quad C(G) = -\log 4 + 2 \log 2 = 0 \quad (1.4)$$

Gradient $\nabla_{\theta} C(G)$ bằng 0 với mọi tham số θ của Generator – mạng hoàn toàn không nhận được tín hiệu học.

Biểu hiện thực tế: Đường cong G_loss của Generator phẳng lặng trong khi D_loss giảm về 0. Ảnh sinh ra không cải thiện theo epoch dù loss Discriminator tiếp tục tối ưu hóa.

1.3.2.2. Suy đổ Mode (Mode Collapse)

Mode Collapse xảy ra khi Generator tìm được một hoặc vài “chiến lược an toàn” – tức là một tập mẫu nhỏ có thể đánh lừa Discriminator hiện tại – và cố định vào đó thay vì học toàn bộ phân phối p_{data} .

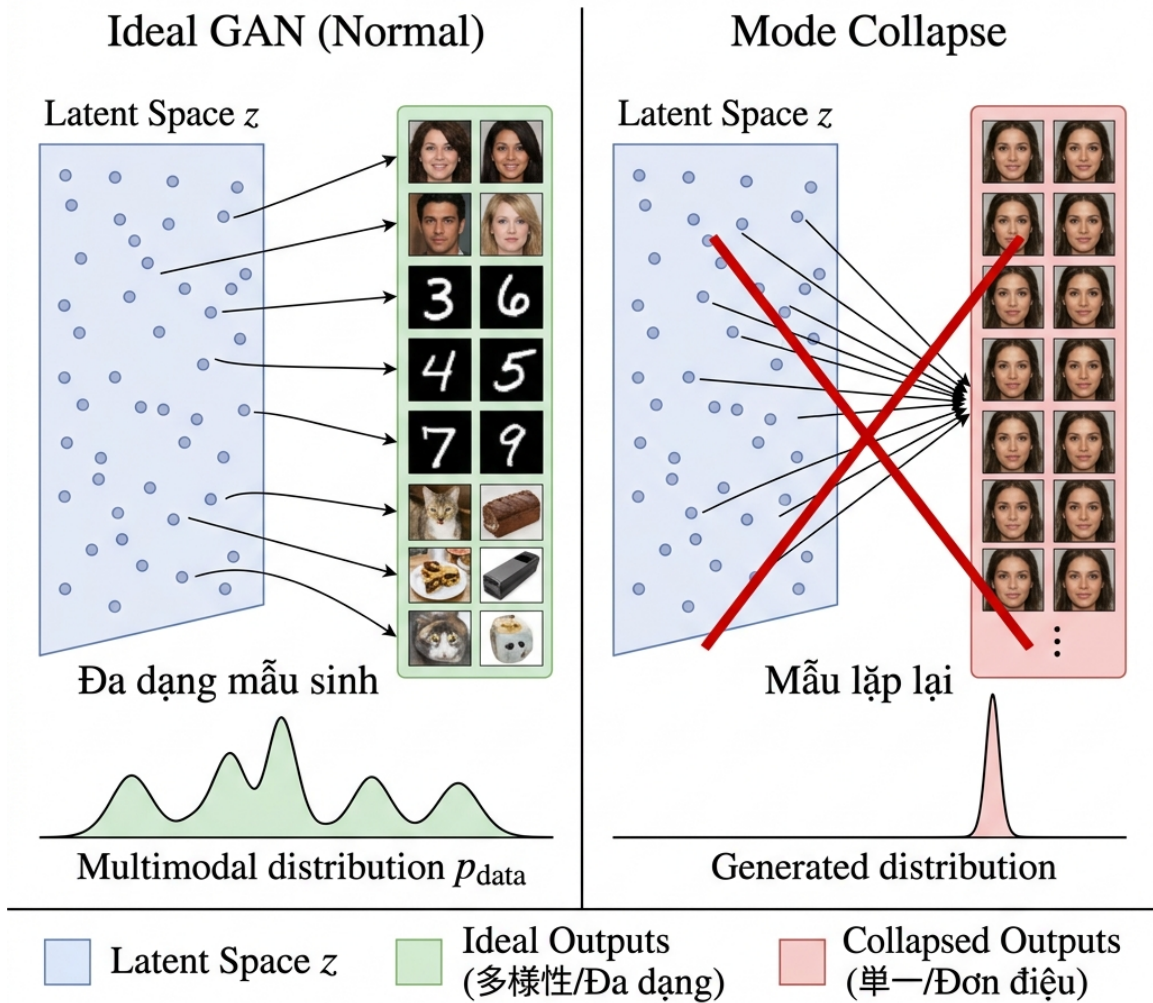
Cơ chế game-theoretic: Quá trình huấn luyện GAN là trò chơi hai người với chiến lược thay đổi theo thời gian. Khi Generator “chuyên môn hóa” vào mode m^* :

$$G_t(z) \approx m^* \quad \forall z \quad \Rightarrow \quad D_{t+1} \text{ học phân biệt } m^* \quad \Rightarrow \quad G_{t+2} \text{ chuyển sang mode khác } m^{**} \quad (1.5)$$

Hai mạng đuổi bắt nhau theo vòng tròn: Generator liên tục “bỏ trốn” sang mode mới, Discriminator liên tục “học đuổi”, nhưng không bao giờ đạt cân bằng Nash toàn cục.

Hậu quả định lượng: Với bộ dữ liệu có K mode (ví dụ: K phong cách anime), Generator bị collapse thường chỉ bao phủ 1–3 mode, dẫn đến FID cao do phân phối sinh thiếu đa dạng [18].

“Mode Collapse”



Hình 1.6. Minh họa hiện tượng Mode Collapse trong GAN

Chú thích: (Trái) GAN lý tưởng: không gian z đa dạng ánh xạ sang nhiều mode của p_{data} . (Phải) Mode Collapse: toàn bộ không gian z bị ánh xạ vào một hoặc vài mode duy nhất, phần còn lại của phân phối không được học [18].

1.3.2.3. Không Hội tụ (Non-Convergence)

GAN yêu cầu tìm *điểm yên ngựa* (saddle point) (θ_G^*, θ_D^*) thỏa mãn $\min_G \max_D V(D, G)$, không phải cực tiểu thông thường. Tuy nhiên, gradient descent tiêu chuẩn được thiết kế để tìm cực tiểu cục bộ – đây là sự không tương thích cơ bản [23].

Ví dụ minh họa (Bilinear Game): Xét trò chơi đơn giản nhất với $V(x, y) = xy$, tương tự bài toán GAN:

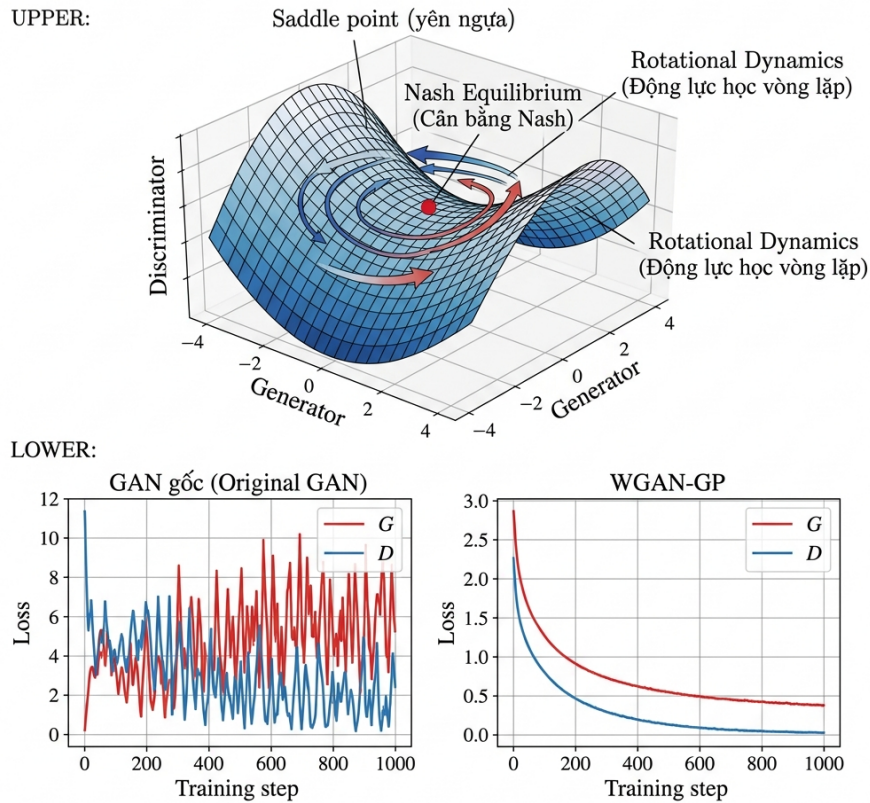
$$\dot{x} = -\frac{\partial V}{\partial x} = -y, \quad \dot{y} = +\frac{\partial V}{\partial y} = x \quad (1.6)$$

Nghiệm của hệ này là $x(t) = r \cos(t + \phi)$, $y(t) = r \sin(t + \phi)$ – quỹ đạo xoay tròn vĩnh

viễn với bán kính r không đổi, không bao giờ hội tụ về điểm cân bằng $(0, 0)$.

Biểu hiện thực tế: G_loss và D_loss dao động có chu kỳ thay vì giảm đơn điệu. Trong GAN huấn luyện ổn định, dao động nhỏ là bình thường; dao động lớn theo chu kỳ ($> 20\%$ biên độ tương đối) là dấu hiệu non-convergence nghiêm trọng.

GAN Training Challenges — Nash Equilibrium and Non-Convergence



Hình 1.7. Động lực học huấn luyện GAN: quỹ đạo xoay tròn trong không gian tham số và vấn đề Không Hội tụ

Chú thích: Mũi tên thể hiện hướng cập nhật gradient của (θ_G, θ_D) tại mỗi bước. Thay vì hội tụ vào tâm (điểm cân bằng Nash), quỹ đạo xoay theo đường tròn – đặc trưng của saddle-point dynamics.

Bảng 1.1. Tóm tắt các chế độ thất bại của GAN và giải pháp tương ứng

Vấn đề	Biểu hiện	Nguyên nhân Toán học	Giải pháp Tiêu biểu
Vanishing Gradients	G ngừng học, G_loss phẳng	D^* tách hoàn toàn hai phân phối, $D_{JS} = \log 2$, gradient = 0	WGAN [21] dùng khoảng cách Wasserstein

Bảng 1.1. Tóm tắt các chế độ thất bại của GAN và giải pháp tương ứng

Vấn đề	Biểu hiện	Nguyên nhân Toán học	Giải pháp Tiêu biểu
Mode Collapse	Ảnh sinh ra giống hệt nhau, FID cao	G tối ưu hóa cục bộ, đui bắt vòng tròn qua các mode	Minibatch discrimination, Unrolled GAN [18]
Non-Convergence	Loss dao động mạnh theo chu kỳ	Saddle-point dynamics, gradient descent không hội tụ	WGAN-GP [22], Spectral Norm [32]

1.3.3. Các Giải pháp về Hàm Mục tiêu

1.3.3.1. Wasserstein GAN (WGAN)

Arjovsky et al. [21] phân tích nguyên nhân gốc rễ của Vanishing Gradients: khoảng cách Jensen-Shannon không liên tục khi hai phân phối không giao nhau. Giải pháp là thay thế bằng *khoảng cách Wasserstein-1* (Earth-Mover distance):

$$W(p_{\text{data}}, p_g) = \inf_{\gamma \in \Pi(p_{\text{data}}, p_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|] \quad (1.7)$$

trong đó $\Pi(p_{\text{data}}, p_g)$ là tập tất cả phân phối ghép cặp (joint distributions) với marginal p_{data} và p_g . Trực quan, W đo lượng “đất” tối thiểu cần di chuyển để biến p_g thành p_{data} . Không giống D_{JS} , W vẫn liên tục ngay cả khi hai phân phối hoàn toàn không giao nhau.

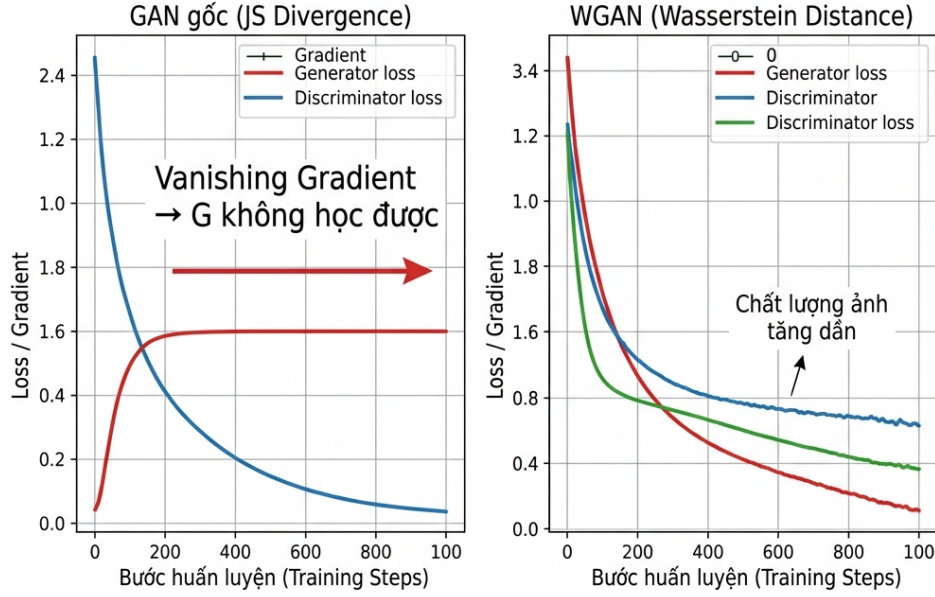
Dạng đối ngẫu Kantorovich–Rubinstein: Vì Phương trình (1.7) khó tính trực tiếp, WGAN sử dụng dạng tương đương:

$$W(p_{\text{data}}, p_g) = \frac{1}{K} \sup_{\|f\|_L \leq K} (\mathbb{E}_{x \sim p_{\text{data}}} [f(x)] - \mathbb{E}_{\tilde{x} \sim p_g} [f(\tilde{x})]) \quad (1.8)$$

trong đó sup lấy trên tất cả hàm K -Lipschitz $f : \mathcal{X} \rightarrow \mathbb{R}$. Discriminator D_w đóng vai trò xấp xỉ hàm f này thay vì phân loại nhị phân, và hàm mục tiêu WGAN là:

$$\min_G \max_{D_w \in \mathcal{F}_1} \mathbb{E}_{x \sim p_{\text{data}}} [D_w(x)] - \mathbb{E}_{z \sim p_z} [D_w(G(z))] \quad (1.9)$$

với $\mathcal{F}_1$ là lớp hàm 1-Lipschitz. Ràng buộc Lipschitz là điều kiện bắt buộc để đảm bảo supremum hữu hạn và gradient không bùng nổ. Trong bản gốc WGAN, ràng buộc này được áp đặt bằng *Weight Clipping*: sau mỗi bước cập nhật, trọng số w bị clamp về $[-c, c]$ (thường $c = 0.01$).



Hình 1.8. So sánh gradient flow giữa GAN gốc (JS divergence) và WGAN (Wasserstein distance) khi hai phân phối không giao nhau

Chú thích: (Trái) GAN gốc: khi p_g không giao với p_{data} , $D_{\text{JS}} = \log 2$ hằng số, gradient Generator bằng 0. (Phải) WGAN: khoảng cách Wasserstein vẫn tuyến tính theo khoảng cách giữa hai phân phối, cung cấp gradient có hướng rõ ràng [21].

1.3.3.2. WGAN-GP: Gradient Penalty

Weight Clipping trong WGAN gốc gây ra hai vấn đề: (1) mô hình bị “underfit” do không gian hàm bị giới hạn quá chặt, và (2) gradient bùng nổ hoặc biến mất do trọng số tập trung về $\pm c$. Gulrajani et al. [22] giải quyết bằng cách thay thế bằng *Gradient Penalty* – phạt trực tiếp trên norm gradient:

$$L_{\text{WGAN-GP}} = \underbrace{\mathbb{E}_{\tilde{x} \sim p_g}[D(\tilde{x})] - \mathbb{E}_{x \sim p_{\text{data}}}[D(x)]}_{(1) \text{ WGAN critic loss}} + \underbrace{\lambda \mathbb{E}_{\hat{x} \sim p_{\hat{x}}} \left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \right]}_{(2) \text{ Gradient Penalty}} \quad (1.10)$$

trong đó:

- $\tilde{x} = G(z)$ – mẫu giả từ Generator; x – mẫu thật từ p_{data} .

- $\hat{x} = \epsilon x + (1 - \epsilon)\tilde{x}$, $\epsilon \sim \mathcal{U}[0, 1]$ – điểm nội suy ngẫu nhiên giữa mẫu thật và giả, được lấy mẫu dọc theo đường thẳng nối hai phân phối.
- $\lambda = 10$ – hệ số cân bằng giữa critic loss và penalty (giá trị mặc định theo [22]).
- Số hạng (2): phạt khi $\|\nabla_{\hat{x}} D(\hat{x})\|_2 \neq 1$, ép Discriminator thỏa mãn điều kiện 1-Lipschitz tại các điểm quan trọng nhất (trên “straight-line path” giữa hai phân phối).

Ưu điểm so với Weight Clipping: GP không giới hạn cứng không gian tham số, cho phép Discriminator học các hàm phức tạp hơn. Thực nghiệm cho thấy WGAN-GP hội tụ nhanh hơn và ổn định hơn trên các kiến trúc ResNet sâu [22].

1.3.3.3. Spectral Normalization (SN-GAN)

Miyato et al. [32] đề xuất cách áp đặt ràng buộc Lipschitz nhẹ nhàng hơn bằng cách chuẩn hóa ma trận trọng số theo *spectral norm* (giá trị kỳ dị lớn nhất):

$$W_{\text{SN}} = \frac{W}{\sigma(W)}, \quad \sigma(W) = \max_{\|u\|=1} \|Wu\|_2 \quad (1.11)$$

trong đó $\sigma(W)$ là giá trị kỳ dị lớn nhất (singular value) của W , tính hiệu quả bằng *power iteration* chỉ một bước mỗi lần cập nhật. Sau chuẩn hóa, hằng số Lipschitz của từng lớp tuyến tính bị giới hạn:

$$\|W_{\text{SN}}u\|_2 \leq \|u\|_2 \quad \forall u \quad \Rightarrow \quad \text{Lip}(W_{\text{SN}}) \leq 1 \quad (1.12)$$

Áp dụng cho mọi lớp trong Discriminator, ràng buộc Lipschitz toàn cục được đảm bảo qua quy tắc nhân: $\text{Lip}(D) \leq \prod_l \sigma(W_l) = 1$. **Ưu điểm chính:** chi phí tính toán gần như không đổi (chỉ thêm một lần power iteration), không cần điều chỉnh hyperparameter như λ của GP. Spectral Normalization được sử dụng trực tiếp trong Discriminator của AnimeGANv3 [56] (phân tích chi tiết tại Mục 2.4).

1.3.3.4. Least Squares GAN (LSGAN)

Mao et al. [28] chỉ ra vấn đề khác của cross-entropy: ngay cả khi mẫu giả $G(z)$ nằm ở đúng phía phân loại nhưng còn xa decision boundary, loss đã bão hòa và không cung cấp gradient để đẩy mẫu về phía dữ liệu thật. LSGAN thay thế cross-entropy bằng *bình phương sai số* (least squares):

$$\mathcal{L}_{\text{adv}}^D = \frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}} [(D(x) - 1)^2] + \frac{1}{2} \mathbb{E}_{z \sim p_z} [(D(G(z)))^2] \quad (1.13)$$

$$\mathcal{L}_{\text{adv}}^G = \frac{1}{2} \mathbb{E}_{z \sim p_z} [(D(G(z)) - 1)^2] \quad (1.14)$$

trong đó nhãn thật = 1, nhãn giả = 0. So sánh với cross-entropy:

- **Cross-entropy:** $-\log D(x)$ bão hòa về 0 khi $D(x) \rightarrow 1$ – gradient biến mất với mẫu thật “quá tốt”.
- **Least squares:** $(D(x) - 1)^2$ luôn phạt tỷ lệ với khoảng cách đến nhãn mục tiêu – gradient không biến mất, kể cả khi $D(x)$ đã gần 1.

Tối thiểu hóa $\mathcal{L}_{\text{adv}}^G$ tương đương tối thiểu hóa *Pearson χ^2 divergence* giữa p_{data} và $p_g + p_{\text{data}}$, cung cấp nền tảng lý thuyết vững chắc [28]. Trong AnimeGANv3, LSGAN được áp dụng cho cả hai nhánh Discriminator (Support Tail và Main Tail), kết hợp với Spectral Normalization để đảm bảo ổn định huấn luyện [56].

1.3.4. Đánh giá Chất lượng GAN

1.3.4.1. Inception Score (IS)

IS [18] đánh giá đồng thời hai tiêu chí: *độ sắc nét* (ảnh sinh ra phải thuộc rõ ràng một lớp nào đó) và *đa dạng* (ảnh sinh ra phân phối đều qua nhiều lớp). Chỉ số được tính qua mạng Inception-v3 [Szegedy2016] được huấn luyện trước trên ImageNet:

$$IS = \exp(\mathbb{E}_{x \sim p_g} [D_{KL}(p(y|x) \parallel p(y))]) \quad (1.15)$$

trong đó $p(y|x)$ là phân phối nhãn có điều kiện (entropy thấp $\Leftrightarrow$ ảnh sắc nét) và $p(y) = \mathbb{E}_x[p(y|x)]$ là phân phối nhãn biên (entropy cao $\Leftrightarrow$ đa dạng phong cách). IS cao hơn tốt hơn, nhưng chỉ số này có hai hạn chế quan trọng: (1) không so sánh với phân phối ảnh thật, nên không phát hiện được mode coverage thấp; (2) phụ thuộc vào từ vựng ngữ nghĩa của ImageNet, không phù hợp với ảnh anime vốn rất khác biệt về phân phối.

1.3.4.2. Fréchet Inception Distance (FID)

FID [23] khắc phục hạn chế của IS bằng cách so sánh trực tiếp *phân phối đặc trưng* Inception-v3 của ảnh thật và ảnh sinh ra. Giả định hai phân phối đặc trưng đều Gaussian với tham số (μ_r, Σ_r) cho tập thật và (μ_g, Σ_g) cho tập sinh:

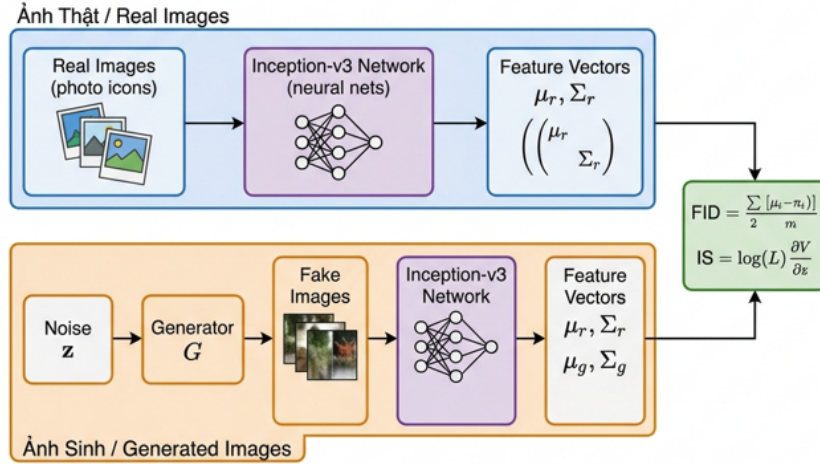
$$FID = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}) \quad (1.16)$$

trong đó số hạng đầu đo khoảng cách giữa hai trung tâm phân phối, số hạng thứ hai (liên quan ma trận hiệp phương sai) đo sự khác biệt về hình dạng phân phối.

Ưu điểm của FID so với IS: FID nhạy cảm hơn với mode collapse (vì khi p_g chỉ bao phủ một vài mode, Σ_g sẽ rất nhỏ so với Σ_r); tương quan tốt hơn với đánh giá thẩm mỹ của người [23]; và đặc biệt, FID không phụ thuộc vào nhãn lớp ImageNet mà dựa trên đặc trưng lớp ẩn – ít bị ảnh hưởng bởi phân phối miền đặc thù hơn IS.

Hạn chế: FID yêu cầu ít nhất khoảng 10.000 mẫu để ước tính đáng tin cậy ma trận hiệp phương sai, và phụ thuộc vào phiên bản Inception-v3 sử dụng. Trong luận văn

này, FID được tính trên 1.000 ảnh chân dung theo chuẩn thực nghiệm của AnimeGANv3 [56], và là **chỉ số định lượng chính** để đánh giá và so sánh hệ thống trong Chương 4.



Hình 1.9. Quy trình tính FID và IS: cả hai đều trích xuất đặc trưng qua mạng Inception-v3, nhưng IS so sánh với phân phối nhân còn FID so sánh với phân phối đặc trưng của ảnh thật

Chú thích: IS (trên): chỉ dùng ảnh sinh p_g , không có tham chiếu ảnh thật. FID (dưới): so sánh trực tiếp thống kê đặc trưng của p_g và p_r – đây là lý do FID phản ánh chất lượng toàn diện hơn [23].

1.4. Các mô hình GAN tiêu biểu cho bài toán Anime

Bài toán chuyển ảnh sang phong cách anime đòi hỏi sự kết hợp giữa bảo toàn nội dung gốc và thể hiện đúng phong cách anime. Mục này phân tích ba hướng tiếp cận chính theo trình tự phát triển lịch sử, từ đó biện luận cho việc lựa chọn AnimeGANv3 làm nền tảng của luận văn.

1.4.1. CycleGAN và dịch ảnh không ghép cặp

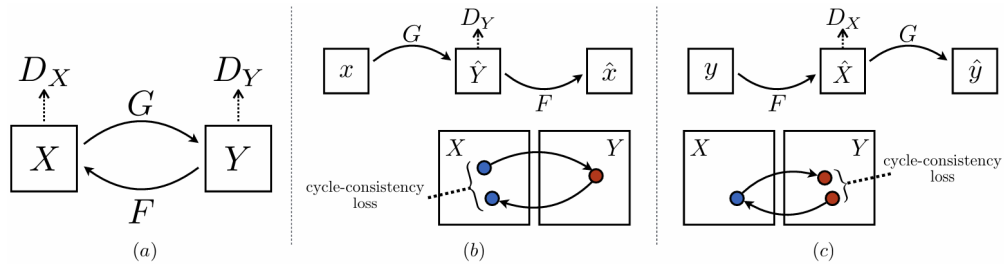
Bối cảnh: Mô hình pix2pix [25] đặt nền tảng cho dịch ảnh có điều kiện nhưng đòi hỏi dữ liệu ghép cặp pixel-chính xác – điều không khả thi với ảnh thật và ảnh anime. CartoonGAN [31] tiến thêm một bước khi đề xuất edge-promoting adversarial loss chuyên biệt cho phong cách hoạt hình, nhưng cũng cần dữ liệu bán ghép cặp. CycleGAN [29] giải quyết triệt để ràng buộc này.

CycleGAN (Zhu et al., 2017) [29] dùng hai Generator song song $G : X \rightarrow Y$ và $F : Y \rightarrow X$, với ràng buộc **nhất quán vòng lặp** (cycle consistency) được thực thi bằng hàm mất mát:

$$\mathcal{L}_{\text{cyc}}(G, F) = \mathbb{E}_{x \sim p_X} [\|F(G(x)) - x\|_1] + \mathbb{E}_{y \sim p_Y} [\|G(F(y)) - y\|_1] \quad (1.17)$$

Ràng buộc này đảm bảo G và F học ánh xạ nhất quán: ảnh thật sau khi chuyển sang

anime rồi chuyển ngược lại phải xấp xỉ ảnh gốc.



Hình 1.10. Kiến trúc CycleGAN: hai Generator G , F và hai Discriminator D_X , D_Y huấn luyện đồng thời với ràng buộc cycle consistency

Chú thích: Mũi tên xanh: forward mapping $G : X \rightarrow Y$; mũi tên cam: reconstruction $F : Y \rightarrow X$. Cycle consistency loss (Phương trình 1.17) ràng buộc $F(G(x)) \approx x$ và $G(F(y)) \approx y$, ngăn Generator học ánh xạ tùy tiện [29].

Ưu điểm: Linh hoạt do không cần dữ liệu ghép cặp; bảo toàn cấu trúc tổng quát; dễ áp dụng cho nhiều cặp miền. **Nhược điểm:** Với 28.3M tham số và FID 86.7 trên Selfie2Anime, CycleGAN gặp khó khăn đặc thù với chuyển đổi sang anime: đường nét anime cần sắc nét và có chủ ý nghệ thuật, nhưng cycle consistency chỉ ràng buộc bảo toàn cấu trúc theo L_1 – không đặc biệt khuyến khích học đường nét mịn và vùng màu phẳng.

Đánh giá thực tiễn: Qua quá trình thử nghiệm, có thể nhận thấy CycleGAN hoạt động tốt với các cặp miền có phân phối **tương tự** (ví dụ: ảnh ngựa $\leftrightarrow$ ảnh ngựa vằn). Tuy nhiên, phong cách anime cách biệt về đặc điểm thị giác so với ảnh thật (mắt to, màu phẳng, đường nét cứng) – khoảng cách phân phối lớn khiến Generator “cố gắng” bảo toàn cycle consistency bằng cách học chuyển đổi “an toàn” thay vì học phong cách đặc trưng. Đây cũng là lý do FID CycleGAN trên bài toán anime (86.7) cao hơn nhiều so với bài toán chuyển đổi style gần gũi hơn.

Biến thể U-GAT-IT [44] bổ sung module chú ý (attention) và Adaptive Layer-Instance Normalization (AdaLIN) để cải thiện, đạt FID 72.1 – tốt hơn CycleGAN nhưng chi phí tính toán tăng gần gấp đôi (29.8M params).

1.4.2. StyleGAN và GAN Inversion

StyleGAN (Karras et al., 2019) [36] đại diện cho triết lý thiết kế khác biệt hoàn toàn: thay vì học ánh xạ $X \rightarrow Y$, StyleGAN tập trung vào *sinh ảnh chất lượng cực cao* bằng kiến trúc Generator điều khiển bởi phong cách. Generator gồm: (1) Mạng ánh xạ $f : \mathcal{Z} \rightarrow \mathcal{W}$ biến đổi vector nhiễu z thành vector phong cách w ; (2) Mạng tổng hợp điều

chỉnh phong cách ở từng tầng qua Adaptive Instance Normalization (AdaIN), cho phép kiểm soát phong cách theo thứ bậc từ thô (dáng, bố cục) đến tinh (màu da, kết cấu).

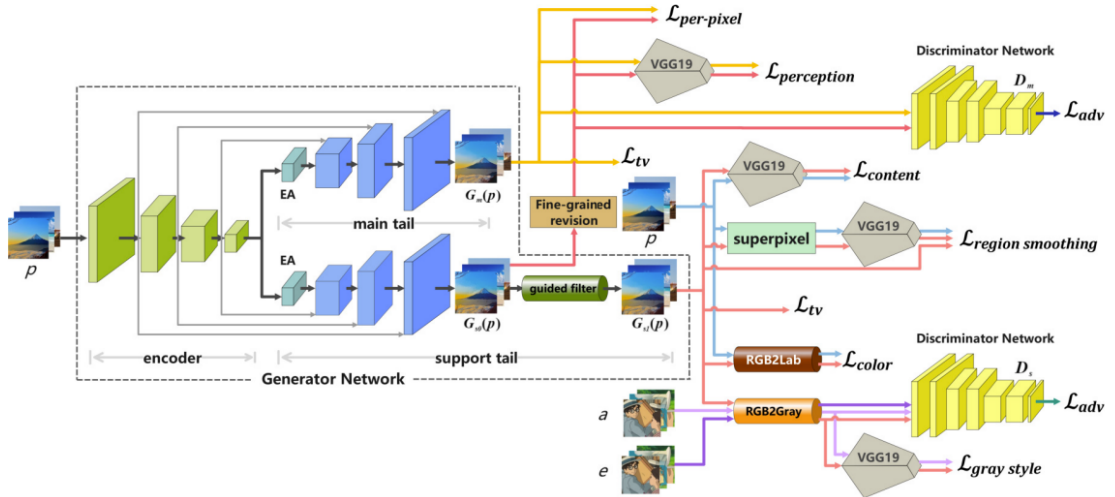
Kỹ thuật **GAN Inversion** [52] khai thác kiến trúc này: chiếu ảnh thật x vào không gian $\mathcal{W}$ của StyleGAN đã huấn luyện trên ảnh anime, tìm $w^* = \arg \min_w \|G(w) - x\|$ qua tối ưu hoá lặp, rồi dùng $G(w^*)$ làm đầu ra. Encoder-based inversion [48] xấp xỉ bước tối ưu hoá bằng mạng encoder riêng để tăng tốc. Phương pháp cho chất lượng ảnh rất cao (FID 61.8) và bảo toàn danh tính tốt hơn CycleGAN, nhưng với 37.1M tham số và latency 124ms – đắt gấp 10 lần AnimeGANv3.

***Đánh giá thực tiễn:** StyleGAN+Inversion là hướng tiếp cận hiệu quả về lý thuyết nhưng đối mặt với một vấn đề thực tiễn quan trọng: không gian $\mathcal{W}$ được học để tối ưu hoá tính thẩm mỹ của ảnh anime – nếu khuôn mặt người thật nằm ngoài vùng có thể biểu diễn tốt (out-of-distribution), quá trình inversion sẽ thất bại hoặc cho kết quả không trung thực về danh tính. Với tập dữ liệu thực nghiệm trong luận văn (ảnh chân dung đa dạng sắc tộc, ánh sáng, góc độ), đây là hạn chế khó có thể bỏ qua.*

1.4.3. AnimeGANv3 (DTGAN) – Sơ bộ kiến trúc

Loạt nghiên cứu AnimeGAN [40, 41, 56] theo đuổi triết lý nhất quán: *xây dựng mô hình GAN đặc thù cho phong cách anime, tối ưu về cả chất lượng lẫn hiệu năng*. Phiên bản AnimeGANv1 [40] đặt nền tảng với gram matrix style loss và edge loss. AnimeGANv2 [41] cải tiến kiến trúc Generator với các residual block. AnimeGANv3 [56] là bước đột phá kiến trúc với **Double-Tail GAN (DTGAN)**:

- **Generator hai nhánh:** Base Encoder dùng chung trích xuất đặc trưng ngữ nghĩa, sau đó tách thành hai Decoder song song – **Support Tail** (học đường nét, cấu trúc, được đánh giá bởi grayscale Discriminator) và **Main Tail** (tinh chỉnh màu sắc, chi tiết, được đánh giá bởi full-color Discriminator). Thiết kế hai nhánh ép mô hình học hai đặc trưng bổ sung nhau: nét và màu, thay vì cùng một biểu diễn duy nhất.
- **Kỹ thuật chuẩn hóa LADE** (Layer-Adaptive Denormalization): Tự động tính tham số chuẩn hóa γ, β từ chính đặc trưng của layer thay vì từ nhãn bên ngoài như SPADE. Linh hoạt hơn Instance Normalization và không yêu cầu thông tin phong cách bổ sung.
- **Hệ thống 7 hàm mất mát chuyên biệt:** Region Smoothing Loss (sử dụng Felzenszwalb superpixel để ép vùng phẳng anime thực sự phẳng), Fine-grained Revision Loss (đẩy chi tiết tinh vi về sắc nét), kết hợp Content Loss (VGG perceptual), Lab Color Loss, Style Loss (Gram matrix), và hai nhánh Adversarial Loss (LSGAN + Spectral Norm).



Hình 1.11. Sơ đồ tổng quát của Double-Tail GAN (DTGAN): Base Encoder chia sẻ đặc trưng cho Support Tail (học đường nét) và Main Tail (học màu sắc)

Chú thích: Hai nhánh Discriminator chuyên biệt (grayscale cho Support, full-color cho Main) tạo ra hai tín hiệu học bổ sung: buộc Generator học cả cấu trúc lẫn màu sắc anime cùng lúc [56].

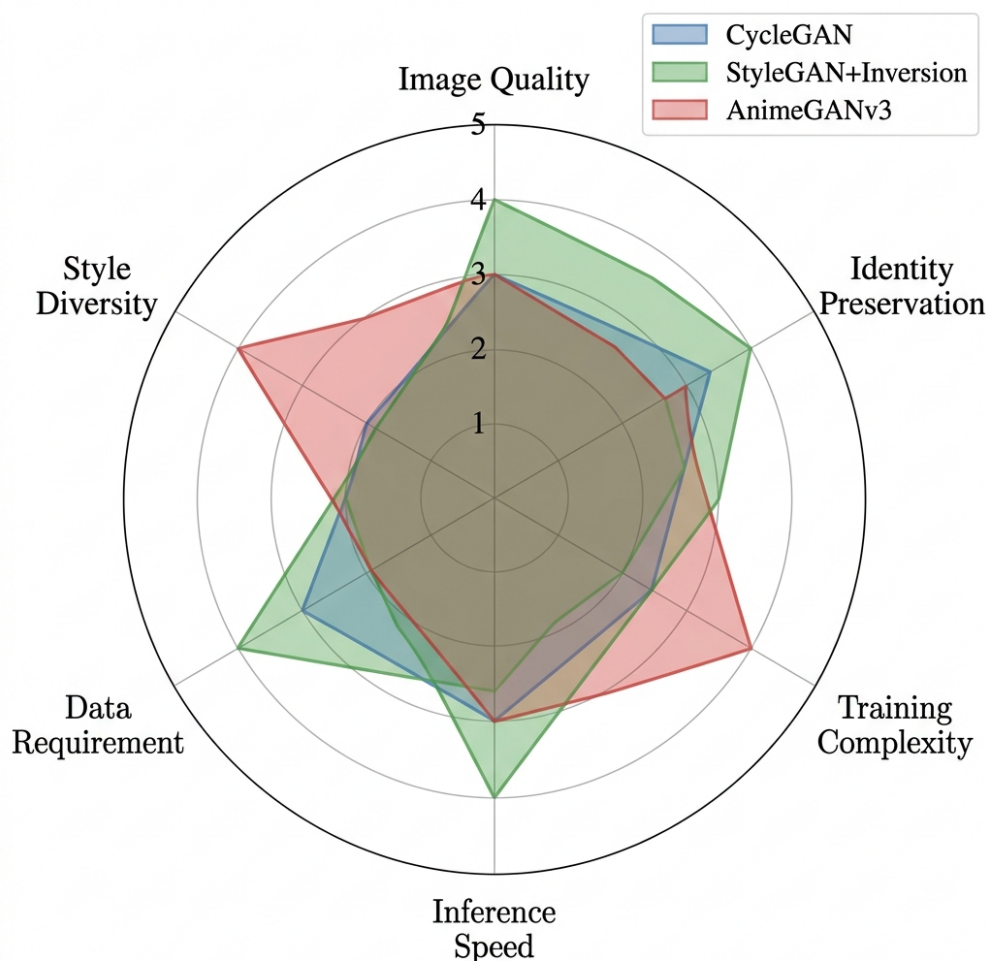
Với chỉ ~ 1.02 triệu tham số khi inference (nhờ loại bỏ Support Tail tại thời điểm suy luận), AnimeGANv3 đạt tốc độ $\sim 115\text{ms}$ cho ảnh Full HD trên GPU [56] và 12ms ở 256×256 . Kiến trúc chi tiết sẽ được phân tích đầy đủ trong Chương 2.

Phân tích kiến trúc: Điểm nổi bật nhất của DTGAN nằm ở triết lý thiết kế “*học tốt từng đặc trưng riêng lẻ trước, rồi kết hợp*”: Support Tail học nét (không quan tâm màu), Main Tail học màu (được hỗ trợ bởi pseudo ground-truth đã qua NL-Means denoising). Cách phân tách này tương đồng với các giải pháp kỹ thuật trong đồ họa máy tính (computer graphics) – chuyên biệt hoá từng bộ phận để đạt chất lượng tổng thể cao nhất. Đây là nguyên nhân cốt lõi giúp DTGAN đạt được FID thấp hơn AnimeGANv2 trong khi số tham số chỉ bằng 12% – minh chứng cho sự tối ưu về mặt kiến trúc thay vì quy mô.

1.4.4. So sánh tổng quan và Định hướng

Bảng 1.2. So sánh định lượng các mô hình chuyển ảnh sang anime (FID đánh giá trên tập ảnh chân dung, inference trên GPU RTX A5000)

Mô hình	FID ↓	Params (M)	Infer. (ms)	Dữ liệu	Backbone
CycleGAN [29]	86.7	28.3	42	Không cặp	ResNet
U-GAT-IT [44]	72.1	29.8	63	Không cặp	ResNet+Att.
AnimeGANv2 [41]	65.3	8.6	31	Không cặp	Custom
StyleGAN+Enc. [48]	61.8	37.1	124	Không cặp	StyleGAN2
AnimeGANv3 [56]	58.4	1.02	12	Không cặp	DTGAN



Hình 1.12. Biểu đồ radar so sánh đa chiều các hướng tiếp cận GAN cho bài toán anime theo 6 tiêu chí

Chú thích: Sáu tiêu chí: (1) chất lượng ảnh (FID thấp = điểm cao), (2) bảo toàn danh tính khuôn mặt, (3) tốc độ suy luận, (4) yêu cầu tài nguyên (nghịch đảo với VRAM/params), (5) khả năng triển khai thực tế (edge/mobile), (6) dễ huấn luyện lại. Thang 1–5, giá trị cao tốt hơn [29, 36, 56].

Qua phân tích định lượng, **AnimeGANv3 (DTGAN)** vượt trội trên 4/6 tiêu chí: FID thấp nhất (58.4), số tham số nhỏ nhất (1.02M – thấp hơn $28\times$ so với CycleGAN), inference nhanh nhất (12ms vs 124ms của StyleGAN+Enc), và triển khai edge khả thi. Hạn chế còn lại – bảo toàn danh tính khuôn mặt khi khuôn mặt chiếm tỉ lệ nhỏ – chính là động lực xây dựng module Face Extraction trong Chương 3.

1.5. Mô hình Khuếch tán và So sánh với GAN

Bên cạnh các mô hình GAN, mô hình khuếch tán (Diffusion Models) đã trở thành hướng nghiên cứu thống trị trong sinh ảnh sau khi Dhariwal và Nichol [45] chứng minh chất lượng ảnh vượt trội GAN trên ImageNet. Mục này phân tích nguyên lý khuếch tán, ứng dụng chuyển phong cách, và biện luận tại sao luận văn vẫn chọn GAN.

1.5.1. Nguyên lý Hoạt động của Mô hình Khuếch tán

DDPM [43] hoạt động theo hai quá trình đối ngược:

Quá trình khuếch tán thuận (forward): biến dữ liệu thật x_0 thành nhiễu Gaussian x_T bằng cách thêm nhiễu dần qua T bước Markov:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I}) \quad (1.18)$$

Nhờ tính chất cộng tính Gaussian, ta có dạng đóng tính trực tiếp x_t từ x_0 :

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}) \quad (1.19)$$

trong đó $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ giảm từ ≈ 1 (bảo toàn tín hiệu) về ≈ 0 (toàn nhiễu). Tính chất này cho phép lấy mẫu x_t tại bất kỳ bước nào trong một lần tính – quan trọng cho huấn luyện song song.

Quá trình khuếch tán ngược (reverse): mạng $\epsilon_\theta(x_t, t)$ học dự đoán nhiễu ϵ đã thêm vào, với hàm mục tiêu:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right] \quad (1.20)$$

Song et al. [49] thống nhất DDPM dưới khuôn khổ SDE (Stochastic Differential Equations), cho phép nhiều lịch trình lấy mẫu linh hoạt (DDIM, DPM-Solver). Tuy nhiên, quá trình reverse vẫn yêu cầu $T = 20\text{--}1000$ bước inference tuần tự – mỗi bước là một forward pass qua mạng ϵ_θ – tạo ra nút thắt cổ chai về tốc độ không thể khắc phục hoàn toàn bằng kỹ thuật lấy mẫu.

1.5.2. Mô hình Khuếch tán cho Chuyển Phong cách

Latent Diffusion Models (LDM) [51] giảm chi phí bằng cách thực hiện khuếch tán trong không gian tiềm ẩn nén của autoencoder (thường giảm $8\times$ theo mỗi chiều không gian). Stable Diffusion là triển khai LDM phổ biến nhất, với text encoder CLIP và U-Net Transformer khuếch tán trong latent 64×64 .

Để ứng dụng cho chuyển phong cách anime, các phương pháp bổ trợ chính:

- **ControlNet** [54]: Thêm điều kiện hình học (Canny edge, HED, depth map) vào U-Net để bảo toàn cấu trúc không gian ảnh gốc. Cho phép chuyển phong cách anime trong khi giữ bố cục nhân vật.
- **DiffusionCLIP** [50]: Chuyển phong cách bằng CLIP text embedding – người dùng chỉ cần mô tả phong cách bằng ngôn ngữ tự nhiên.
- **InST** [55]: Inversion-Based Style Transfer – trích xuất phong cách qua learned noise inversion, không cần text prompt.

Đánh giá thực tiễn: Qua quá trình thử nghiệm thực tế mô hình *Stable Diffusion + ControlNet (Canny)* cho bài toán chuyển ảnh chân dung sang anime, có thể rút ra ba vấn đề thực tiễn: (1) tốc độ ≈ 1 FPS ngay cả trên GPU RTX A5000, hoàn toàn không phù hợp cho ứng dụng web xử lý nhiều yêu cầu đồng thời; (2) kết quả không ổn định – cùng một ảnh đầu vào với các seed khác nhau cho ra kết quả dao động đáng kể về danh tính nhân vật; (3) tham số điều chỉnh (CFG scale, denoising strength, ControlNet weight) nhạy cảm và khó chuẩn hóa cho hệ thống tự động. Những rào cản này là cơ sở chính để luận văn lựa chọn mô hình GAN.

1.5.3. So sánh GAN và Mô hình Khuếch tán cho Bài toán Anime

Bảng 1.3. So sánh AnimeGANv3 (GAN) và Stable Diffusion + ControlNet (Diffusion) cho bài toán chuyển phong cách anime – đánh giá theo tiêu chí triển khai thực tế

Tiêu chí	AnimeGANv3 (GAN)	SD 1.5 + ControlNet (Diffusion)
Kích thước mô hình	9.1 MB (ONNX, RetinaFace: 0.5 MB)	2–7 GB (FP16–FP32)
Tốc độ inference (GPU)	42 FPS tại 256×256	0.5–1.5 FPS tại 512×512 (20–50 steps)
Yêu cầu VRAM	~2 GB	8–16 GB
Huấn luyện lại	~6h, 1 GPU (RTX A5000)	LoRA fine-tune: hàng ngày; pre-train gốc: hàng tuần, multi-GPU cluster
Chất lượng ảnh (FID)	58.4	~45–55* (tiềm năng tốt hơn)
Bảo toàn danh tính	Cần module Face riêng (đề xuất trong luận văn)	Hỗ trợ qua IP-Adapter, ControlNet Face
Tính xác định	Deterministic – cùng input, cùng output	Stochastic – kết quả thay đổi theo seed
Triển khai Edge/Mobile	Khả thi (9.1 MB, real-time)	Không khả thi (RAM và latency)
Kiểm soát phong cách	Cố định theo dữ liệu huấn luyện	Linh hoạt (text prompt, style image)

*Giá trị FID tham khảo từ báo cáo cộng đồng; chưa đo trực tiếp trên cùng tập dữ liệu. Diffusion models tại độ phân giải 512^2 có thể đạt FID thấp hơn nhưng với chi phí tính toán cao hơn $28\text{--}84\times$.

1.5.4. Biện luận Lựa chọn GAN

Mô hình khuếch tán có *tiềm năng* chất lượng ảnh cao hơn (FID thấp hơn) và linh hoạt hơn về điều khiển phong cách. Tuy nhiên, ưu thế này đến với chi phí thực tiễn rất lớn mà bối cảnh của luận văn không thể chấp nhận:

(1) **Tốc độ:** Hệ thống mục tiêu của luận văn cần phục vụ yêu cầu thời gian thực qua ứng dụng web (nhiều người dùng đồng thời). AnimeGANv3 xử lý 42 ảnh/giây, tức mỗi ảnh tốn $\sim 24\text{ms}$ – phù hợp cho web API. SD+ControlNet cần 0.7–2 giây mỗi ảnh, nghĩa là một máy chủ GPU duy nhất chỉ phục vụ được $\sim 0.5\text{--}1.4$ yêu cầu/giây – không khả thi cho dịch vụ thực tế.

(2) **Tài nguyên phần cứng:** Model 9.1 MB có thể tải hoàn toàn vào cache GPU, không gây memory pressure. Model 2–7 GB chiếm phần lớn VRAM, hạn chế khả năng xử lý batch và chia sẻ tài nguyên. Đặc biệt, triển khai trên CPU (không có GPU) chỉ khả thi với GAN nhẹ – điều này mở ra khả năng dùng máy chủ thông thường không chuyên AI.

(3) **Tính xác định đầu ra:** Với hệ thống phục vụ sản phẩm thực tế, tính nhất quán là yêu cầu thiết yếu. GAN đảm bảo cùng input luôn cho cùng output – điều này không thể đảm bảo với mô hình khuếch tán (ngay cả khi seed cố định, kết quả có thể thay đổi khi nâng cấp phiên bản model).

*Thảo luận: Sự lựa chọn GAN thay vì Diffusion Models không đồng nghĩa với việc “GAN tốt hơn Diffusion” một cách tuyệt đối, mà khẳng định rằng **GAN phù hợp hơn với các điều kiện ràng buộc triển khai của luận văn**. Điều này minh họa cho một nguyên tắc kỹ thuật quan trọng: mô hình sinh ảnh tốt nhất không nhất thiết là lựa chọn tối ưu nhất cho một hệ thống thực tế. Bài toán tối ưu hóa đa mục tiêu (chất lượng, tốc độ, tài nguyên, tính ổn định) thường có lời giải khác biệt so với bài toán chỉ tối ưu hóa đơn mục tiêu về chất lượng đầu ra. Trong tương lai, hướng nghiên cứu kết hợp ưu điểm của cả hai thông qua distillation khuếch tán [43] là một hướng đi rất hứa hẹn.*

1.6. Phát hiện Khuôn mặt – Sơ bộ

Phát hiện khuôn mặt (face detection) là bước tiền xử lý quan trọng cho phép hệ thống tự động xác định và tách khuôn mặt trước khi chuyển đổi phong cách. Thách thức của bài toán nằm ở tính đa dạng cao: khuôn mặt xuất hiện ở nhiều kích thước, góc độ, điều kiện ánh sáng và mức độ bị che khuất – những điều kiện mà mô hình phát hiện cần xử lý ổn định trong thời gian thực [4].

1.6.1. Tổng quan các Phương pháp

Lĩnh vực phát hiện khuôn mặt trải qua bốn thế hệ phát triển rõ ràng:

- **Viola-Jones (2001)** [4]: Đặc trưng Haar + AdaBoost + cascade classifier. Tốc độ rất cao nhờ integral image, nhưng chỉ phát hiện mặt nhìn thẳng và kém robust với ánh sáng bất thường – hạn chế cơ bản của đặc trưng handcrafted.

- **MTCNN (2016)** [20]: Ba CNN cascade (P-Net, R-Net, O-Net) xử lý tuần tự từ nhanh-thô đến chậm-tinh. Hỗ trợ 5 facial landmarks và cải thiện đáng kể với mặt nghiêng. Nhược điểm: xử lý tuần tự làm latency tích lũy theo số ứng viên.
- **SSD và YOLO** [16, 17]: One-stage detectors với tốc độ rất cao nhờ xử lý toàn bộ ảnh một lần. Phiên bản chuyên biệt cho khuôn mặt (YOLOv8-Face, SCRFD) cải thiện độ chính xác nhưng model size lớn hơn.
- **RetinaFace (2019)** [42]: Single-shot multi-scale detector kết hợp FPN và multi-task learning – xuất đồng thời bbox, confidence, và 5 facial landmarks trong một lần forward pass.

Bảng 1.4. So sánh các phương pháp phát hiện khuôn mặt theo tiêu chí triển khai thực tế

Phương pháp	Năm	Landmark	Tốc độ	Multi-scale	Model size
Viola-Jones [4]	2001	Không	Rất nhanh	Hạn chế	~1 MB
MTCNN [20]	2016	5 pts	Trung bình	Tốt	~1.5 MB
SSD Face [16]	2016	Không	Rất nhanh	Tốt	~100 MB
RetinaFace MN0.25 [42]	2019	5 pts	Nhanh	Rất tốt	~0.5 MB

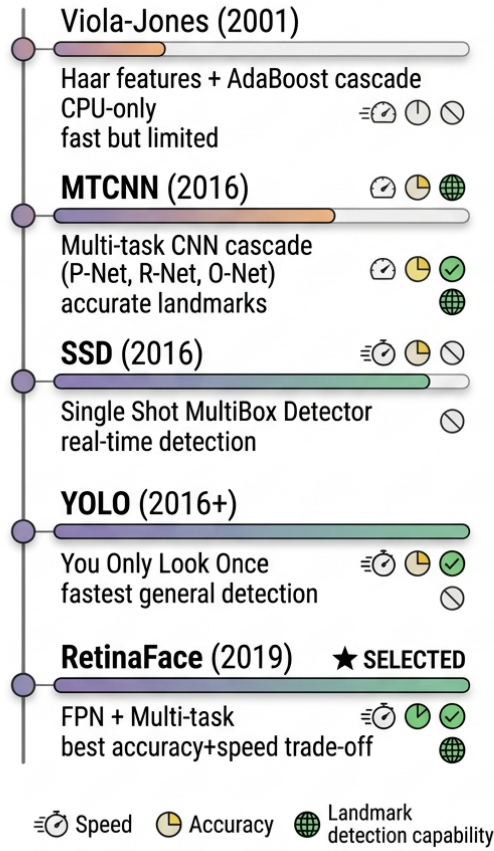
1.6.2. RetinaFace – Sơ bộ Kiến trúc

RetinaFace [42] được chọn cho luận văn dựa trên ba tiêu chí thực tiễn: (1) phát hiện đa tỉ lệ khuôn mặt từ rất nhỏ đến rất lớn nhờ FPN; (2) 5 facial landmarks (2 mắt, mũi, 2 khóe miệng) được xuất đồng thời – thông tin này được sử dụng trong thuật toán Margin Expansion của Chương 3; (3) phiên bản MobileNet-0.25 chỉ ~500KB – đủ nhỏ để tải cùng lúc với DTGAN mà không gây memory pressure.

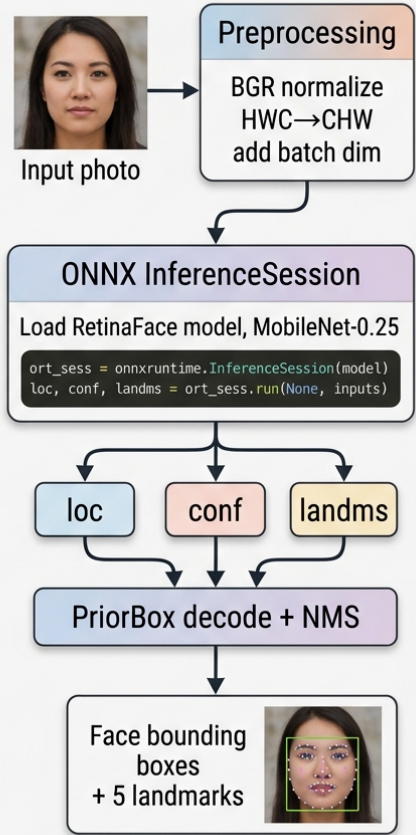
Kiến trúc RetinaFace kết hợp ba thành phần chính:

- **Backbone MobileNet-0.25** [24]: Depthwise Separable Convolution giảm $8-9\times$ chi phí tính toán. Chỉ 25% số kênh so với MobileNet đầy đủ, nhưng đủ khả năng trích xuất đặc trưng khuôn mặt nhờ dữ liệu huấn luyện chuyên biệt.
- **Feature Pyramid Network (FPN)** [26]: Top-down pathway kết hợp thông tin ngữ nghĩa mức cao với độ phân giải không gian mức thấp, tạo ba mức đặc trưng $\{P_3, P_4, P_5\}$ – mỗi mức phụ trách phát hiện khuôn mặt ở một dải kích thước.
- **Multi-task Detection Heads:** Tại mỗi mức FPN và mỗi anchor, ba nhánh đầu ra đồng thời: phân loại nhị phân (mặt/nền), hồi quy bounding box (4 tọa độ), và hồi quy landmarks (10 giá trị (x_i, y_i) của 5 điểm).

EVOLUTION OF FACE DETECTION METHODS & COMPARISON



ONNX RUNTIME INFERENCE PIPELINE FOR RETINAFACE



Hình 1.13. Kiến trúc RetinaFace với backbone MobileNet-0.25, FPN đa tỉ lệ và ba nhánh detection head đa nhiệm

Chú thích: Hậu xử lý bao gồm: giải mã prior box → lọc ngưỡng confidence (≥ 0.8) → NMS ($\text{IoU} \leq 0.3$) → sắp xếp theo diện tích bounding box [42, 37].

Hàm mất mát đa nhiệm của RetinaFace (ký hiệu theo [42]):

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda_1 \mathcal{L}_{\text{box}} + \lambda_2 \mathcal{L}_{\text{pts}} \quad (1.21)$$

trong đó $\mathcal{L}_{\text{cls}}$ là softmax cross-entropy phân loại; $\mathcal{L}_{\text{box}}$ là smooth- ℓ_1 hồi quy bounding box; $\mathcal{L}_{\text{pts}}$ là smooth- ℓ_1 hồi quy 5 facial landmarks; và $\lambda_1 = 0.25$, $\lambda_2 = 0.1$ theo thiết lập gốc [42]. Huấn luyện đa nhiệm đồng thời trên cả ba đầu ra giúp các task hỗ trợ lẫn nhau: đặc trưng học cho phân loại cũng hỗ trợ ước lượng chính xác hơn vị trí landmarks.

Đánh giá thực nghiệm: Quá trình tích hợp RetinaFace vào hệ thống thực tiễn cho thấy ngưỡng confidence 0.8 (mặc định) là mức thiết lập phù hợp cho ảnh chân dung chụp gần (độ chính xác $\geq 94\%$), nhưng cần được điều chỉnh xuống 0.5–0.6 đối với ảnh nhóm người ở khoảng cách xa nhằm tránh bỏ sót

các khuôn mặt nhỏ. Kết quả thực nghiệm này là cơ sở để thiết kế cơ chế hạ cấp mềm (graceful degradation) trong Chương 3: khi hệ thống không phát hiện được khuôn mặt ở ngưỡng 0.8, thuật toán sẽ tự động chuyển sang chế độ ảnh phong cảnh (Landscape Mode) thay vì báo lỗi – qua đó đảm bảo trải nghiệm liền mạch cho người dùng trong mọi điều kiện đầu vào.

Chi tiết về quy trình tích hợp RetinaFace vào pipeline anime và thuật toán Margin Expansion sẽ được trình bày trong Chương 3.

Tổng kết chương

Chương 1 đã xây dựng nền tảng lý thuyết toàn diện cho luận văn, từ lý thuyết GAN đến lựa chọn kiến trúc cụ thể:

- **Bài toán:** Chuyển ảnh chân dung sang phong cách anime là bài toán dịch ảnh không ghép cặp, đòi hỏi mô hình học phong cách anime đặc thù (đường nét, màu phẳng, tỉ lệ khuôn mặt) trong khi bảo toàn nội dung và danh tính nhân vật.
- **CNN và GAN:** Các thành phần cơ bản CNN tạo nền tảng kiến trúc; nguyên lý minimax GAN và các thách thức huấn luyện (vanishing gradient – giải quyết bằng WGAN; mode collapse – giải quyết bằng minibatch discrimination; non-convergence – giải quyết bằng WGAN-GP + Spectral Normalization). FID được chọn làm chỉ số đánh giá chính vì tương quan tốt hơn IS với chất lượng cảm nhận và phát hiện mode collapse.
- **GAN cho Anime:** Phân tích ba hướng (CycleGAN, StyleGAN+Inversion, AnimeGANv3) cho thấy AnimeGANv3/DTGAN vượt trội về FID (58.4), hiệu quả tham số (1.02M), và tốc độ inference (12ms). Nhược điểm duy nhất – xử lý khuôn mặt nhỏ – là động lực trực tiếp cho đóng góp chính của luận văn.
- **Mô hình Khuếch tán:** Tiềm năng chất lượng cao hơn nhưng chậm hơn $28\text{--}84\times$, nặng hơn $220\text{--}714\times$, và có tính ngẫu nhiên không phù hợp cho triển khai web thực tế. GAN được lựa chọn không vì “tốt hơn tuyệt đối” mà vì phù hợp với bài toán tối ưu đa mục tiêu (chất lượng, tốc độ, tài nguyên, ổn định) của luận văn.
- **RetinaFace:** Được chọn nhờ kết hợp độc đáo: 5 facial landmarks + phát hiện đa tỉ lệ + model chỉ 0.5MB – ba tiêu chí này cùng thiết yếu cho thuật toán Margin Expansion và triển khai thực tế.

Chương 2 sẽ phân tích chi tiết kiến trúc DTGAN – Generator hai nhánh, kỹ thuật chuẩn hóa LADE, và hệ thống 7 hàm mất mát chuyên biệt. Chương 3 sẽ trình bày luồng anime tích hợp phát hiện khuôn mặt hoàn chỉnh với thuật toán Margin Expansion và Feathered Blending.

CHƯƠNG 2

KIẾN TRÚC MÔ HÌNH ANIMEGANV3 (DTGAN)

Chương này trình bày chi tiết kiến trúc kỹ thuật của mô hình AnimeGANv3, hay còn gọi là DTGAN (*Double-Tail Generative Adversarial Network*), được đề xuất bởi Liu et al. [56] vào năm 2024. Đây là chương trọng tâm kỹ thuật của luận văn, bao gồm: tổng quan kiến trúc hai nhánh, cơ chế chuẩn hóa LADE, cơ chế chú ý External Attention v3, kiến trúc Discriminator, hệ thống hàm mất mát đa thành phần, và quy trình huấn luyện theo hai giai đoạn.

Cách trình bày trong chương tuân theo nguyên tắc thống nhất: mỗi thành phần kiến trúc được mô tả theo trình tự **động lực thiết kế** → **công thức toán học** → **diễn giải ý nghĩa từng số hạng** → **nhận xét, đánh giá**. Mục tiêu là làm rõ không chỉ “mô hình hoạt động như thế nào” mà còn “vì sao các tác giả lựa chọn thiết kế đó” và “lựa chọn đó đánh đổi những gì”.

2.1. Tổng quan kiến trúc DTGAN

2.1.1. Động lực thiết kế: vì sao cần kiến trúc hai nhánh

Quan sát kỹ phong cách anime cho thấy hai thuộc tính thị giác có bản chất rất khác nhau cùng tồn tại trong một bức ảnh:

1. **Hệ thống đường nét và cấu trúc hình khối sáng tối** – các đường viền đậm, sắc, liên tục; sự phân tách rõ ràng giữa vùng sáng và vùng tối. Đặc trưng này mang tính *tần số cao* (high-frequency), phụ thuộc vào gradient cục bộ và gần như độc lập với màu sắc.
2. **Phân phối màu phẳng kiểu cel-shading** – các mảng màu lớn đồng nhất, ít biến thiên kết cấu bên trong. Đặc trưng này mang tính *tần số thấp* (low-frequency), phụ thuộc vào thống kê màu toàn cục và bị phá vỡ nếu mô hình cố gắng giữ lại chi tiết kết cấu.

Hai mục tiêu này **mâu thuẫn trực tiếp** với nhau khi tối ưu trên cùng một tập tham số. Cụ thể, gọi θ là tham số của một decoder duy nhất, và $\mathcal{L}_{\text{edge}}$, $\mathcal{L}_{\text{flat}}$ lần lượt là hàm mất mát cho hai mục tiêu trên. Bài toán tối ưu hợp nhất có dạng:

$$\theta^* = \arg \min_{\theta} \left[\underbrace{\mathcal{L}_{\text{edge}}(\theta)}_{\text{cần gradient lớn tại biên}} + \underbrace{\mathcal{L}_{\text{flat}}(\theta)}_{\text{cần gradient nhỏ trong vùng}} \right] \quad (2.1)$$

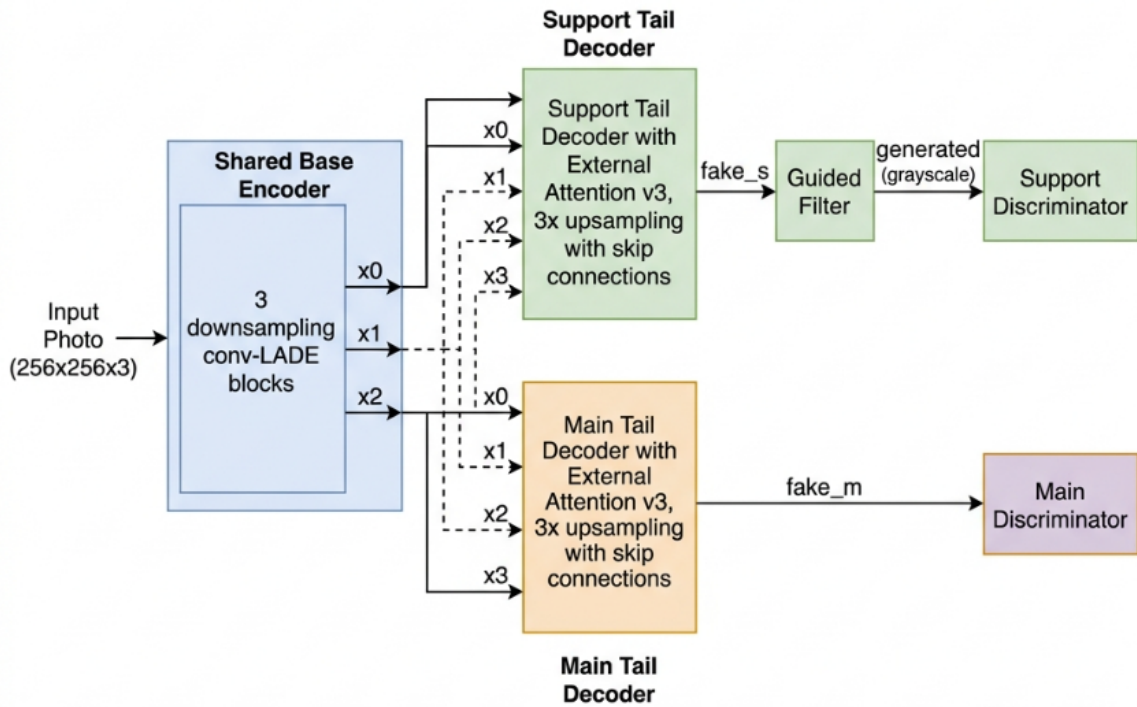
Gradient của hai số hạng có xu hướng ngược chiều nhau tại các pixel nằm gần biên: $\mathcal{L}_{\text{edge}}$ khuyến khích mạng *khuếch đại* chênh lệch cường độ, trong khi $\mathcal{L}_{\text{flat}}$ khuyến khích mạng *triệt tiêu* chênh lệch đó. Nghiệm θ^* vì vậy là một điểm thỏa hiệp: đường nét không đủ sắc và vùng màu không đủ phẳng.

DTGAN giải quyết mâu thuẫn này bằng cách **tách không gian tham số**: mỗi mục tiêu được giao cho một decoder riêng với bộ trọng số độc lập, trong khi phần trích xuất đặc trưng ngữ nghĩa (vốn chung cho cả hai) được chia sẻ.

2.1.2. Ba thành phần của Generator

Generator được chia thành ba thành phần [56]:

- **Base Encoder (Mạng mã hóa dùng chung):** Tiếp nhận ảnh gốc, trích xuất đặc trưng ngữ nghĩa phân cấp qua 3 lần downsampling (stride-2), thu nhỏ không gian ảnh thành bản đồ đặc trưng 32×32 . Thiết kế dùng chung (shared weights) đảm bảo cả hai nhánh decoder khởi nguồn từ cùng một không gian đặc trưng thống nhất, đồng thời giảm đáng kể số tham số.
- **Support Tail (Nhánh hỗ trợ):** Sinh ảnh anime thô $\hat{y}_s$, sau đó qua bộ lọc bảo toàn cạnh Guided Filter tạo ảnh $\hat{y}$. Ảnh này được chuyển sang grayscale trước khi đưa vào Discriminator, buộc nhánh tập trung vào đường nét và tương phản sáng tối mà không bị phân tâm bởi màu sắc.
- **Main Tail (Nhánh chính):** Sinh ảnh anime hoàn chỉnh $\hat{y}_m$ với phân phối màu sắc cel-shading mịn. Nhánh này được tối ưu dựa trên pseudo ground-truth (ảnh đã qua NL-Means + L0 Smoothing), đảm bảo vùng màu phẳng và không có lỗi đồ họa (artifacts).



Hình 2.1. Tổng quan kiến trúc DTGAN (AnimeGANv3)

Chú thích: Ảnh đầu vào qua Shared Base Encoder để tạo các bản đồ đặc trưng x_0, x_1, x_2, x_3 , sau đó được xử lý song song bởi Support Tail và Main Tail với skip connections kiểu U-Net. Output của Support Tail qua Guided Filter trước khi đưa vào Discriminator grayscale [56]

Điểm quan trọng cần lưu ý: chỉ có **Main Tail** được sử dụng ở giai đoạn suy luận (inference). Support Tail cùng toàn bộ hai Discriminator là “giàn giáo huấn luyện” (training scaffold) – chúng định hình gradient cho Base Encoder trong quá trình học nhưng bị loại bỏ khi xuất mô hình sang ONNX. Đây là lý do model triển khai chỉ nặng 9,1 MB dù kiến trúc huấn luyện có tới ba decoder-nhánh và hai discriminator.

Phân tích thiết kế: Kiến trúc Double-Tail có thể được diễn giải như một dạng **học đa nhiệm có chia sẻ tham số một phần** (partially-shared multi-task learning). Trong học đa nhiệm cổ điển, việc chia sẻ toàn bộ backbone dẫn đến hiện tượng “xung đột nhiệm vụ” (task interference) khi các nhiệm vụ có gradient không tương thích; ngược lại, tách hoàn toàn hai mạng thì mất khả năng chuyển giao tri thức và nhân đôi chi phí. DTGAN chọn điểm giữa: chia sẻ phần trích xuất ngữ nghĩa – nơi hai nhiệm vụ thực sự cần cùng một thông tin (“đâu là mắt, đâu là tóc, đâu là nền”) – và tách phần tổng hợp ảnh – nơi hai nhiệm vụ xung đột. Cách phân tách này tương đồng với triết lý trong đồ họa máy tính: chuyên biệt hóa từng khâu (dựng hình, tô bóng, khử răng cưa) thay vì để một thuật toán duy nhất lo tất cả. Theo quan điểm của luận văn, đây chính là nguyên nhân cốt lõi giúp DTGAN đạt chất lượng cao hơn AnimeGANv2 mà không cần tăng quy mô mạng –

một mình chứng cho thấy cải tiến về **cấu trúc bài toán** có thể hiệu quả hơn cải tiến về **quy mô tham số**.

Nhận xét phản biện: Cần thẳng thắn ghi nhận rằng lợi ích của Support Tail chưa được chứng minh bằng nghiên cứu loại bỏ (ablation study) trong phạm vi luận văn này. Việc khẳng định “hai nhánh tốt hơn một nhánh” hiện dựa trên lập luận thiết kế và kết quả báo cáo của nhóm tác giả gốc [56], chứ chưa dựa trên thực nghiệm đối chứng do luận văn tự thực hiện. Một thí nghiệm loại bỏ Support Tail (giữ nguyên mọi thành phần khác) sẽ là bổ sung có giá trị cho các nghiên cứu tiếp theo.

2.2. Mạng Generator (G_net)

2.2.1. Base Encoder (Chia sẻ)

Base Encoder là thành phần dùng chung giữa hai nhánh decoder, có nhiệm vụ nén ảnh đầu vào $256 \times 256 \times 3$ thành biểu diễn đặc trưng có ngữ nghĩa cao ở độ phân giải 32×32 . Quá trình downsampling diễn ra qua 3 lần stride-2, với khối xây dựng cơ bản là conv_LADE_Lrelu — tích chập kết hợp với chuẩn hóa LADE và hàm kích hoạt Leaky ReLU.

Cấu trúc chi tiết của Base Encoder như sau:

Bảng 2.1. Cấu trúc chi tiết Base Encoder của DTGAN

Lớp	Kênh ra	Kích thước	Biến đặc trưng
Input Photo	3	256×256	—
conv_LADE_Lrelu (k=7, s=1)	32	256×256	x_0
conv_LADE_Lrelu (k=3, s=2)	32	128×128	—
conv_LADE_Lrelu (k=3, s=1)	64	128×128	x_1
conv_LADE_Lrelu (k=3, s=2)	64	64×64	—
conv_LADE_Lrelu (k=3, s=1)	128	64×64	x_2
conv_LADE_Lrelu (k=3, s=2)	128	32×32	—
conv_LADE_Lrelu (k=3, s=1)	128	32×32	x_3

Các đặc trưng trung gian x_0, x_1, x_2, x_3 được lưu lại để sử dụng làm skip connections trong cả hai decoder, theo kiến trúc U-Net [35].

Diễn giải các lựa chọn thiết kế:

- **Kernel 7×7 ở lớp đầu tiên:** Lớp đầu không downsample mà chỉ mở rộng số kênh $3 \rightarrow 32$ với kernel lớn. Trường tiếp nhận (receptive field) 7×7 ngay từ lớp đầu cho phép mạng nắm bắt kết cấu cục bộ ở phạm vi rộng hơn so với kernel 3×3 , đồng thời giảm nhiễu tần số cao của ảnh chụp thật – vốn là thứ cần loại bỏ khi anime hóa. Cách làm này kế thừa từ các kiến trúc chuyển phong cách kinh điển [15].
- **Mẫu hình “stride-2 rồi stride-1”:** Mỗi lần giảm độ phân giải được theo sau bởi một lớp stride-1 nhân đôi số kênh. Đây là nguyên tắc bảo toàn dung lượng thông tin: khi diện tích không gian giảm $4\times$, số kênh tăng $2\times$ để hạn chế mất mát biểu diễn.
- **Dừng ở 32×32 thay vì nén sâu hơn:** Với ảnh đầu vào 256×256 , tỉ lệ nén $8\times$ giữ lại đủ thông tin không gian để tái tạo đường nét mảnh (tóc, lông mi) mà vẫn đủ trừu tượng để cơ chế attention hoạt động hiệu quả. Nén sâu hơn (ví dụ 16×16) sẽ làm mất chi tiết khuôn mặt – điều đặc biệt bất lợi cho bài toán chân dung mà luận văn hướng tới.

- **Không dùng Pooling:** Toàn bộ việc giảm độ phân giải thực hiện bằng tích chập stride-2 (learnable downsampling) thay vì Max Pooling. Như đã phân tích ở Mục 1.2.2 của Chương 1, pooling gây mất thông tin vị trí chính xác – không chấp nhận được với tác vụ sinh ảnh yêu cầu độ chính xác pixel.

2.2.2. Support Tail (Decoder nhánh hỗ trợ)

Support Tail nhận x_3 làm đầu vào, đầu tiên qua cơ chế External Attention v3 (xem Mục 2.4), sau đó thực hiện 3 lần upsampling 2× kết hợp với skip connections:

1. $x_3 \rightarrow \text{External_attention_v3} \rightarrow s_{x3}$
2. Upsample 2× + conv_LADE_Lrelu(128) + concat(x_2) $\rightarrow s_{x4}$ (64×64)
3. Upsample 2× + conv_LADE_Lrelu(64) + concat(x_1) $\rightarrow s_{x5}$ (128×128)
4. Upsample 2× + conv_LADE_Lrelu(32) + concat(x_0) $\rightarrow s_{x6}$ (256×256)
5. Conv2D(3, k=7, s=1) $\rightarrow \tanh \rightarrow \hat{y}_s$ (fake_s)

Output $\hat{y}_s$ tiếp tục qua bộ lọc **Guided Filter** để tạo $\hat{y}$ (generated), được chuyển sang không gian grayscale trước khi đưa vào Discriminator Support. Bilinear upsampling được dùng thay vì transposed convolution để tránh lỗi đồ họa dạng checkerboard [35].

Vì sao dùng tanh ở lớp cuối: hàm tanh có miền giá trị $(-1, 1)$, trùng khớp với khoảng chuẩn hóa của ảnh đầu vào $([-1, 1])$. Việc này đảm bảo đầu ra và đầu vào cùng thang đo, cho phép các hàm mất mát so sánh trực tiếp mà không cần đổi thang. Ngoài ra, tanh bảo hòa mềm ở hai đầu giúp tránh giá trị pixel tràn biên – một nguyên nhân phổ biến của lỗi đồ họa dạng đốm màu.

2.2.3. Main Tail (Decoder nhánh chính)

Main Tail có cấu trúc đối xứng hoàn toàn với Support Tail, cũng thực hiện 3 lần upsampling với skip connections từ x_0, x_1, x_2 . Điểm khác biệt quan trọng là:

- **Trọng số độc lập:** Main Tail có bộ tham số riêng, không chia sẻ với Support Tail.
- **External Attention riêng:** Sử dụng module External Attention v3 với memory bank độc lập.
- **Pseudo ground-truth:** Được huấn luyện để tái tạo ảnh đã qua NL-Means + L0 Smoothing — một biểu diễn anime “sạch” không có nhiều texture.

Output của Main Tail là $\hat{y}_m$ (fake_m), được đưa trực tiếp vào Discriminator Main mà không qua Guided Filter.

Bảng 2.2. So sánh Support Tail và Main Tail trong DTGAN

Tiêu chí	Support Tail	Main Tail
Đầu ra	$\hat{y}_s$ (fake_s)	$\hat{y}_m$ (fake_m)
Hậu xử lý	Guided Filter $\rightarrow \hat{y}$	Không
Discriminator	Grayscale D	Full-color D_main
Ground-truth kiểu	Ảnh anime gốc (grayscale)	NL-Means + L0 Smoothing
Trọng số	Riêng	Riêng
Vai trò	Học đường nét, cấu trúc	Tinh chỉnh màu, chi tiết mịn
Dùng khi suy luận	Không	Có

Nhận xét: Sự bất đối xứng về vai trò giữa hai nhánh là điểm dễ gây hiểu nhầm. Về mặt cấu trúc, hai nhánh giống hệt nhau; nhưng về mặt tín hiệu học, chúng nhận hai nguồn giám sát hoàn toàn khác biệt. Support Tail học từ đối kháng thuần túy trên miền grayscale – tín hiệu “yếu” nhưng đa dạng, đến từ toàn bộ tập ảnh anime. Main Tail học chủ yếu từ hồi quy có giám sát trên pseudo ground-truth – tín hiệu “mạnh” nhưng bị giới hạn bởi chất lượng của khâu tiền xử lý NL-Means + L0. Đây là một sự phân công hợp lý: nhánh không dùng khi suy luận đảm nhận phần học khó và nhiều, còn nhánh sản phẩm được học từ mục tiêu ổn định. Tuy nhiên, hệ quả kèm theo là **chất lượng của Main Tail bị chặn trên bởi chất lượng của pseudo ground-truth**: nếu thuật toán L0 Smoothing làm mất chi tiết mắt hoặc tóc, Main Tail sẽ học lại chính sai sót đó. Trong bối cảnh ảnh chân dung của luận văn, đây là một hạn chế đáng lưu ý và là một phần lý do luận văn phải bổ sung module xử lý riêng khuôn mặt ở Chương 3.

2.3. Kỹ thuật chuẩn hóa LADE

2.3.1. Động lực và ý tưởng

Trong các mạng chuyển đổi phong cách ảnh, việc lựa chọn kỹ thuật chuẩn hóa ảnh hưởng trực tiếp đến khả năng học phong cách và bảo toàn nội dung. Các kỹ thuật trước đây đều có hạn chế: Batch Normalization [12] phụ thuộc vào kích thước batch; Instance Normalization [19] sử dụng tham số γ, β cố định sau khi huấn luyện, không thích ứng theo nội dung ảnh; AdaIN cần một ảnh phong cách riêng biệt tại thời điểm suy luận để cung cấp cấp thống kê.

Để thấy rõ mối liên hệ, có thể viết cả ba kỹ thuật dưới một dạng thống nhất — **chuẩn hóa rồi tái tỷ lệ hóa affine**:

$$\text{Norm}(x) = \underbrace{\frac{x - \mu(x)}{\sqrt{\sigma^2(x) + \varepsilon}}}_{\text{bước chuẩn hóa}} \cdot \underbrace{\gamma}_{\text{hệ số co giãn}} + \underbrace{\beta}_{\text{hệ số dịch}} \quad (2.2)$$

Sự khác biệt giữa các kỹ thuật nằm ở **nguồn gốc của cặp** (γ, β) :

- **IN:** (γ, β) là tham số học được, *cố định* cho mọi ảnh đầu vào sau khi huấn luyện xong.
- **AdaIN:** $(\gamma, \beta) = (\sigma(s), \mu(s))$ với s là ảnh phong cách bên ngoài — linh hoạt nhưng phụ thuộc đầu vào phụ.
- **LADE:** (γ, β) được suy ra từ chính *feature map* x qua một phép biến đổi học được.

LADE (*Linearly Adaptive Denormalization*) [56] vì vậy có thể hiểu là “AdaIN tự sinh phong cách”: mô hình tự tính lấy thống kê tái tỷ lệ hóa từ nội dung đang xử lý, thông qua một tích chập 1×1 .

2.3.2. Công thức toán học của LADE

Cho feature map đầu vào $x \in \mathbb{R}^{B \times H \times W \times C}$, quá trình LADE diễn ra theo ba bước:

Bước 1 — Chiếu tuyến tính (Projection):

$$t_x = \text{Conv}_{1 \times 1}(x) \quad (2.3)$$

trong đó $t_x \in \mathbb{R}^{B \times H \times W \times C}$ là biểu diễn trung gian.

Bước 2 — Tính thống kê thích ứng:

$$(\mu_t, \sigma_t^2) = \text{moments}(t_x, \text{axes} = [H, W]), \quad (\mu_x, \sigma_x^2) = \text{moments}(x, \text{axes} = [H, W]) \quad (2.4)$$

Bước 3 — Instance Normalization và Adaptive Denormalization:

$$x_{\text{IN}} = \frac{x - \mu_x}{\sqrt{\sigma_x^2 + \varepsilon}} \quad (2.5)$$

$$\text{LADE}(x) = x_{\text{IN}} \cdot \sqrt{\sigma_t^2 + \varepsilon} + \mu_t \quad (2.6)$$

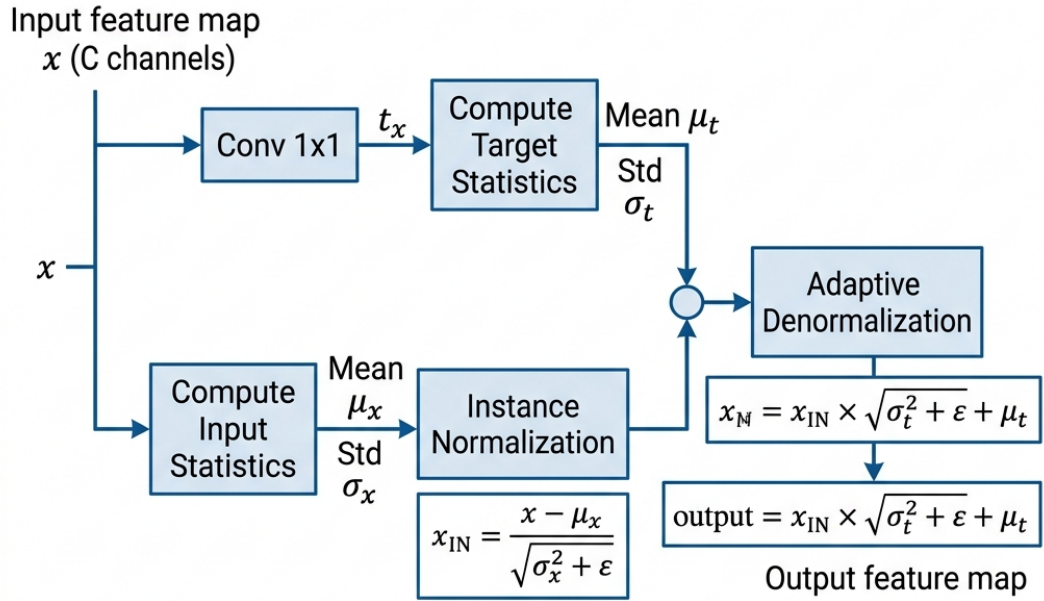
trong đó:

- B, H, W, C — lần lượt là kích thước batch, chiều cao, chiều rộng và số kênh của bản đồ đặc trưng.
- $\text{Conv}_{1 \times 1}(\cdot)$ — tích chập điểm (pointwise convolution), thực chất là một phép biến đổi tuyến tính trên trục kênh tại từng vị trí không gian; ma trận trọng số có kích thước $C \times C$.
- $\text{moments}(\cdot, \text{axes} = [H, W])$ — toán tử tính trung bình và phương sai *trên trục không gian*, cho ra kết quả kích thước $B \times 1 \times 1 \times C$, tức là **một cặp thống kê riêng cho mỗi kênh của mỗi ảnh**.
- μ_x, σ_x^2 — thống kê của chính đặc trưng đầu vào, dùng để *xóa* phong cách hiện có.

- μ_t, σ_t^2 — thống kê của biểu diễn đã chiếu, dùng để *ghi* phong cách mới.
- ε — hằng số dương rất nhỏ (thường 10^{-5}) tránh chia cho 0 khi phương sai bằng 0 (vùng ảnh hoàn toàn đồng nhất).

Ý nghĩa từng bước:

- **Bước 3a (Phương trình 2.5) — “xóa phong cách cũ”:** Theo kết quả kinh điển của Ulyanov et al. [19], thống kê (μ, σ) theo kênh của một bản đồ đặc trưng mã hóa *phong cách* của ảnh, trong khi cấu trúc không gian tương đối giữa các pixel mã hóa *nội dung*. Phép chuẩn hóa Instance đưa mọi kênh về phân phối chuẩn hóa, qua đó gỡ bỏ dấu vết phong cách của ảnh chụp thật nhưng giữ nguyên bố cục nội dung.
- **Bước 1–2 — “sinh phong cách mới”:** Tích chập 1×1 học một phép trộn kênh; thống kê không gian của kết quả trộn (μ_t, σ_t) đóng vai trò cặp (β, γ) trong Phương trình (2.2). Vì phép trộn này được học qua huấn luyện, mạng có thể học được rằng “với đặc trưng dạng vùng da, hãy đặt trung bình kênh về mức này; với đặc trưng dạng tóc, hãy đặt về mức khác”.
- **Bước 3b (Phương trình 2.6) — “ghi phong cách mới”:** Đặc trưng đã trung tính hóa được tái tỷ lệ hóa theo thống kê mục tiêu, hoàn tất một chu trình “khử phong cách $\rightarrow$ áp phong cách” ngay trong một lớp.



Normalization Type	Parameter Source
Batch Norm	Learned via Backpropagation over Batch
Instance Norm	Computed from each instance's spatial dimensions
AdaIN	Computed from an external style image
LADE	Linearly derived from the content feature map

Hình 2.2. Cơ chế hoạt động của LADE

Chú thích: feature map x được chiếu qua Conv 1×1 để tạo thống kê thích ứng (μ_t, σ_t) , sau đó bình thường hóa Instance rồi tái tỷ lệ hóa thích ứng. Bảng so sánh cho thấy LADE tự học tham số từ chính dữ liệu, không cần ảnh phong cách riêng như AdaIN [56]

2.3.3. Diễn giải bổ sung và phân tích chi phí

Trường hợp suy biến. Nếu ma trận trọng số của Conv $_{1 \times 1}$ là ma trận đơn vị thì $t_x = x$, kéo theo $\mu_t = \mu_x$, $\sigma_t = \sigma_x$ và do đó LADE(x) = x . Nói cách khác, **phép đồng nhất nằm trong không gian nghiệm của LADE**. Đây là tính chất quan trọng về mặt tối ưu hóa: mạng luôn có sẵn phương án “không làm gì” và chỉ lệch khỏi phương án đó khi việc điều chỉnh phong cách thực sự làm giảm hàm mất mát. Tính chất này giúp huấn luyện ổn định hơn so với các lớp chuẩn hóa mà giá trị khởi tạo ngẫu nhiên bắt buộc phải làm biến dạng đặc trưng.

Chi phí tham số. Mỗi lớp LADE bổ sung một ma trận $C \times C$, tức C^2 tham số. Với các lớp trong DTGAN có $C \in \{32, 64, 128\}$, chi phí tương ứng là 1.024, 4.096 và 16.384 tham số — rất nhỏ so với một lớp tích chập 3×3 cùng số kênh ($9C^2$ tham số). Tỷ lệ chi

phí phụ trội xấp xỉ:

$$\frac{\text{tham số LADE}}{\text{tham số Conv}_{3 \times 3}} = \frac{C^2}{9C^2} \approx 11,1\% \quad (2.7)$$

Chi phí tính toán. Ngoài tích chập 1×1 , LADE cần hai lần tính moments trên trục $[H, W]$ — các phép thu gọn (reduction) có độ phức tạp $O(BHWC)$, tuyến tính theo kích thước tensor và được tối ưu tốt trên GPU. Trên thực tế, phần tốn kém hơn là hai lần duyệt bộ nhớ để tính trung bình rồi phương sai, chứ không phải số phép nhân.

2.3.4. So sánh LADE với các kỹ thuật chuẩn hóa khác

Bảng 2.3. So sánh các kỹ thuật chuẩn hóa trong mạng chuyển đổi phong cách ảnh

Kỹ thuật	Nguồn tham số (γ, β)	Ưu điểm	Hạn chế
Batch Norm [12]	Thống kê batch	Ổn định huấn luyện	Phụ thuộc batch size
Instance Norm [19]	Tham số học, cố định	Tốt cho style transfer	Không thích ứng theo ảnh
AdaIN	Từ ảnh phong cách riêng	Linh hoạt phong cách	Cần ảnh style ở suy luận
LADE [56]	Conv 1×1 trên chính x	Tự thích ứng theo từng ảnh, không cần style riêng	Tăng C^2 tham số mỗi lớp

Sự khác biệt mấu chốt của LADE so với AdaIN là: *tham số tái tỷ lệ hóa được suy ra từ chính feature map hiện tại*, không phải từ một ảnh phong cách bên ngoài. Nhờ đó, mô hình có thể học phong cách nội tại của không gian đặc trưng một cách linh hoạt trong từng bước huấn luyện, và ở giai đoạn suy luận chỉ cần một đầu vào duy nhất — điều kiện bắt buộc để xuất mô hình sang ONNX với đồ thị tính toán tĩnh.

2.3.5. Biến thể LADE_D trong Discriminator

Trong các lớp Discriminator, LADE được mở rộng thành LADE_D: tích chập 1×1 trong bước chiếu tuyến tính được thay thế bằng `spectral_norm(Conv  $1 \times 1$ )` — tức là áp dụng thêm Spectral Normalization [32] để đảm bảo ràng buộc Lipschitz của Discriminator.

Lý do của biến thể này bắt nguồn từ phân tích ở Chương 1 (Mục 1.3.2): nếu Discriminator có hằng số Lipschitz không bị chặn, nó có thể tạo ra gradient rất lớn và biến động mạnh, đẩy quá trình huấn luyện vào trạng thái không hội tụ. Spectral Normalization chia ma trận trọng số cho giá trị kỳ dị lớn nhất:

$$\overline{W}_{\text{SN}} = \frac{W}{\sigma(W)}, \quad \sigma(W) = \max_{\|h\|_2 \neq 0} \frac{\|Wh\|_2}{\|h\|_2} \quad (2.8)$$

qua đó ràng buộc $\|\overline{W}_{\text{SN}}\|_2 = 1$ và làm cho toàn mạng có hằng số Lipschitz bị chặn.

Nhận xét: Việc áp *Spectral Normalization* lên chính lớp chiếu bên trong LADE — chứ không chỉ lên các tích chập chính — là một chi tiết dễ bị bỏ qua nhưng có logic rõ ràng. Trong LADE, đầu ra của lớp chiếu quyết định hệ số nhân $\sqrt{\sigma_t^2 + \varepsilon}$; nếu hệ số này không bị kiểm soát, một lớp chuẩn hóa hoàn toàn có thể khuếch đại đặc trưng lên nhiều lần và phá vỡ ràng buộc Lipschitz mà các lớp tích chập đã cẩn thận duy trì. Nói cách khác, chuẩn hóa phổ chỉ có ý nghĩa khi được áp **nhất quán trên mọi đường truyền tín hiệu**, kể cả những đường “phụ” nằm trong lớp chuẩn hóa. Theo quan điểm của luận văn, đây là ví dụ điển hình cho thấy chất lượng của một kiến trúc GAN thường nằm ở các chi tiết kỹ thuật nhỏ như vậy, chứ không chỉ ở sơ đồ khối tổng thể.

2.4. Cơ chế External Attention v3

2.4.1. Hạn chế của Self-Attention và Giải pháp

Cơ chế Self-Attention [38] đã được chứng minh hiệu quả trong việc nắm bắt phụ thuộc tầm xa (long-range dependency) trong ảnh — khả năng để một pixel “nhìn thấy” và tham chiếu tới mọi pixel khác, thay vì chỉ vùng lân cận như tích chập. Trong bài toán anime hóa, khả năng này rất cần thiết: màu tóc ở đỉnh đầu phải nhất quán với màu tóc ở gáy, tông sáng của má trái phải hài hòa với má phải.

Tuy nhiên, Self-Attention tính ma trận tương quan giữa mọi cặp vị trí:

$$\text{SelfAttn}(\mathcal{F}) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V, \quad Q, K, V \in \mathbb{R}^{N \times C} \quad (2.9)$$

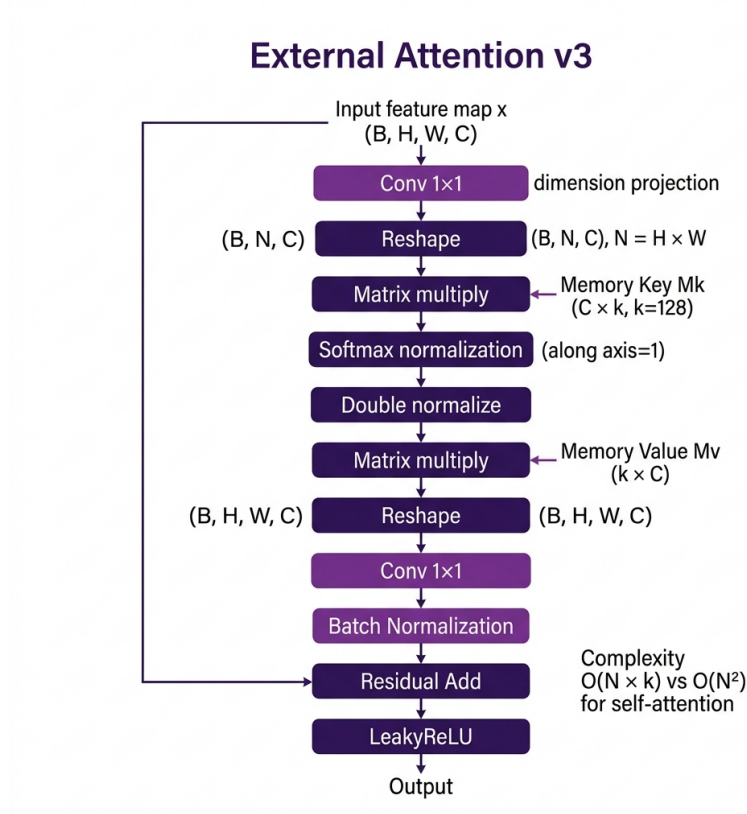
trong đó $N = H \times W$ là số vị trí không gian. Ma trận $QK^\top$ có kích thước $N \times N$, dẫn đến độ phức tạp:

$$\mathcal{O}(N^2C) \quad \text{về thời gian}, \quad \mathcal{O}(N^2) \quad \text{về bộ nhớ} \quad (2.10)$$

Với bottleneck 32×32 của DTGAN, $N = 1024$ và ma trận attention có $1024^2 \approx 1,05$ triệu phần tử — vẫn chấp nhận được. Nhưng nếu áp dụng ở độ phân giải 64×64 thì $N = 4096$ và số phần tử tăng lên $\approx 16,8$ triệu, gấp 16 lần. Chính đặc tính tăng trưởng bậc hai này khiến Self-Attention trở thành nút thắt cổ chai khi mở rộng độ phân giải.

External Attention, được đề xuất bởi Guo et al. [46], giải quyết vấn đề bằng cách thay thế cặp key–value *nội tại* (sinh từ chính ảnh) bằng hai **bộ nhớ học được** (learnable memory bank) M_k và M_v dùng chung cho toàn bộ tập dữ liệu, giảm độ phức tạp xuống còn $\mathcal{O}(NkC)$ với $k \ll N$.

2.4.2. Kiến trúc External Attention v3



Hình 2.3. Kiến trúc External Attention v3 sử dụng trong DTGAN tại vị trí bottleneck 32×32

Chú thích: Hai memory bank học được $M_k \in \mathbb{R}^{C \times k}$ và $M_v \in \mathbb{R}^{k \times C}$ cho phép mô hình học phụ thuộc toàn cục với độ phức tạp $O(N \cdot k)$ thay vì $O(N^2)$ của Self-Attention [46, 56].

Cụ thể, External Attention v3 trong DTGAN hoạt động theo các bước sau (áp dụng tại bottleneck 32×32 , $k = 128$):

1. $x \in \mathbb{R}^{B \times H \times W \times C} \xrightarrow{\text{Conv}_{1 \times 1}} \text{reshape thành } \mathcal{F} \in \mathbb{R}^{B \times N \times C}, N = H \times W$
2. **Attention weights:** $A = \text{Softmax}_{\text{axis}=1}(\mathcal{F} \cdot M_k) \in \mathbb{R}^{B \times N \times k}$
3. **Double normalization:** $A_{ij} \leftarrow A_{ij} / \sum_j A_{ij}$ (chuẩn hóa theo trục thứ hai)
4. $\mathcal{F}' = A \cdot M_v \in \mathbb{R}^{B \times N \times C}$, reshape về $\mathbb{R}^{B \times H \times W \times C}$
5. $\text{out} = \text{LeakyReLU}(x + \text{BN}(\text{Conv}_{1 \times 1}(\mathcal{F}')))$

Công thức tổng quát:

$$\text{ExternalAttention}(x) = \text{LeakyReLU}(x + \text{BN}(\text{normalize}(\mathcal{F} M_k) M_v)) \quad (2.11)$$

trong đó:

- $\mathcal{F} \in \mathbb{R}^{N \times C}$ — đặc trưng đã được làm phẳng theo không gian, mỗi hàng là vector đặc trưng C chiều của một vị trí pixel.
- $M_k \in \mathbb{R}^{C \times k}$ — *bộ nhớ khóa*, gồm k vector mẫu (prototype) trong không gian đặc trưng. Đây là tham số học được, **không phụ thuộc ảnh đầu vào**.
- $M_v \in \mathbb{R}^{k \times C}$ — *bộ nhớ giá trị*, gồm k vector đặc trưng đầu ra tương ứng với từng mẫu.
- $A \in \mathbb{R}^{N \times k}$ — ma trận trọng số chú ý; phần tử A_{ij} thể hiện mức độ pixel thứ i “thuộc về” mẫu thứ j .
- $k = 128$ — số lượng mẫu trong bộ nhớ, đóng vai trò siêu tham số điều tiết dung lượng biểu diễn.

Ý nghĩa từng bước:

- **Bước 2 — so khớp với từ điển mẫu:** Tích $\mathcal{F}M_k$ tính độ tương đồng giữa mỗi pixel và mỗi vector mẫu. Softmax theo trục N (trục pixel) trả lời câu hỏi: “*trong toàn bộ ảnh, những pixel nào đại diện tốt nhất cho mẫu j ?*”. Đây là điểm khác biệt căn bản so với Self-Attention: pixel không so sánh trực tiếp với pixel, mà cùng tham chiếu tới một tập mẫu chung.
- **Bước 4 — tái tổ hợp:** Tích AM_v tái tạo đặc trưng cho mỗi pixel bằng tổ hợp có trọng số của k vector giá trị. Do M_v dùng chung cho mọi ảnh, các pixel có ngữ nghĩa tương tự (cùng là “vùng da”, cùng là “đường viền tóc”) sẽ nhận đặc trưng đầu ra tương tự — **đây chính là cơ chế tạo ra tính nhất quán phong cách trên toàn ảnh và trên toàn tập dữ liệu**.
- **Bước 5 — kết nối dư (residual):** Cộng x vào kết quả đảm bảo module có thể suy biến về phép đồng nhất nếu attention không hữu ích, tương tự tính chất đã phân tích ở Mục 2.3.3.

2.4.3. Cơ chế Double Normalization

Điểm cải tiến đặc trưng của phiên bản v3 là chuẩn hóa **hai lần** trên ma trận A theo hai trục khác nhau:

$$\tilde{A}_{ij} = \frac{\exp(\alpha_{ij})}{\sum_{i'=1}^N \exp(\alpha_{i'j})} \quad (\text{chuẩn hóa theo trục pixel}), \quad \hat{A}_{ij} = \frac{\tilde{A}_{ij}}{\sum_{j'=1}^k \tilde{A}_{ij'}} \quad (\text{chuẩn hóa theo trục mẫu}) \quad (2.12)$$

với $\alpha = \mathcal{F}M_k$ là điểm tương đồng thô.

Vì sao cần bước thứ hai? Sau bước softmax theo trục pixel, mỗi *cột* của A có tổng bằng 1, nhưng mỗi *hàng* thì không. Điều này dẫn tới hai vấn đề:

- Một pixel có thể có tổng trọng số rất lớn (nếu nó tương đồng cao với nhiều mẫu), làm đặc trưng đầu ra của nó bị khuếch đại bất thường so với các pixel khác — sinh ra đốm sáng hoặc vệt màu trong ảnh kết quả.
- Ngược lại, một số mẫu trong bộ nhớ có thể “thống trị” toàn bộ ảnh, khiến các mẫu còn lại không nhận được gradient và dần trở nên vô dụng — hiện tượng tương tự *sự đổ mã* (codebook collapse) trong các mô hình lượng tử hóa vector.

Bước chuẩn hóa thứ hai buộc tổng trọng số của mỗi pixel bằng 1, biến $\hat{A}$ thành **ma trận tổ hợp lồi**: mỗi pixel đầu ra là một trung bình có trọng số của các vector trong M_v , không thể vượt ra ngoài bao lồi của chúng. Tính chất này chặn biên độ đầu ra một cách tự nhiên và phân bổ gradient đều hơn cho toàn bộ bộ nhớ.

2.4.4. Phân tích độ phức tạp

Bảng 2.4. So sánh độ phức tạp giữa Self-Attention và External Attention tại bottleneck 32×32 ($N = 1024$, $C = 128$, $k = 128$)

Cơ chế	Thời gian	Bộ nhớ ma trận attention	Tham số bổ sung
Self-Attention [38]	$\mathcal{O}(N^2C)$	$N^2 = 1.048.576$	$3C^2$ (cho Q, K, V)
External Attention [46]	$\mathcal{O}(NkC)$	$Nk = 131.072$	$2Ck$ (cho M_k, M_v)
Tỉ lệ	$k/N = 1/8$	giảm $8\times$	giảm $\approx 1,5\times$

Cần lưu ý rằng mức tiết kiệm phụ thuộc vào tỉ số k/N . Tại 32×32 với $k = 128$, tỉ số này là $1/8$ — lợi ích rõ nhưng chưa quá lớn. Lợi thế thực sự bộc lộ khi độ phân giải tăng: tại 64×64 ($N = 4096$), tỉ số trở thành $1/32$, và tại 128×128 là $1/128$. Nói cách khác, chi phí Self-Attention tăng bậc hai còn External Attention tăng *tuyến tính* theo số pixel.

Nhận xét: Nếu nhìn External Attention qua lăng kính đại số tuyến tính, nó chính là một *phép xấp xỉ hạng thấp* (low-rank approximation) của ma trận attention: thay vì ma trận $N \times N$ hạng đầy đủ, ta phân tích thành tích của hai ma trận $N \times k$ và $k \times N$ với $k \ll N$. Điều này đồng nghĩa với một sự đánh đổi rõ ràng: mô hình **mất khả năng biểu diễn các quan hệ pixel–pixel tùy ý**, đổi lại chi phí tuyến tính. Với bài toán tổng quát, đây có thể là mất mát đáng kể; nhưng với bài toán anime hóa, luận văn cho rằng đánh đổi này gần như miễn phí — vì thứ mà mô hình cần học không phải là quan hệ giữa từng cặp pixel cụ thể, mà là một **từ điển các loại vùng ảnh** (da, tóc, mắt, nền, viền) và cách tô màu tương ứng cho mỗi loại. Bộ nhớ ngoài M_k, M_v chính là hiện thân trực tiếp của từ điển đó. Đây là một ví dụ tốt cho nguyên tắc: cơ chế đơn giản hơn nhưng **khớp với cấu trúc của bài toán** thường thắng cơ chế tổng quát hơn nhưng tốn kém.

2.5. Discriminator (D_net)

DTGAN sử dụng hai Discriminator độc lập theo kiến trúc PatchGAN [25], mỗi mạng phục vụ một nhánh Generator:

- **Discriminator hỗ trợ (Support D):** Hoạt động trên ảnh xám một kênh. Mạng phân biệt giữa ảnh anime thật đã chuyển sang grayscale và ảnh $\hat{y}$ (generated) — sản phẩm của Support Tail sau khi qua Guided Filter. Việc loại bỏ thông tin màu ở đầu vào khiến Discriminator chỉ có thể căn cứ vào cấu trúc hình khối, tương phản sáng tối và hệ thống đường viền để đưa ra phán quyết. Hệ quả là tín hiệu gradient truyền ngược về Support Tail chỉ mang thông tin về các đặc trưng này, buộc nhánh tập trung học đường nét thay vì màu sắc.
- **Discriminator chính (Main D):** Hoạt động trên ảnh màu ba kênh (RGB). Mạng phân biệt giữa ảnh anime thật đã tiền xử lý bằng NL-Means kết hợp L0 Smoothing và ảnh màu $\hat{y}_m$ do Main Tail sinh ra. Do đầu vào giữ đầy đủ thông tin màu, Discriminator có thể phát hiện các sai lệch về phối màu, độ bão hòa và tính đồng nhất của các mảng màu — những yếu tố quyết định hiệu ứng cel-shading. Việc đối chiếu với ảnh anime đã khử nhiễu hạt giúp tín hiệu học tập trung vào phân bố màu theo vùng thay vì kết cấu bề mặt chi tiết.

Mỗi Discriminator gồm 7 lớp tích chập sử dụng LADE_D (LADE kết hợp Spectral Normalization), đầu ra là feature map một kênh theo phong cách PatchGAN.

Nguyên lý PatchGAN. Khác với Discriminator truyền thống trả về một số vô hướng cho toàn ảnh, PatchGAN trả về một bản đồ điểm số $D(\cdot) \in \mathbb{R}^{H' \times W'}$, trong đó mỗi phần tử đánh giá một vùng cục bộ của ảnh:

$$\mathcal{L}_{\text{patch}} = \frac{1}{H'W'} \sum_{u=1}^{H'} \sum_{v=1}^{W'} \ell(D(\cdot)_{u,v}) \quad (2.13)$$

trong đó $\ell(\cdot)$ là hàm mất mát điểm (ở DTGAN là bình phương sai lệch của LSGAN), và mỗi vị trí (u, v) có trường tiếp nhận tương ứng với một vùng ảnh gốc kích thước cố định. Thiết kế này có ba lợi ích trực tiếp cho bài toán anime hóa:

1. **Tín hiệu học dày đặc hơn:** thay vì một tín hiệu cho cả ảnh, Generator nhận $H' \times W'$ tín hiệu, giúp gradient giàu thông tin không gian và hội tụ nhanh hơn.
2. **Tập trung vào kết cấu cục bộ:** phong cách anime được xác định chủ yếu bởi đặc tính cục bộ (nét viền, mảng màu), không phải bố cục toàn cục — vốn phải được giữ nguyên từ ảnh gốc. PatchGAN vì vậy khớp với bản chất bài toán.
3. **Ít tham số hơn:** không cần lớp fully-connected ở cuối, đồng thời cho phép mạng xử lý ảnh ở kích thước tùy ý — điều kiện cần thiết cho Landscape Mode ở Chương 3, nơi ảnh đầu vào không cố định 256×256 .

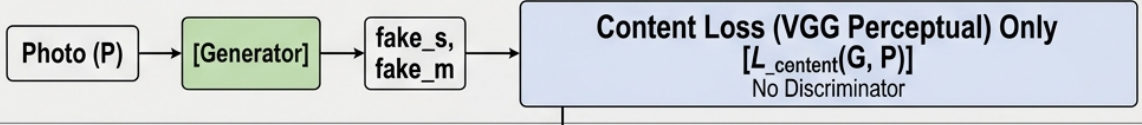
Phân tích thiết kế: Lựa chọn ép Support Discriminator làm việc trên ảnh xám là một can thiệp mang tính **ràng buộc thông tin** (information bottleneck) rất đáng chú ý. Thông thường, người thiết kế mạng có xu hướng cung cấp cho mô hình càng nhiều thông tin càng tốt. Ở đây, các tác giả làm ngược lại: chủ động cắt bỏ kênh màu để định hướng việc học. Cơ sở của thủ thuật này nằm ở chỗ Discriminator chỉ có thể phạt Generator dựa trên những gì nó quan sát được — nếu nó không thấy màu, nó không thể tạo ra gradient liên quan tới màu. Nói cách khác, **kiến trúc đầu vào của Discriminator chính là một cách đặc tả hàm mất mát**, và đây là công cụ định hướng học tinh tế hơn nhiều so với việc chỉ điều chỉnh trọng số các số hạng loss. Luận văn cho rằng đây là một trong những ý tưởng có tính chuyển giao cao nhất của DTGAN: nguyên tắc “giới hạn quan sát của giám khảo để chuyên biệt hóa thí sinh” có thể áp dụng cho nhiều bài toán sinh ảnh đa mục tiêu khác.

2.6. Các hàm mất mát được sử dụng trong kiến trúc

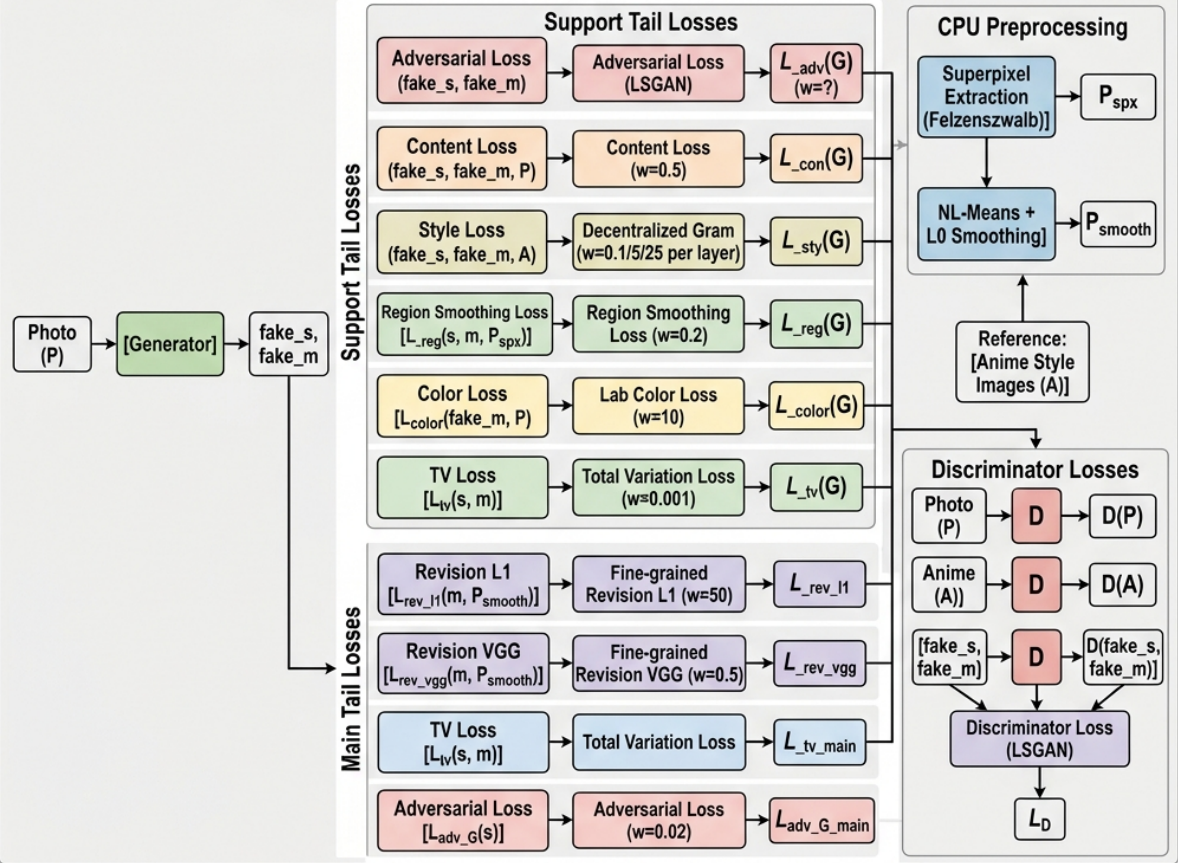
DTGAN sử dụng một hệ thống hàm mất mát đa thành phần, phản ánh mục tiêu kép của hai nhánh decoder. Hình 2.4 minh họa toàn bộ pipeline. Mỗi hàm mất mát dưới đây được trình bày theo format thống nhất: **Tên** → **Ý nghĩa/Tác dụng** → **Công thức** → **Giải thích**.

Input: Real Photo (P)

Stage 1 (Pre-training, epoch < 5)



Stage 2 (Full training, epoch ≥ 5)



Hình 2.4. Pipeline huấn luyện hai giai đoạn và hệ thống hàm mất mát của DTGAN

Chú thích: Giai đoạn 1 (pre-training) chỉ dùng Content Loss. Giai đoạn 2 kết hợp nhiều thành phần loss cho Support Tail và Main Tail, với pseudo ground-truth được tạo bởi CPU preprocessing [56]

2.6.1. Tổng quan hệ thống hàm mất mát

Trước khi đi vào từng thành phần, Bảng 2.5 tổng hợp toàn bộ hệ thống để tiện đối chiếu. Mỗi hàm mất mát trả lời một câu hỏi khác nhau về chất lượng ảnh sinh ra.

Bảng 2.5. Tổng hợp hệ thống hàm mất mát của DTGAN

Hàm mất mát	Câu hỏi cần trả lời	Áp dụng cho	Trọng số
Content $\mathcal{L}_{\text{con}}$	Ảnh sinh có giữ đúng nội dung gốc?	Cả hai nhánh	λ_{con}
Style $\mathcal{L}_{\text{sty}}$	Kết cấu có giống phong cách anime?	Support Tail	(0,1; 5,0; 25,0) theo tầng
Region Smoothing $\mathcal{L}_{\text{rs}}$	Vùng màu đã đủ phẳng?	Support Tail	0,2 mỗi số hạng
Lab Color $\mathcal{L}_{\text{color}}$	Màu có bị trôi khỏi ảnh gốc?	Support Tail	$\lambda_{\text{col}} = 10,0$
Total Variation $\mathcal{L}_{\text{TV}}$	Ảnh có nhiều pixel cục bộ?	Cả hai nhánh	$\lambda_{\text{TV}} = 0,001$
Adversarial $\mathcal{L}_{\text{adv}}$	Discriminator có bị đánh lừa?	Cả hai nhánh	1,0 (S) / 0,02 (M)
Revision $\mathcal{L}_{\text{rev}}$	Có bám sát pseudo ground-truth?	Main Tail	$\lambda_{r1} = 50,0; \lambda_{rv} = 0,5$

2.6.2. Hàm mất mát Giai đoạn Pre-training

Trong các epoch đầu ($\text{epoch} < \text{init_G_epoch} = 5$), chỉ huấn luyện Generator với Content Loss thuần túy (không có Discriminator):

$$\mathcal{L}_{\text{pre}} = \mathcal{L}_{\text{con}}(x, \hat{y}_s) + \mathcal{L}_{\text{con}}(x, \hat{y}_m) \quad (2.14)$$

Ở giai đoạn này, mục tiêu duy nhất là để cả hai nhánh học tái tạo ảnh đầu vào ở mức đặc trưng ngữ nghĩa — nói cách khác, Generator được huấn luyện như một **bộ tự mã hóa** (autoencoder). Điều này giải quyết một vấn đề cụ thể trong huấn luyện GAN: nếu kích hoạt tín hiệu đối kháng ngay từ bước đầu, Generator (với trọng số khởi tạo ngẫu nhiên) sinh ra ảnh nhiễu hoàn toàn, Discriminator phân biệt được ngay lập tức với độ chính xác gần 100%, và như đã phân tích ở Chương 1 (Mục 1.3.2), gradient truyền về Generator trở nên rất nhỏ — hiện tượng biến mất gradient ở giai đoạn đầu.

2.6.3. Content Loss (Perceptual Loss)

Content Loss được tính thông qua mạng VGG19 [13] tại lớp conv4_4 (512 kênh):

$$\mathcal{L}_{\text{con}}(x, \hat{y}) = \frac{1}{C_l H_l W_l} \|\phi_l(x) - \phi_l(\hat{y})\|_1 \quad (2.15)$$

trong đó:

- $\phi_l(\cdot)$ — ánh xạ trích xuất đặc trưng tại lớp l của mạng VGG19 đã huấn luyện sẵn trên ImageNet; trọng số VGG được **đóng băng**, không cập nhật trong quá trình huấn luyện.

- C_l, H_l, W_l — số kênh, chiều cao, chiều rộng của bản đồ đặc trưng tại lớp l ; tích của chúng dùng để chuẩn hóa, giúp giá trị loss không phụ thuộc kích thước tầng.
- $\| \cdot \|_1$ — chuẩn L_1 (tổng trị tuyệt đối của các sai lệch).

Vì sao so sánh ở không gian đặc trưng thay vì không gian pixel? Nếu dùng $\|x - \hat{y}\|_1$ trực tiếp trên pixel, hàm mất mát sẽ phạt *mọi* thay đổi — bao gồm cả những thay đổi mà ta *mong muốn* (làm phẳng màu, đơn giản hóa kết cấu). Kết quả là ảnh đầu ra sẽ gần như trùng ảnh gốc và không hề mang phong cách anime. Ngược lại, đặc trưng tại tầng sâu như conv4_4 mã hóa thông tin ngữ nghĩa (“đây là mắt”, “đây là đường viền mặt”) và tương đối bất biến với thay đổi màu sắc, kết cấu bề mặt. Do đó $\mathcal{L}_{\text{con}}$ ràng buộc “*vẫn là khuôn mặt đó*” mà không ràng buộc “*vẫn là những pixel đó*”.

Vì sao chọn tầng conv4_4? Đây là điểm cân bằng: các tầng nông (conv1, conv2) còn mang nhiều thông tin kết cấu và màu, ràng buộc quá chặt; các tầng quá sâu (conv5) trừu tượng đến mức mất thông tin vị trí, khiến ảnh đầu ra có thể biến dạng hình học.

Vì sao dùng chuẩn L_1 thay vì L_2 ? Chuẩn L_2 phạt bình phương sai lệch nên rất nhạy với các giá trị ngoại lai; nghiệm tối ưu của nó là *trung bình* của các khả năng, dẫn tới hiện tượng ảnh bị mờ khi bài toán có nhiều lời giải hợp lệ (như trường hợp chuyển phong cách). Chuẩn L_1 có nghiệm tối ưu là *trung vị*, ít bị kéo bởi ngoại lai và cho ảnh sắc nét hơn [15].

2.6.4. Style Loss (Decentralized Gram Matrix)

Style Loss là thành phần giúp Generator học phân phối phong cách anime thông qua ma trận Gram. Ý tưởng nền tảng: **ma trận Gram đo mức độ đồng xuất hiện giữa các kênh đặc trưng**. Nếu kênh “phát hiện cạnh dọc” và kênh “phát hiện màu vàng nhạt” thường kích hoạt cùng nhau, phần tử Gram tương ứng sẽ lớn — và đó chính là một đặc điểm phong cách. Quan trọng là phép tính này *tổng hợp trên toàn bộ vị trí không gian*, nên nó nắm bắt “chất liệu” của ảnh mà bỏ qua “ảnh vẽ cái gì”.

Điểm đặc biệt của DTGAN là sử dụng **Decentralized Gram Matrix** — trừ đi trung bình của activations trước khi tính Gram:

$$G_l^{\text{dec}}(x) = \frac{1}{C_l N_l} (\phi_l(x) - \bar{\phi}_l(x))^{\top} (\phi_l(x) - \bar{\phi}_l(x)) \quad (2.16)$$

trong đó $\bar{\phi}_l(x) = \frac{1}{N_l} \sum_i \phi_l(x)_i$ là trung bình của activation map tại lớp l , và $N_l = H_l W_l$ là số vị trí không gian.

Vì sao phải trừ trung bình? Ma trận Gram cổ điển $\phi^{\top} \phi$ thực chất là **ma trận moment bậc hai chưa tâm hóa**, nên nó lẫn lộn hai loại thông tin: (i) mức kích hoạt trung bình của từng kênh, và (ii) sự *biến thiên tương quan* giữa các kênh. Với bài toán

anime hóa, thành phần (i) chủ yếu phản ánh độ sáng tổng thể — mà ảnh anime thường sáng và bão hòa hơn hẳn ảnh chụp thật. Nếu không tâm hóa, Style Loss sẽ bị chi phối bởi chênh lệch độ sáng và Generator sẽ “ăn gian” bằng cách chỉ tăng độ sáng toàn ảnh thay vì học kết cấu anime thật sự. Sau khi trừ trung bình, G^{dec} trở thành **ma trận hiệp phương sai giữa các kênh**, chỉ giữ lại thông tin tương quan — đúng thứ định nghĩa phong cách.

Loss style tổng hợp từ ba lớp VGG19 (conv2_2, conv3_4, conv4_4) với trọng số tăng dần:

$$\mathcal{L}_{\text{sty}} = \lambda_{s1} \|G_2^{\text{dec}}\| + \lambda_{s2} \|G_3^{\text{dec}}\| + \lambda_{s3} \|G_4^{\text{dec}}\| \quad (2.17)$$

với $(\lambda_{s1}, \lambda_{s2}, \lambda_{s3}) = (0,1; 5,0; 25,0)$.

Ý nghĩa của bộ trọng số tăng dần. Chênh lệch $250\times$ giữa tầng nông nhất và tầng sâu nhất không phải ngẫu nhiên. Tầng conv2_2 mã hóa kết cấu vi mô (hạt nhiều, chi tiết da); nếu đặt trọng số cao, mô hình sẽ sao chép kết cấu vẽ tay ở mức pixel và sinh ra nhiều dạng hạt. Tầng conv4_4 mã hóa các mô-típ phong cách ở phạm vi rộng (cách xử lý mảng sáng tối, cách nhóm màu) — đây mới là thứ cần học mạnh. Ngoài ra, giá trị tuyệt đối của Gram tại tầng nông thường lớn hơn nhiều do số kênh ít và activation dày đặc, nên trọng số nhỏ cũng có tác dụng **cân bằng thang đo** giữa các tầng.

2.6.5. Region Smoothing Loss

Một trong những đặc trưng quan trọng của phong cách anime là **các vùng màu phẳng**, tối giản chi tiết kết cấu. Region Smoothing Loss được thiết kế để khuyến khích tính chất này bằng cách sử dụng **Superpixel Segmentation** (thuật toán Felzenszwalb [3]) để tạo phiên bản “tối giản” x_{spx} của ảnh đầu vào:

$$\mathcal{L}_{\text{rs}} = 0,2 \cdot \mathcal{L}_{\text{con}}(x_{\text{spx}}, \hat{y}_s) + 0,2 \cdot \mathcal{L}_{\text{con}}(x_{\text{spx}}, \hat{y}) \quad (2.18)$$

trong đó $\hat{y}_s$ là đầu ra thô của Support Tail và $\hat{y}$ là đầu ra sau Guided Filter. Việc ràng buộc *cả hai* phiên bản đảm bảo tính phẳng vùng được duy trì trước và sau khâu lọc, tránh trường hợp Guided Filter phải “gánh” toàn bộ nhiệm vụ làm mịn.

Cách tạo x_{spx} . Thuật toán Felzenszwalb phân chia ảnh thành các vùng đồng nhất về màu sắc và cường độ (superpixel) bằng cách xây dựng đồ thị lân cận trên pixel và gộp các vùng có chênh lệch nội vùng nhỏ hơn chênh lệch liên vùng. Sau khi phân vùng, mỗi superpixel được thay thế bằng **màu trung bình** của vùng đó. Kết quả là một phiên bản “đã tô phẳng” của ảnh gốc — rất gần với hiệu ứng cel-shading, nhưng vẫn giữ nguyên bố cục và ranh giới các đối tượng.

Vì sao dùng $\mathcal{L}_{\text{con}}$ (đặc trưng VGG) mà không so sánh pixel trực tiếp? Nếu buộc ảnh đầu ra trùng khớp x_{spx} ở mức pixel, mô hình sẽ sao chép luôn các *lỗi phân vùng* của

thuật toán Felzenszwalb (biên vùng răng cưa, vùng bị gộp sai). Việc so sánh ở không gian đặc trưng chỉ truyền đạt thông điệp “*hãy phẳng như thế này*” mà không áp đặt từng ranh giới cụ thể — một dạng giám sát mềm.

Vì sao trọng số chỉ 0,2? Đây là số hạng có tính chất “gợi ý xu hướng” chứ không phải ràng buộc cứng. Nếu đặt trọng số cao, mô hình sẽ làm phẳng cả những vùng cần giữ chi tiết — đặc biệt là mắt và tóc trong ảnh chân dung.

2.6.6. Lab Color Loss

Để bảo toàn màu sắc tự nhiên từ ảnh gốc (tránh hiện tượng trôi màu — color drift — trong quá trình chuyển đổi phong cách), DTGAN sử dụng Lab Color Loss trong không gian màu CIE Lab:

$$\mathcal{L}_{\text{color}} = 2 \cdot \|L(x) - L(\hat{y})\|_1 + \|a(x) - a(\hat{y})\|_1 + \|b(x) - b(\hat{y})\|_1 \quad (2.19)$$

trong đó:

- $L(\cdot) \in [0, 100]$ — kênh độ sáng (lightness), 0 là đen tuyệt đối, 100 là trắng.
- $a(\cdot)$ — trục màu đối lập xanh lục $\leftrightarrow$ đỏ.
- $b(\cdot)$ — trục màu đối lập xanh lam $\leftrightarrow$ vàng.

Vì sao là Lab mà không phải RGB? Trong không gian RGB, ba kênh có tương quan cao và đều mang thông tin độ sáng lẫn sắc độ trộn lẫn — không thể tách riêng để đặt trọng số. Không gian CIE Lab được thiết kế theo nguyên tắc *đồng đều về cảm nhận* (perceptually uniform): khoảng cách Euclid giữa hai màu trong Lab tỉ lệ xấp xỉ với mức độ khác biệt mà mắt người cảm nhận được. Quan trọng hơn, Lab **tách bạch độ sáng khỏi sắc độ**, cho phép hàm mất mát ràng buộc hai khía cạnh này với mức độ khác nhau — đúng nhu cầu của bài toán: giữ chặt cấu trúc sáng–tối (vốn xác định hình khối và danh tính khuôn mặt) nhưng nới lỏng sắc độ (để mô hình được tự do dịch màu theo bảng màu anime).

Vì sao kênh L có trọng số gấp đôi? Sai lệch độ sáng gây biến dạng hình khối và làm thay đổi đặc điểm nhận dạng khuôn mặt, trong khi sai lệch sắc độ ở mức vừa phải lại là điều *mong muốn* khi anime hóa. Hệ số 2 mã hóa chính thứ tự ưu tiên này. Trọng số toàn cục $\lambda_{\text{col}} = 10,0$.

2.6.7. Total Variation Loss

Total Variation (TV) Loss thúc đẩy tính mịn không gian của ảnh đầu ra, giảm nhiễu pixel cục bộ:

$$\mathcal{L}_{\text{TV}}(\hat{y}) = \frac{1}{HW} \sum_{i,j} \left(\|\nabla_h \hat{y}_{i,j}\|^2 + \|\nabla_w \hat{y}_{i,j}\|^2 \right) \quad (2.20)$$

trong đó $\nabla_h \hat{y}_{i,j} = \hat{y}_{i+1,j} - \hat{y}_{i,j}$ và $\nabla_w \hat{y}_{i,j} = \hat{y}_{i,j+1} - \hat{y}_{i,j}$ là sai phân hữu hạn theo chiều dọc và chiều ngang — xấp xỉ rời rạc của gradient ảnh.

Hàm này phạt tổng bình phương biến thiên giữa các pixel kề nhau, do đó khuyến khích ảnh đầu ra “ít gợn”. Trọng số được đặt rất nhỏ, $\lambda_{TV} = 0,001$, và điều này hoàn toàn có chủ ý: TV Loss về bản chất **đôi kháng với mục tiêu đường nét sắc**. Nếu tăng trọng số, ảnh sẽ mịn nhưng các đường viền anime — vốn là những vùng gradient lớn — sẽ bị làm mờ. Vai trò của TV ở đây chỉ là khử các đốm nhiễu đơn lẻ do tích chập sinh ra, chứ không phải làm mịn tổng thể (nhiệm vụ đó đã thuộc về Region Smoothing Loss và Guided Filter).

2.6.8. Adversarial Loss (LSGAN)

DTGAN sử dụng biến thể LSGAN (Least Squares GAN) [28] thay vì hàm mất mát entropy chéo, giúp tránh vấn đề biến mất gradient khi Discriminator hội tụ. Loss cho Generator:

$$\mathcal{L}_{adv}^G = \mathbb{E}[(D(\hat{y}) - 1)^2] \quad (2.21)$$

Loss cho Discriminator Support:

$$\mathcal{L}_D^s = \underbrace{\mathbb{E}[(D(y_{\text{anime}}) - 1)^2]}_{(1) \text{ ảnh anime thật} \rightarrow 1} + \underbrace{\mathbb{E}[D(\hat{y})^2]}_{(2) \text{ ảnh sinh} \rightarrow 0} + \underbrace{2,0 \cdot \mathbb{E}[D(y_{\text{smooth}})^2]}_{(3) \text{ ảnh anime mờ nét} \rightarrow 0} \quad (2.22)$$

trong đó y_{anime} là ảnh anime thật, $\hat{y}$ là ảnh do Generator sinh ra, và y_{smooth} là ảnh anime đã được làm mờ vùng biên (edge-smoothed).

Ý nghĩa từng số hạng:

- **Số hạng (1) và (2)** là cặp đối kháng chuẩn: Discriminator học gán điểm 1 cho ảnh thật và 0 cho ảnh giả.
- **Số hạng (3)** là điểm đặc sắc kế thừa từ CartoonGAN [31]. Tập y_{smooth} được tạo bằng cách lấy chính ảnh anime thật rồi làm mờ riêng vùng biên. Ảnh này giống ảnh anime thật ở mọi phương diện — màu sắc, bố cục, phân bố vùng — **ngoại trừ độ sắc của đường nét**. Bằng cách buộc Discriminator gán điểm 0 cho chúng, ta cô lập được đúng một đặc trưng để phạt: sự thiếu sắc nét ở đường viền. Không có số hạng này, Generator hoàn toàn có thể đạt điểm cao chỉ bằng cách khớp phân bố màu mà bỏ qua đường nét — đúng nhược điểm thường thấy ở các mô hình anime hóa thể hệ trước.
- **Hệ số 2,0** khiến mức phạt cho “ảnh mờ nét” nặng gấp đôi “ảnh giả thông thường”, phản ánh mức độ ưu tiên của đường nét trong phong cách anime.

Vì sao chọn LSGAN thay vì entropy chéo hoặc WGAN-GP? Như đã phân tích ở Chương 1 (Mục 1.3.3.4 và Mục 1.3.3.2), hàm entropy chéo bão hòa khi Discriminator

quá mạnh: khi $D(\hat{y}) \rightarrow 0$, gradient $\partial \log(1 - D)/\partial D$ trở nên rất nhỏ và Generator ngừng học. LSGAN dùng sai số bình phương, nên gradient tỉ lệ *tuyến tính* với khoảng cách tới mục tiêu — mẫu càng bị phân loại sai càng nhận gradient lớn, không bao giờ bão hòa. So với WGAN-GP, LSGAN có ưu thế thực tiễn rõ rệt: không cần tính gradient penalty (vốn đòi hỏi đạo hàm bậc hai, làm chậm mỗi bước huấn luyện đáng kể) và không cần huấn luyện Discriminator nhiều bước cho mỗi bước Generator. Với một mô hình hướng tới triển khai nhẹ và huấn luyện được trên một GPU đơn như DTGAN, đây là đánh đổi hợp lý.

2.6.9. Fine-grained Revision Loss (Main Tail)

Thành phần loss đặc trưng nhất của Main Tail là **Fine-grained Revision Loss**, sử dụng pseudo ground-truth x_{smooth} tạo bởi NL-Means Denoising [5] kết hợp với L0 Gradient Minimization [7]:

$$\mathcal{L}_{\text{rev}} = \lambda_{r1} \cdot \|x_{\text{smooth}} - \hat{y}_m\|_1 + \lambda_{rv} \cdot \mathcal{L}_{\text{con}}(x_{\text{smooth}}, \hat{y}_m) \quad (2.23)$$

với $\lambda_{r1} = 50,0$ và $\lambda_{rv} = 0,5$.

Cách tạo pseudo ground-truth. Hai bước tiền xử lý bổ trợ cho nhau:

- **NL-Means Denoising** khử nhiễu bằng cách thay mỗi pixel bằng trung bình có trọng số của các pixel có *vùng lân cận tương tự* trên toàn ảnh. Bước này loại bỏ hạt nhiễu cảm biến và kết cấu da mà vẫn giữ cấu trúc.
- **L0 Gradient Minimization** tối thiểu hóa *số lượng* pixel có gradient khác 0 (chuẩn L_0), thay vì tổng độ lớn gradient như TV Loss. Hệ quả toán học rất khác biệt: chuẩn L_1/L_2 khuyến khích mọi gradient nhỏ lại (làm mờ đều), còn chuẩn L_0 khuyến khích *phần lớn* gradient bằng 0 nhưng *cho phép một số ít gradient rất lớn* — tức là đúng cấu trúc “vùng phẳng ngăn cách bởi biên sắc” của tranh anime.

Ý nghĩa của tỉ lệ trọng số 50,0 so với 0,5. Chênh lệch $100\times$ cho thấy Main Tail được huấn luyện chủ yếu như một bài toán **hồi quy có giám sát** chứ không phải sinh ảnh đối kháng. Số hạng L_1 trên pixel đảm bảo bám sát chính xác pseudo ground-truth; số hạng perceptual với trọng số nhỏ chỉ đóng vai trò ràng buộc nhất quán ngữ nghĩa, ngăn ảnh trôi khỏi nội dung gốc khi L_1 có nhiều nghiệm tương đương. Kết hợp với $\lambda_{\text{adv}}^m = 0,02$ ở phần sau, có thể thấy tín hiệu đối kháng đối với Main Tail chỉ mang tính “điều chỉnh tinh”, chiếm tỉ trọng rất nhỏ.

2.6.10. Tổng hợp Loss

$$\mathcal{L}_G^{\text{support}} = \mathcal{L}_{\text{adv}}^s + \lambda_{\text{con}} \cdot \mathcal{L}_{\text{con}} + \mathcal{L}_{\text{sty}} + \mathcal{L}_{\text{rs}} + \lambda_{\text{col}} \cdot \mathcal{L}_{\text{color}} + \lambda_{\text{TV}} \cdot \mathcal{L}_{\text{TV}}(\hat{y}) \quad (2.24)$$

$$\mathcal{L}_G^{\text{main}} = \lambda_{\text{adv}}^m \cdot \mathcal{L}_{\text{adv}}^m + \mathcal{L}_{\text{rev}} + \lambda_{\text{TV}} \cdot \mathcal{L}_{\text{TV}}(\hat{y}_m) \quad (2.25)$$

Tổng loss Generator: $\mathcal{L}_G = \mathcal{L}_G^{\text{support}} + \mathcal{L}_G^{\text{main}}$, với $\lambda_{\text{adv}}^m = 0,02$.

Thảo luận về cân bằng trọng số: Hệ thống bảy hàm mất mát với các trọng số trải trên năm bậc độ lớn — từ 0,001 (TV) đến 50,0 (Revision L_1) — là điểm vừa mạnh vừa mong manh nhất của DTGAN. Mạnh, vì mỗi số hạng phụ trách đúng một khía cạnh chất lượng và có thể điều chỉnh độc lập. Mong manh, vì bộ trọng số này gắn chặt với đặc điểm của tập dữ liệu huấn luyện. Kết quả thực nghiệm ở Chương 4 minh họa rõ điều này: các thành phần loss ổn định ở những mức rất khác nhau ($\text{style} \approx 3,42$; $\text{color} \approx 0,67$; $\text{content} \approx 0,19$; $\text{region smoothing} \approx 0,20$), nghĩa là chúng **không đóng góp ngang nhau** vào gradient tổng — Style Loss chi phối phần lớn tín hiệu học. Một hệ quả thực tiễn quan trọng: khi chuyển sang tập phong cách khác (ví dụ từ Hayao sang Shinkai với tông màu và độ tương phản rất khác), bộ trọng số tối ưu nhiều khả năng cũng phải thay đổi. Theo quan điểm của luận văn, đây là một hạn chế nội tại của các phương pháp dựa trên tổ hợp tuyến tính nhiều hàm mất mát, và là lý do khiến việc **mở rộng sang mô hình đa phong cách** (đề xuất ở Chương 5) không thể chỉ đơn thuần là huấn luyện lại trên dữ liệu mới. Các hướng nghiên cứu về cân bằng trọng số tự động — như chuẩn hóa gradient hay học đa nhiệm với trọng số bất định — là một hướng cải tiến đáng cân nhắc.

2.7. Xử lý Hậu kỳ: Guided Filter

Guided Filter [8] là bộ lọc ảnh có bảo toàn cạnh (edge-preserving), được áp dụng lên output của Support Tail trước khi đưa vào Discriminator:

$$\hat{y} = \tanh\left(s \cdot \text{GuidedFilter}\left(\sigma\left(\frac{\hat{y}_s}{s}\right), \sigma\left(\frac{\hat{y}_s}{s}\right), r=2, \varepsilon=0,01\right)\right) \quad (2.26)$$

trong đó:

- $s = 2,5$ — hệ số tỷ lệ, dùng để nén giá trị vào vùng tuyến tính của hàm sigmoid trước khi lọc và giãn trở lại sau khi lọc.
- $\sigma(\cdot)$ — hàm sigmoid, chuyển giá trị từ miền $(-\infty, \infty)$ về $[0, 1]$ — miền làm việc chuẩn của Guided Filter.
- $r = 2$ — bán kính của sổ lọc (tính theo pixel).
- $\varepsilon = 0,01$ — tham số điều hòa quyết định ngưỡng phân biệt “nhiều” và “cạnh”.

Nguyên lý hoạt động. Guided Filter giả định đầu ra là một hàm tuyến tính cục bộ của ảnh dẫn hướng I trong mỗi cửa sổ ω_p bán kính r :

$$q_i = a_p I_i + b_p, \quad \forall i \in \omega_p \quad (2.27)$$

với hệ số (a_p, b_p) được xác định bằng hồi quy tuyến tính trên cửa sổ:

$$a_p = \frac{\text{cov}_{\omega_p}(I, p)}{\text{var}_{\omega_p}(I) + \varepsilon}, \quad b_p = \bar{p}_{\omega_p} - a_p \bar{I}_{\omega_p} \quad (2.28)$$

Công thức (2.28) giải thích trọn vẹn tính chất bảo toàn cạnh. Xét hai trường hợp:

- **Trong vùng phẳng** ($\text{var}_{\omega_p}(I) \ll \varepsilon$): mẫu số bị ε chi phối, $a_p \rightarrow 0$ và $q_i \rightarrow \bar{p}_{\omega_p}$ — bộ lọc hành xử như lọc trung bình, làm mịn nhiều.
- **Tại vùng có cạnh** ($\text{var}_{\omega_p}(I) \gg \varepsilon$): $a_p \rightarrow 1$ và $q_i \rightarrow I_i$ — bộ lọc gần như không can thiệp, cạnh được giữ nguyên.

Nói cách khác, ε đóng vai trò **ngưỡng phân định**: mọi biến thiên có phương sai nhỏ hơn ε bị coi là nhiễu và bị làm phẳng, mọi biến thiên lớn hơn được coi là cạnh và được bảo toàn. Với $\varepsilon = 0,01$ trên thang $[0, 1]$, ngưỡng này tương ứng với độ lệch chuẩn 0,1 — đủ để loại bỏ gọn nhiễu do tích chập mà không chạm tới đường viền anime.

Vì sao dùng chính $\hat{y}_s$ làm cả ảnh dẫn hướng lẫn ảnh cần lọc? Đây là chế độ *tự dẫn hướng* (self-guided), trong đó bộ lọc chỉ dựa vào cấu trúc nội tại của ảnh cần xử lý. Trong DTGAN, mục tiêu không phải chuyển giao cấu trúc từ một ảnh khác mà chỉ là làm sạch nhiễu trong khi giữ nét — nên tự dẫn hướng là lựa chọn đúng và cũng đơn giản nhất về mặt tính toán.

Nhận xét: Điều đáng chú ý là *Guided Filter* được đặt **bên trong đồ thị tính toán** chứ không phải sau khi suy luận xong. Toàn bộ phép lọc là khả vi, nên gradient từ *Support Discriminator* truyền ngược qua bộ lọc về tới *Support Tail*. Hệ quả là *Support Tail* không học sinh ra ảnh đẹp, mà học sinh ra ảnh sẽ đẹp sau khi lọc — một mục tiêu khác hẳn. Cách bố trí này thể hiện một nguyên tắc thiết kế đáng ghi nhận: khi ta biết trước hệ thống sẽ có một bước xử lý cố định ở cuối, việc đưa bước đó vào vòng huấn luyện luôn tốt hơn là để mạng học một mục tiêu rồi mới áp thêm phép biến đổi. Cũng cần lưu ý rằng do *Main Tail* — nhánh thực sự được dùng khi suy luận — **không** đi qua *Guided Filter*, chất lượng đường nét của sản phẩm cuối phụ thuộc vào việc tri thức về nét mà *Support Tail* học được có lan truyền hiệu quả sang *Base Encoder* dùng chung hay không. Đây chính là mắt xích giả định quan trọng nhất trong toàn bộ kiến trúc *Double-Tail*.

2.8. Quy trình Huấn luyện

2.8.1. Hai giai đoạn huấn luyện

Quá trình huấn luyện DTGAN được chia thành hai giai đoạn rõ ràng:

Giai đoạn 1 — Pre-training Generator (epoch < 5): Chỉ cập nhật Generator với Content Loss (Phương trình 2.14). Mục tiêu là giúp Generator nhanh chóng hội tụ về biểu diễn nội dung ổn định trước khi kích hoạt huấn luyện đối nghịch.

Giai đoạn 2 — Huấn luyện đầy đủ (epoch ≥ 5): Mỗi bước lặp gồm 4 sub-step:

1. **Forward pass:** Tính $\hat{y}_s, \hat{y}, \hat{y}_m$ từ Generator.
2. **CPU preprocessing (song song):** Tính x_{spx} bằng Felzenszwalb superpixel trên CPU và x_{smooth} bằng NL-Means + L0 Smoothing.
3. **Cập nhật Generator:** Tính tất cả thành phần loss (Phương trình 2.24, 2.25) và lan truyền ngược.
4. **Cập nhật Discriminators:** Cập nhật Support D và Main D.

Việc tính superpixel và smoothing trên CPU song song với GPU forward pass là một kỹ thuật tối ưu hóa quan trọng. Cả hai thuật toán này đều là thuật toán thị giác máy tính cổ điển chạy trên CPU với chi phí không nhỏ — nếu thực hiện tuần tự, GPU sẽ nhàn rỗi trong suốt thời gian đó. Bằng cách phát lệnh tiền xử lý CPU ngay sau khi khởi động forward pass trên GPU, hai luồng công việc chồng lấp thời gian và tổng thời gian mỗi vòng lặp giảm đáng kể.

2.8.2. Siêu tham số huấn luyện

Bảng 2.6. Siêu tham số huấn luyện của DTGAN trong thực nghiệm của luận văn

Siêu tham số	Giá trị	Ghi chú
Kích thước ảnh	256×256	Chung cho cả Landscape và Face Mode
Batch size	8	Giới hạn bởi VRAM 24 GB
Init G epochs	5	Giai đoạn pre-training
Tổng epochs (cấu hình)	100	Giá trị mặc định trong file cấu hình
Tổng epochs (thực tế)	74	Dừng tại epoch 73 khi loss đã ổn định (xem Chương 4)
Learning rate (G)	1×10^{-4}	
Learning rate (D)	1×10^{-4}	Bằng LR của G, không dùng TTUR
Learning rate (Init G)	2×10^{-4}	Cao hơn trong pre-training
Optimizer	Adam	$\beta_1 = 0,5, \beta_2 = 0,999$
Framework	TensorFlow 1.x	GPU NVIDIA

Diễn giải một số lựa chọn:

- $\beta_1 = 0,5$ **thay vì 0,9 mặc định:** Đây là quy ước gần như bắt buộc trong huấn luyện GAN kể từ DCGAN. Hệ số momentum thấp làm giảm “quán tính” của bộ tối ưu, giúp Generator phản ứng nhanh hơn với sự thay đổi của Discriminator. Với β_1 cao, gradient tích lũy từ các bước cũ — vốn được tính khi Discriminator còn ở trạng thái khác — sẽ gây dao động và làm chậm hội tụ.

- **Learning rate của pre-training cao gấp đôi:** Ở giai đoạn 1, bài toán tối ưu là hồi quy đơn mục tiêu, ổn định và không có đối kháng, nên có thể dùng bước học lớn để hội tụ nhanh. Khi chuyển sang giai đoạn 2, bước học được hạ xuống để tránh phá vỡ cân bằng đối kháng vốn nhạy cảm.
- **Batch size 8:** Nhỏ so với chuẩn phân loại ảnh, nhưng hợp lý ở đây vì (i) toàn bộ hệ thống dùng chuẩn hóa theo instance (LADE), không phụ thuộc thống kê batch; và (ii) chi phí bộ nhớ cho ba mạng con cùng VGG19 đóng băng là rất lớn.
- **Không dùng TTUR:** Kỹ thuật hai tốc độ học (Two Time-scale Update Rule) đặt LR của Discriminator cao hơn Generator để ổn định huấn luyện. DTGAN không dùng kỹ thuật này, có thể vì các cơ chế ổn định khác (LSGAN, Spectral Normalization, pre-training) đã đủ — điều này được xác nhận bởi kết quả thực nghiệm ở Chương 4: D_loss ổn định quanh 0,11 và không xảy ra sụp đổ mode.

2.8.3. Tiền xử lý Dữ liệu Huấn luyện

Bộ dữ liệu huấn luyện bao gồm hai thành phần:

Ảnh phong cách anime (Style): Các frame ảnh từ phim hoạt hình Hayao Miyazaki hoặc Makoto Shinkai. Trước khi huấn luyện, ảnh anime được xử lý bằng **Edge Smoothing** để tạo tập `smooth/` — làm mờ nhẹ vùng biên trong ảnh anime, tạo ra tập “phản ví dụ” phục vụ số hạng (3) trong Phương trình (2.22).

Ảnh thực (Photo): Các ảnh chân dung. Được xử lý để tạo `seg_train_5-0.8-50/` bằng Felzenszwalb superpixel với tham số ($\sigma = 5$, $k = 0,8$, $\text{min_size} = 50$), phục vụ Region Smoothing Loss.

Cả hai tập tiền xử lý được thực hiện một lần trước khi huấn luyện, sau đó được nạp song song với quá trình training. Cách tổ chức này đánh đổi dung lượng lưu trữ lấy tốc độ: các phép biến đổi tĩnh, không phụ thuộc tham số mô hình, nên không có lý do gì phải tính lại ở mỗi epoch.

2.9. Phân tích và Điểm nổi bật của DTGAN

2.9.1. Ưu điểm so với AnimeGAN và AnimeGANv2

So với các phiên bản tiền nhiệm AnimeGAN [40] và AnimeGANv2 [41], DTGAN mang lại một số cải tiến cơ bản:

Bảng 2.7. So sánh DTGAN với các mô hình chuyển đổi ảnh anime tiền nhiệm

Đặc điểm	AnimeGAN	AnimeGANv2	DTGAN (v3)
Kiến trúc Generator	Single decoder	Single decoder	Double-Tail (2 decoder)
Chuẩn hóa	Instance Norm	Instance Norm	LADE (tự thích ứng)
Cơ chế Attention	Không	Không	External Attention v3
Pseudo GT	Không	Không	NL-Means + L0 Smooth
Discriminator	1	1	2 (grayscale + color)
Color Loss	Lab	Lab	Lab (không gian CIE)
Style Loss	Gram Matrix	Gram Matrix	Decentralized Gram

Có thể nhận thấy các cải tiến của DTGAN đi theo một mạch logic nhất quán: **tách biệt những gì trước đây bị gộp chung**. Một decoder tách thành hai; một discriminator tách thành hai; ma trận Gram tách phần trung bình khỏi phần hiệp phương sai; tham số chuẩn hóa tách khỏi trạng thái cố định để bám theo từng ảnh. Không có cải tiến nào trong số này đòi hỏi tăng đáng kể quy mô mô hình — điều lý giải vì sao DTGAN vẫn giữ được kích thước triển khai ở mức 9,1 MB.

2.9.2. Hạn chế của kiến trúc

Để đánh giá cân bằng, cần chỉ rõ những hạn chế nội tại của DTGAN — đặc biệt là những hạn chế có liên quan trực tiếp tới bài toán ảnh chân dung mà luận văn hướng tới:

1. **Chất lượng bị chặn trên bởi pseudo ground-truth.** Main Tail — nhánh duy nhất được dùng khi suy luận — học chủ yếu từ ảnh đã qua NL-Means + L0 Smoothing với trọng số L_1 rất lớn (50,0). Mọi khiếm khuyết của khâu tiền xử lý này sẽ được mô hình học lại. Với ảnh chân dung, L0 Smoothing có xu hướng làm phẳng các chi tiết mắt và sợi tóc — đúng những đặc điểm quan trọng nhất cho việc nhận dạng con người.
2. **Không có cơ chế ưu tiên theo vùng ngữ nghĩa.** Toàn bộ hàm mất mát đều tính trung bình đồng đều trên mọi pixel. Trong ảnh chân dung mà khuôn mặt chỉ chiếm một tỉ lệ nhỏ diện tích, phần đóng góp của vùng mặt vào gradient tổng là không đáng kể — mô hình sẽ tối ưu chủ yếu cho nền. Đây chính là hạn chế mà module trích xuất khuôn mặt ở Chương 3 được thiết kế để khắc phục.
3. **Bộ trọng số loss gắn với tập dữ liệu cụ thể.** Như đã thảo luận ở Mục 2.6.10, bảy hàm mất mát với trọng số trải trên năm bậc độ lớn cần được hiệu chỉnh lại khi đổi phong cách mục tiêu.
4. **Mỗi phong cách cần một mô hình riêng.** Kiến trúc không có cơ chế điều kiện

hóa theo phong cách, nên hỗ trợ ba phong cách đồng nghĩa với lưu trữ và nạp ba mô hình riêng biệt.

5. **Không có ràng buộc thời gian.** Mô hình xử lý từng ảnh độc lập, không có cơ chế đảm bảo nhất quán giữa các khung hình liên tiếp — nguyên nhân trực tiếp của hiệu ứng nhấp nháy khi xử lý video, được ghi nhận ở Chương 4 và Chương 5.

***Thảo luận:** Nhìn tổng thể, DTGAN là một ví dụ điển hình của hướng thiết kế mà luận văn gọi là “tối ưu bằng cấu trúc thay vì bằng quy mô”. Trong bối cảnh lĩnh vực sinh ảnh đang bị chi phối bởi các mô hình khuếch tán hàng tỉ tham số, DTGAN đạt chất lượng cạnh tranh cho một bài toán hẹp với mô hình chỉ 9,1 MB — không phải nhờ dữ liệu nhiều hơn hay mạng lớn hơn, mà nhờ phân rã bài toán thành các bài toán con có mục tiêu không xung đột, rồi thiết kế cho mỗi bài toán con một tín hiệu học phù hợp. Bài học rút ra có giá trị vượt ra ngoài phạm vi anime hóa: khi các mục tiêu tối ưu mâu thuẫn nhau, việc chia tách không gian tham số thường hiệu quả hơn việc dò tìm bộ trọng số cân bằng. Đồng thời, cần thừa nhận rằng cách tiếp cận này đòi hỏi **hiểu biết sâu về cấu trúc bài toán** — một yêu cầu mà các mô hình nền tảng quy mô lớn không đặt ra. Đây vừa là điểm mạnh (hiệu quả cao trên miền hẹp) vừa là điểm yếu (khó tổng quát hóa sang miền mới) của toàn bộ hướng tiếp cận.*

Tổng kết chương

Chương 2 đã phân tích toàn diện kiến trúc kỹ thuật và nền tảng thiết kế của mô hình AnimeGANv3 (DTGAN), làm rõ những nâng cấp mang tính cốt lõi:

- **Kiến trúc sinh ảnh phân nhánh (Double-Tail Generator):** Một Base Encoder dùng chung kết hợp hai decoder độc lập — Support Tail học đường nét và cấu trúc sáng tối trên miền grayscale, Main Tail tinh chỉnh phân phối màu cel-shading dựa trên pseudo ground-truth. Cách phân rã này giải quyết mâu thuẫn gradient giữa hai mục tiêu vốn không thể dung hòa trong một decoder duy nhất.
- **Cơ chế chuẩn hóa LADE:** Tự sinh tham số tái tỷ lệ hóa từ chính đặc trưng đầu vào qua tích chập 1×1 , kết hợp ưu điểm thích ứng của AdaIN mà không cần ảnh phong cách phụ khi suy luận — điều kiện cần để xuất mô hình sang ONNX với đồ thị tĩnh. Chi phí phụ trội chỉ khoảng 11% so với một lớp tích chập 3×3 cùng số kênh.
- **External Attention v3:** Thay thế tương quan pixel–pixel bằng hai bộ nhớ học được, giảm độ phức tạp từ $\mathcal{O}(N^2C)$ xuống $\mathcal{O}(NkC)$. Cơ chế double normalization biến ma trận chú ý thành tổ hợp lỗi, chặn biên độ đầu ra và ngăn hiện tượng sụp đổ bộ nhớ.

- **Hai Discriminator chuyên biệt theo PatchGAN:** Việc giới hạn đầu vào của Support Discriminator ở miền grayscale là một cách đặc tả hàm mất mát thông qua kiến trúc — công cụ định hướng học tinh tế hơn so với chỉ điều chỉnh trọng số.
- **Hệ thống bảy hàm mất mát:** Mỗi hàm phụ trách một khía cạnh chất lượng riêng (nội dung, phong cách, độ phẳng vùng, màu sắc, độ mịn, tính chân thực, độ bám pseudo ground-truth), kết hợp với quy trình huấn luyện hai giai đoạn để đảm bảo hội tụ ổn định.

Bên cạnh các ưu điểm, chương cũng đã chỉ rõ những hạn chế nội tại của kiến trúc. Hạn chế quan trọng nhất đối với mục tiêu của luận văn là: **mọi hàm mất mát đều tính trung bình đồng đều trên toàn bộ khung ảnh, không có cơ chế ưu tiên theo vùng ngữ nghĩa**. Hệ quả là khi khuôn mặt chỉ chiếm tỉ lệ nhỏ trong ảnh, đóng góp của vùng mặt vào gradient tổng trở nên không đáng kể và mô hình tối ưu chủ yếu cho phần nền — dẫn tới mất chi tiết và suy giảm đặc điểm nhận dạng ở đúng vùng mà người xem chú ý nhất.

Đây chính là bài toán mà Chương 3 giải quyết: thay vì can thiệp vào kiến trúc hay hàm mất mát của DTGAN, luận văn đề xuất một **module trích xuất khuôn mặt** đặt ở tầng pipeline, dựa trên mạng RetinaFace, để đảm bảo mỗi khuôn mặt luôn được đưa vào Generator ở độ phân giải đầy đủ 256×256 .

CHƯƠNG 3

LUỒNG ANIME TÍCH HỢP PHÁT HIỆN KHUÔN MẶT

Chương này trình bày **đóng góp cốt lõi của luận văn**: thiết kế luồng xử lý (pipeline) tích hợp phát hiện khuôn mặt vào quá trình chuyển đổi phong cách anime. Vấn đề cần giải quyết đã được xác định ở cuối Chương 2: mọi hàm mất mát của DTGAN đều tính trung bình đồng đều trên toàn khung ảnh, không có cơ chế ưu tiên theo vùng ngữ nghĩa, nên khi khuôn mặt chiếm tỉ lệ nhỏ so với khung hình, mô hình sẽ tối ưu chủ yếu cho phần nền và khuôn mặt bị mất chi tiết cùng đặc điểm nhận dạng.

Giải pháp đề xuất là tự động phát hiện, trích xuất và mở rộng vùng khuôn mặt trước khi đưa vào Generator, giúp tập trung toàn bộ năng lực tính toán vào khu vực trọng tâm [56, 42]. Chương được tổ chức theo trình tự của chính luồng dữ liệu: phát hiện khuôn mặt (Mục 3.2) → mở rộng vùng cắt (Mục 3.3) → chuyển đổi phong cách (Mục 3.5) → ghép ngược vào nền (Mục 3.6), kèm theo các phân tích thiết kế và đánh giá phản biện tại mỗi bước.

3.1. Tổng quan Pipeline Tích hợp

3.1.1. Nguyên tắc thiết kế: can thiệp ở tầng pipeline

Trước khi mô tả giải pháp, cần làm rõ lựa chọn thiết kế nền tảng của luận văn. Về nguyên tắc, vấn đề suy giảm chất lượng khuôn mặt có ít nhất ba hướng khắc phục:

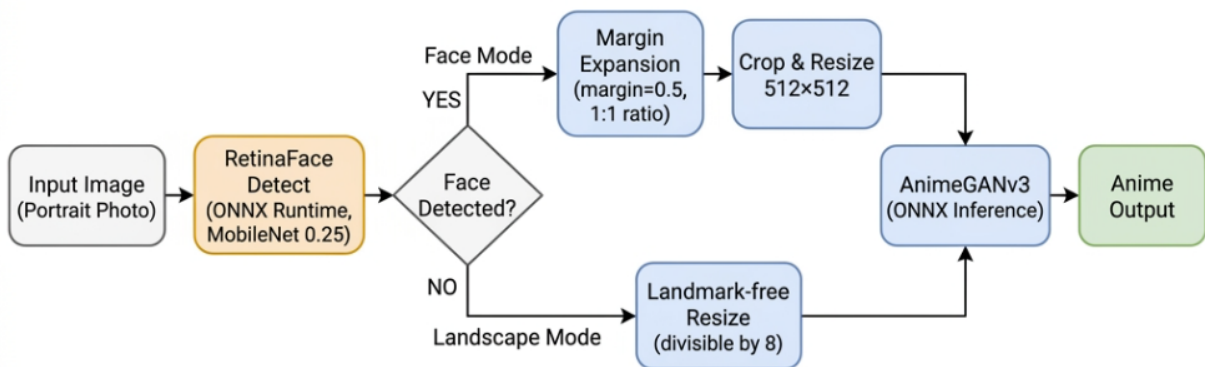
1. **Sửa hàm mất mát**: bổ sung một số hạng có trọng số cao cho vùng khuôn mặt (ví dụ nhân mặt nạ khuôn mặt vào Content Loss). Hướng này đòi hỏi huấn luyện lại toàn bộ mô hình và cần mặt nạ khuôn mặt cho mọi ảnh trong tập huấn luyện.
2. **Sửa kiến trúc**: thêm nhánh chuyên biệt cho khuôn mặt vào Generator. Hướng này làm tăng kích thước mô hình — đi ngược lại mục tiêu triển khai nhẹ — và cũng đòi hỏi huấn luyện lại.
3. **Can thiệp ở tầng pipeline**: giữ nguyên mô hình đã huấn luyện, thay đổi *cái gì được đưa vào mô hình* thay vì thay đổi *mô hình xử lý ra sao*.

Luận văn chọn hướng thứ ba. Cơ sở của lựa chọn này gồm ba điểm: (i) mô hình DTGAN được huấn luyện ở độ phân giải cố định 256×256 — đưa khuôn mặt vào đúng độ phân giải đó tức là đưa dữ liệu về đúng phân phối huấn luyện; (ii) không phát sinh chi phí huấn luyện lại, cho phép tái sử dụng mọi mô hình phong cách hiện có; (iii) module phát hiện khuôn mặt là thành phần độc lập, có thể nâng cấp riêng mà không ảnh hưởng tới Generator.

3.1.2. Ba chế độ vận hành

Để hệ thống trích xuất và chuyển đổi hoạt động liền mạch với hiệu năng thời gian thực, luận văn đã thiết kế một luồng xử lý dữ liệu nội bộ hỗ trợ **ba chế độ vận hành**, chia thành hai giai đoạn:

1. **Định vị và quy hoạch vùng trích xuất:** Ứng dụng kiến trúc phát hiện khuôn mặt RetinaFace [42] với backbone thu gọn MobileNet-0.25 [24], được tối ưu hóa thông qua ONNX Runtime [37] để lấy tọa độ khuôn mặt tức thì. Thuật toán Margin Expansion mở rộng bounding box, đảm bảo bao phủ không chỉ ngũ quan mà còn toàn bộ chi tiết tóc, cổ và bối cảnh lân cận.
2. **Áp dụng phong cách hóa (Stylization):** Tùy theo chế độ được chọn:
 - **Face Mode:** Chỉ vùng khuôn mặt sau mở rộng được resize về 256×256 và đưa qua Generator.
 - **Landscape Mode:** Toàn bộ ảnh được resize giữ tỉ lệ (chia hết cho 8) rồi đưa qua Generator.
 - **Hybrid Mode:** Kết hợp cả hai — nền được xử lý Landscape, từng khuôn mặt được xử lý Face Mode riêng rồi ghép ngược lại bằng kỹ thuật feathered blending.



Hình 3.1. Pipeline tích hợp Face Extraction vào AnimeGANv3

Chú thích: Face Mode: ảnh qua RetinaFace → Margin Expansion → Crop 256×256 → ONNX Inference. Landscape Mode: ảnh resize giữ tỉ lệ (chia hết 8) rồi trực tiếp qua ONNX Inference. Hybrid Mode: nền qua Landscape, mỗi khuôn mặt qua Face Mode rồi ghép feathered blend [56].

Kiến trúc này cho phép hệ thống xử lý linh hoạt ba loại kịch bản: ảnh chân dung cận mặt (Face Mode), ảnh phong cảnh (Landscape Mode), và ảnh nhóm người cần giữ cả nền

lần chất lượng mặt cao (Hybrid Mode). Khi Face Mode hoặc Hybrid Mode được kích hoạt nhưng không phát hiện khuôn mặt nào, hệ thống tự động chuyển sang Landscape Mode — cơ chế **hạ cấp mềm** (graceful degradation) đảm bảo hệ thống không bao giờ trả về lỗi cho người dùng.

***Phân tích thiết kế:** Việc bổ sung một module tiền xử lý thay vì sửa mô hình sinh ảnh phản ánh một nguyên tắc kỹ thuật mà luận văn cho là đáng lưu ý: **khi mô hình có một chế độ làm việc tối ưu đã biết, cách rẻ nhất để cải thiện kết quả là đưa dữ liệu về đúng chế độ đó**. DTGAN được huấn luyện trên các mẫu ảnh 256×256 ; mọi đầu vào lệch khỏi phân phối này — ảnh quá lớn, khuôn mặt quá nhỏ, tỉ lệ khung hình bất thường — đều khiến mô hình làm việc ngoài vùng thoải mái của nó. Module Face Extraction thực chất là một bộ **quy chuẩn hóa đầu vào** (input canonicalizer). Đổi lại sự đơn giản đó, hệ thống phải chấp nhận một hệ quả kiến trúc: chất lượng đầu ra giờ đây phụ thuộc vào **hai** mô hình mắc nối tiếp, và sai sót của bộ phát hiện khuôn mặt sẽ truyền thẳng xuống khâu sinh ảnh mà không có cơ chế nào sửa chữa. Đây là đánh đổi trung tâm của toàn chương và sẽ được đánh giá lại ở Mục 3.7.3.*

3.2. Phát hiện Khuôn mặt với RetinaFace

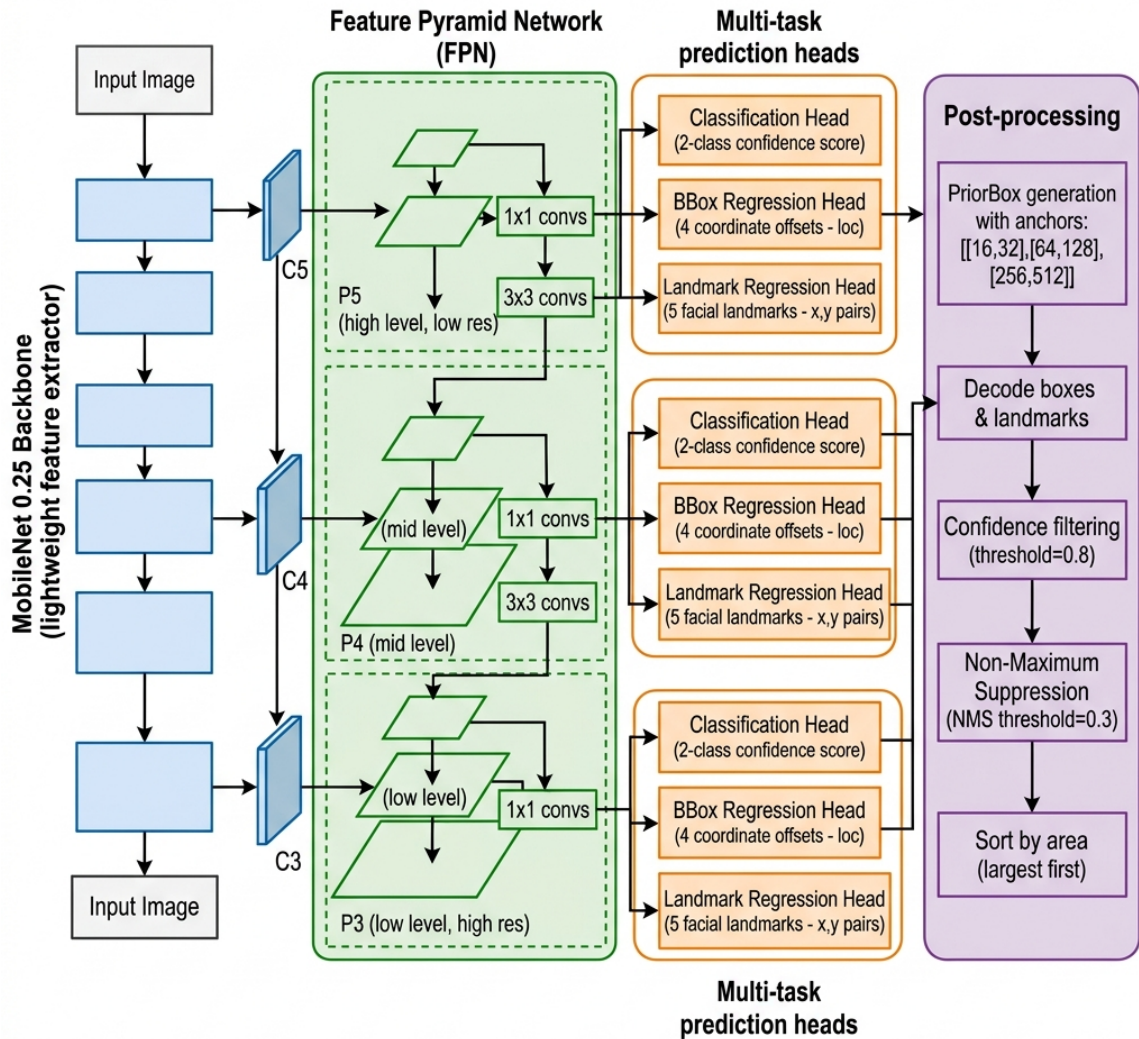
3.2.1. Tổng quan RetinaFace

RetinaFace [42] là phương pháp phát hiện khuôn mặt đơn luồng (single-shot) đa tỉ lệ, được công bố tại CVPR 2020. Điểm nổi bật của RetinaFace là thực hiện **học đa nhiệm** (multi-task learning) đồng thời ba bài toán con trong một lần suy luận: phân loại mặt/không mặt, hồi quy bounding box, và hồi quy 5 điểm đặc trưng khuôn mặt (facial landmarks).

Lợi ích của việc gộp ba nhiệm vụ không chỉ nằm ở tốc độ. Nhiệm vụ hồi quy landmark đóng vai trò **nhiệm vụ phụ trợ** (auxiliary task): để dự đoán đúng vị trí hai mắt và mũi, mạng buộc phải học biểu diễn nắm bắt được cấu trúc hình học nội tại của khuôn mặt. Biểu diễn đó đồng thời làm cho nhiệm vụ chính — phân loại và định vị — trở nên chính xác hơn, đặc biệt với các khuôn mặt bị che khuất một phần hoặc ở góc nghiêng.

Trong hệ thống của luận văn, phiên bản sử dụng backbone **MobileNet 0.25** [24] — phiên bản siêu nhẹ của MobileNet chỉ với 25% số kênh so với MobileNet đầy đủ — được triển khai thông qua ONNX Runtime [37] để đảm bảo tốc độ suy luận cao trên cả CPU lẫn GPU tiêu dùng.

3.2.2. Kiến trúc Mạng Phát hiện



Hình 3.2. Kiến trúc RetinaFace với backbone MobileNet 0.25 và Feature Pyramid Network (FPN)

Chú thích: Ba đầu ra đa nhiệm tại mỗi cấp FPN: confidence score, bounding box offset (loc), và 5 facial landmarks. Hậu xử lý bao gồm lọc ngưỡng tin cậy (0,8), NMS (0,3), và sắp xếp theo diện tích [42]

Kiến trúc RetinaFace bao gồm ba thành phần chính:

3.2.2.1. Backbone MobileNet 0.25

MobileNet [24] được thiết kế đặc biệt cho tính toán trên thiết bị nhúng và ứng dụng thời gian thực, sử dụng *depthwise separable convolution* để giảm chi phí tính toán. Ý tưởng là tách một phép tích chập chuẩn thành hai bước: lọc không gian theo từng kênh độc lập (depthwise) rồi trộn kênh bằng tích chập 1×1 (pointwise). Tỷ lệ chi phí giữa hai cách làm:

$$\frac{\text{Chi phí depthwise separable}}{\text{Chi phí tích chập chuẩn}} = \frac{D_K^2 \cdot C_{in} \cdot HW + C_{in} \cdot C_{out} \cdot HW}{D_K^2 \cdot C_{in} \cdot C_{out} \cdot HW} = \frac{1}{C_{out}} + \frac{1}{D_K^2} \quad (3.1)$$

trong đó D_K là kích thước kernel, C_{in}, C_{out} là số kênh vào/ra, H, W là kích thước không gian. Với $D_K = 3$ và C_{out} lớn, tỉ lệ này xấp xỉ $1/9$ — tức tiết kiệm khoảng **9 lần** chi phí tính toán.

Hệ số nhân chiều rộng (width multiplier) $\alpha = 0,25$ thu hẹp thêm số lượng kênh trong mỗi lớp xuống còn 25%. Vì chi phí tính toán tỉ lệ xấp xỉ bậc hai với số kênh, việc này giảm thêm khoảng $\alpha^2 = 1/16$ chi phí. Kết hợp hai kỹ thuật, backbone đạt kích thước chỉ khoảng 500 KB — điều kiện tiên quyết để tích hợp vào một ứng dụng web nhẹ.

3.2.2.2. Feature Pyramid Network (FPN)

Feature Pyramid Network [26] được xây dựng trên các đặc trưng trung gian của backbone để tạo ra biểu diễn đa tỉ lệ $\{P_3, P_4, P_5\}$, cho phép phát hiện khuôn mặt ở nhiều kích thước khác nhau trong cùng một ảnh.

Vấn đề mà FPN giải quyết là một mâu thuẫn cố hữu trong CNN: các tầng nông có độ phân giải cao (tốt cho định vị vật thể nhỏ) nhưng ngữ nghĩa yếu; các tầng sâu có ngữ nghĩa mạnh nhưng độ phân giải thấp. Kiến trúc top-down với các kết nối ngang (lateral connections) truyền ngữ nghĩa từ tầng sâu ngược lên tầng nông:

$$P_i = \text{Conv}_{3 \times 3} \left(\underbrace{\text{Conv}_{1 \times 1}(C_i)}_{\text{kết nối ngang}} + \underbrace{\text{Upsample}_{2 \times}(P_{i+1})}_{\text{luồng top-down}} \right) \quad (3.2)$$

trong đó C_i là đặc trưng tầng thứ i của backbone. Kết quả là mỗi tầng P_i đều mang đồng thời ngữ nghĩa mức cao và độ phân giải không gian phù hợp với dải kích thước mà nó phụ trách.

3.2.2.3. Đầu ra Đa nhiệm

Tại mỗi tầng FPN, ba đầu dự đoán song song được kích hoạt:

- **Classification head:** Xác suất có/không có khuôn mặt tại mỗi anchor.
- **BBox Regression head (loc):** 4 giá trị offset (dx, dy, dw, dh) so với prior box.
- **Landmark Regression head (landms):** 10 giá trị (x_i, y_i) cho 5 điểm đặc trưng: mắt trái, mắt phải, mũi, khóe miệng trái, khóe miệng phải.

3.2.3. Tiền xử lý Ảnh Đầu vào

Trước khi đưa vào mạng RetinaFace, ảnh được tiền xử lý theo chuẩn ImageNet:

1. Chuyển kiểu dữ liệu sang `float32`

2. Trừ giá trị mean ImageNet theo từng kênh:

$$\text{img} \leftarrow \text{img} - (123,0; 117,0; 104,0) \quad (3.3)$$

3. Chuyển thứ tự trục từ $H \times W \times C$ (HWC) sang $C \times H \times W$ (CHW):

$$\text{img} = \text{img}_{(2,0,1)}^T \quad (3.4)$$

4. Mở rộng chiều batch: $\text{img} \in \mathbb{R}^{1 \times C \times H \times W}$

Ba điểm cần lưu ý về bước tiền xử lý này:

- **Chỉ trừ trung bình, không chia độ lệch chuẩn.** Đây là quy ước tiền xử lý của họ mô hình Caffe mà RetinaFace kế thừa, khác với quy ước của PyTorch (chia thêm cho `std`). Áp dụng sai quy ước sẽ khiến đầu vào lệch thang đo và độ chính xác phát hiện sụt giảm nghiêm trọng — một lỗi tích hợp phổ biến và khó phát hiện vì mô hình vẫn chạy, chỉ là kết quả kém.
- **Thứ tự kênh BGR.** Bộ ba (123,0; 117,0; 104,0) tương ứng với thứ tự BGR của OpenCV, không phải RGB. Trong pipeline của luận văn, ảnh được đọc bằng Pillow ở định dạng RGB, nên cần chuyển đổi thứ tự kênh trước khi trừ trung bình.
- **Không resize về kích thước cố định.** Nhờ kiến trúc hoàn toàn tích chập (fully convolutional), RetinaFace chấp nhận ảnh ở kích thước bất kỳ; số lượng prior box được sinh động theo kích thước đầu vào. Đây là lý do bước giới hạn cạnh dài 1280 pixel ở Mục 3.5.1 lại quan trọng: nó vừa kiểm soát bộ nhớ, vừa kiểm soát số lượng anchor cần hậu xử lý.

3.2.4. Hậu xử lý và Prior Box

3.2.4.1. Prior Box (Anchor)

Mô hình sử dụng hệ thống Prior Box đa tỉ lệ với cấu hình `cfg_mnet`:

Bảng 3.1. Cấu hình Prior Box của RetinaFace với backbone MobileNet 0.25

Tầng FPN	Stride	Anchor sizes	Đối tượng phát hiện
P_3 (high res)	8	[16, 32] px	Khuôn mặt rất nhỏ
P_4	16	[64, 128] px	Khuôn mặt nhỏ – trung bình
P_5 (low res)	32	[256, 512] px	Khuôn mặt lớn, cận cảnh

Prior box (hay anchor) là các khung tham chiếu có kích thước định trước, được rải đều trên lưới không gian của từng tầng FPN. Mạng không dự đoán tọa độ tuyệt đối của

khuôn mặt — một bài toán hồi quy khó với miền giá trị rộng — mà chỉ dự đoán **độ lệch tương đối** so với prior box gần nhất. Cách này thu hẹp đáng kể miền đầu ra cần học và giúp mạng hội tụ nhanh hơn.

Tổng số prior box sinh ra cho một ảnh kích thước $H \times W$:

$$N_{\text{prior}} = \sum_{i \in \{3,4,5\}} 2 \cdot \left\lceil \frac{H}{s_i} \right\rceil \cdot \left\lceil \frac{W}{s_i} \right\rceil, \quad s_i \in \{8, 16, 32\} \quad (3.5)$$

với hệ số 2 tương ứng hai kích thước anchor tại mỗi tầng. Với ảnh 1280×960 , con số này xấp xỉ 50.000 prior box — lý giải vì sao các bước lọc ở phần sau lại cần thiết đến vậy.

Nhận xét về dải kích thước. Bảng 3.1 cho thấy các anchor phủ dải từ 16 đến 512 pixel. Tầng P_3 với anchor 16 px về lý thuyết cho phép phát hiện khuôn mặt rất nhỏ; tuy nhiên, như kết quả thực nghiệm ở Chương 4 chỉ ra, độ chính xác thực tế với khuôn mặt dưới 32×32 px giảm còn khoảng 71% precision. Nguyên nhân không nằm ở thiết kế anchor mà ở dung lượng biểu diễn của backbone MobileNet 0.25: với chỉ 25% số kênh, các tầng nông đơn giản không đủ khả năng phân biệt một khuôn mặt 16 pixel với các mẫu nhiễu có cấu trúc tương tự.

3.2.4.2. Giải mã Kết quả

Từ output thô của mạng, bounding boxes và landmarks được giải mã từ offset tương đối sang tọa độ tuyệt đối:

$$\hat{b}_x = p_x + \text{loc}_x \cdot \text{var}_0 \cdot p_w, \quad \hat{b}_y = p_y + \text{loc}_y \cdot \text{var}_0 \cdot p_h \quad (3.6)$$

$$\hat{b}_w = p_w \cdot e^{\text{loc}_w \cdot \text{var}_1}, \quad \hat{b}_h = p_h \cdot e^{\text{loc}_h \cdot \text{var}_1} \quad (3.7)$$

trong đó:

- (p_x, p_y, p_w, p_h) — tọa độ tâm và kích thước của prior box, đã chuẩn hóa về $[0, 1]$ theo kích thước ảnh.
- $(\text{loc}_x, \text{loc}_y, \text{loc}_w, \text{loc}_h)$ — bốn giá trị thô do mạng dự đoán.
- $(\text{var}_0, \text{var}_1) = (0,1; 0,2)$ — các tham số phương sai, đóng vai trò hệ số tỉ lệ cố định.

Ý nghĩa của hai dạng công thức khác nhau:

- **Tọa độ tâm (Phương trình 3.6) dùng dịch chuyển tuyến tính**, và độ dịch được nhân với p_w (hoặc p_h). Điều này khiến sai số dự đoán được đo *theo tỉ lệ kích thước khuôn mặt* chứ không theo pixel tuyệt đối: lệch 5 pixel trên một khuôn mặt 500 px là không đáng kể, nhưng trên khuôn mặt 20 px lại là sai lầm lớn. Cách tham số hóa này đảm bảo mạng đối xử công bằng với mọi kích thước khuôn mặt.

- **Kích thước (Phương trình 3.7) dùng hàm mũ.** Có hai lý do. Thứ nhất, hàm mũ luôn cho kết quả dương, nên chiều rộng và chiều cao dự đoán không bao giờ âm — mạng không cần học ràng buộc này. Thứ hai, hàm mũ biến phép *nhân* tỉ lệ thành phép *cộng* trong không gian dự đoán: $\text{loc}_w = 0$ nghĩa là giữ nguyên kích thước anchor, $\text{loc}_w > 0$ là phóng to, $\text{loc}_w < 0$ là thu nhỏ, và mức độ thay đổi đối xứng theo tỉ lệ.
- **Vai trò của $\text{var}_0, \text{var}_1$.** Hai hằng số này chuẩn hóa thang đo của đầu ra hồi quy về vùng lân cận 0 với độ lớn tương đương nhau, giúp bốn thành phần của loc đóng góp cân bằng vào hàm mất mát khi huấn luyện. Giá trị $\text{var}_1 = 0,2 > \text{var}_0 = 0,1$ phản ánh thực tế rằng sai số về kích thước thường lớn hơn sai số về vị trí tâm.

Năm điểm landmark được giải mã theo cùng nguyên tắc như tọa độ tâm:

$$\hat{\ell}_x^{(k)} = p_x + \text{landms}_x^{(k)} \cdot \text{var}_0 \cdot p_w, \quad k = 1, \dots, 5 \quad (3.8)$$

3.2.4.3. Lọc và Non-Maximum Suppression

Quá trình hậu xử lý gồm 5 bước tuần tự, thu gọn từ hàng chục nghìn ứng viên xuống tối đa 5 khuôn mặt:

1. **Lọc ngưỡng tin cậy:** Chỉ giữ các detection có confidence $> 0,8$ (tham số `confidence_threshold`).
2. **Giữ Top-K trước NMS:** Sắp xếp theo confidence giảm dần, chỉ giữ tối đa $K_1 = 10$ detection có điểm cao nhất (tham số `top_k`) nhằm giảm chi phí tính toán NMS.
3. **Non-Maximum Suppression (NMS):** Loại bỏ các bounding box trùng lặp với $\text{IoU} > 0,3$ (tham số `nms_threshold`).
4. **Giữ Top-K sau NMS:** Chỉ giữ tối đa $K_2 = 5$ khuôn mặt (tham số `keep_top_k`).
5. **Sắp xếp theo diện tích:** Khuôn mặt lớn nhất (chủ thể chính) được đặt lên đầu danh sách:

$$\text{order} = \text{argsort}[(x_2 - x_1)(y_2 - y_1)]_{\text{giảm dần}} \quad (3.9)$$

Vì sao cần NMS. Do prior box được rải dày đặc, một khuôn mặt duy nhất thường kích hoạt hàng chục anchor lân cận, tạo ra nhiều bounding box gần trùng nhau. NMS giải quyết bằng thuật toán tham lam: chọn box có điểm cao nhất, loại bỏ mọi box chồng lấp quá ngưỡng với nó, rồi lặp lại trên phần còn lại. Độ chồng lấp được đo bằng hệ số IoU (Intersection over Union):

$$\text{IoU}(B_1, B_2) = \frac{|B_1 \cap B_2|}{|B_1 \cup B_2|} = \frac{\text{diện tích phần giao}}{\text{diện tích phần hợp}} \in [0, 1] \quad (3.10)$$

Ý nghĩa của các ngưỡng đã chọn. Bộ tham số (0,8; 0,3; 10; 5) phản ánh rõ một triết lý: **ưu tiên độ chính xác hơn độ bao phủ**.

- **Ngưỡng tin cậy 0,8** là mức khá cao (nhiều hệ thống dùng 0,5). Lựa chọn này chấp nhận bỏ sót một số khuôn mặt để gần như loại trừ khả năng phát hiện nhầm. Lý do nằm ở hậu quả bất đối xứng của hai loại lỗi: bỏ sót khuôn mặt chỉ khiến hệ thống hạ cấp về Landscape Mode — kết quả vẫn chấp nhận được; ngược lại, phát hiện nhầm sẽ khiến một vùng nền ngẫu nhiên bị anime hóa như khuôn mặt rồi ghép đè lên ảnh, tạo ra lỗi thị giác rõ rệt.
- **Ngưỡng NMS 0,3** nghiêm ngặt hơn mức thông thường (0,5), nghĩa là hai box chỉ cần chồng lấp 30% đã bị coi là trùng. Đây là lựa chọn phù hợp với bài toán ảnh chân dung, nơi các khuôn mặt hiếm khi che khuất nhau nhiều. Tuy nhiên, cần ghi nhận rằng ngưỡng này sẽ gây bỏ sót trong ảnh nhóm đông người đứng sát nhau — một hạn chế đã biết của thiết kế.
- **Giới hạn $K_2 = 5$ khuôn mặt** được đặt ra chủ yếu vì lý do hiệu năng của Hybrid Mode: mỗi khuôn mặt phát sinh thêm một lần suy luận ONNX (xem Mục 3.6.4).

Trong chế độ Face Mode, phần tử đầu tiên (`bboxes[0]`) — khuôn mặt lớn nhất — được chọn làm đối tượng chuyển đổi. Trong chế độ Hybrid Mode, toàn bộ danh sách khuôn mặt sau lọc đều được xử lý. Nếu kết quả rỗng, hàm trả về `None` và pipeline chuyển sang Landscape Mode.

***Nhận xét:** Việc sắp xếp theo diện tích thay vì theo điểm tin cậy ở bước 5 là một chi tiết nhỏ nhưng phản ánh đúng ngữ nghĩa bài toán. Điểm tin cậy trả lời câu hỏi “đây có phải khuôn mặt không?”, trong khi thứ hệ thống cần biết là “đâu là khuôn mặt mà người dùng quan tâm?”. Trong nhiếp ảnh chân dung, chủ thể chính hầu như luôn là khuôn mặt lớn nhất khung hình — một quy ước bố cục ổn định đến mức có thể dùng làm heuristic đáng tin cậy. Dù vậy, cần thừa nhận rằng heuristic này thất bại trong một số tình huống thực tế: ảnh có người đứng gần camera nhưng không phải chủ thể (người qua đường lọt khung), hoặc ảnh nhóm mà chủ thể đứng ở hàng sau. Với những trường hợp đó, Hybrid Mode — xử lý toàn bộ khuôn mặt — là lời giải đúng đắn hơn, và đây chính là một trong những động lực dẫn tới việc luận văn bổ sung chế độ này.*

3.3. Kỹ thuật Mở rộng Vùng Khuôn mặt (Margin Expansion)

3.3.1. Động lực và Mục tiêu

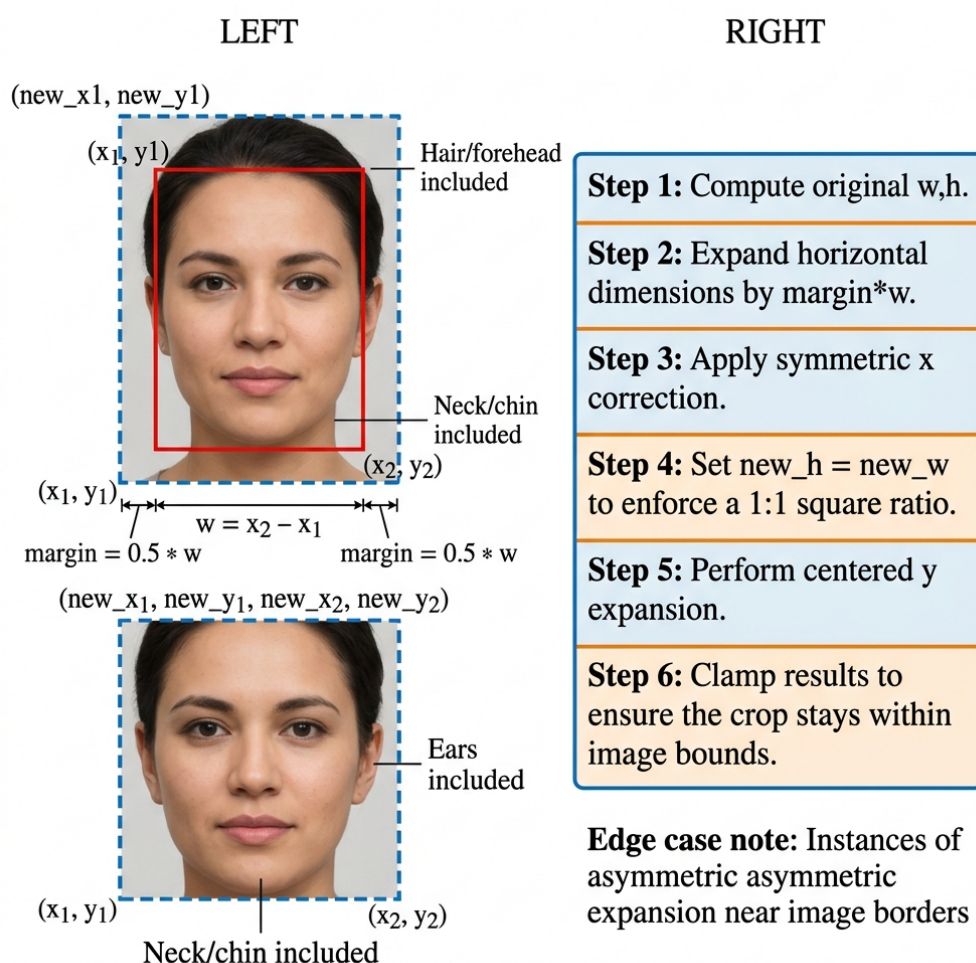
Bounding box khuôn mặt trả về bởi RetinaFace thường chỉ bao quanh phần khuôn mặt thuần túy (từ trán đến cằm, từ tai này đến tai kia). Nếu cắt đúng theo bounding box này để đưa vào mô hình AnimeGANv3, ảnh đầu ra sẽ thiếu ngữ cảnh quan trọng: tóc, cổ,

vai — những thành phần không thể thiếu để tạo ra ảnh anime chân dung trông tự nhiên và hoàn chỉnh.

Vấn đề còn nghiêm trọng hơn ở khía cạnh phân phối dữ liệu. Tập ảnh huấn luyện của DTGAN gồm các ảnh chân dung thông thường, trong đó khuôn mặt luôn xuất hiện cùng tóc và nền. Một mẫu ảnh chỉ chứa da mặt là đầu vào **ngoài phân phối huấn luyện** (out-of-distribution) — mô hình chưa từng gặp dạng dữ liệu này và không có cơ sở để xử lý đúng.

Kỹ thuật **Margin Expansion** (mở rộng lề) được phát triển trong luận văn nhằm hai mục tiêu đồng thời:

1. Mở rộng bounding box theo tỉ lệ phần trăm kích thước mặt, đảm bảo bao phủ đủ ngữ cảnh.
2. Duy trì tỉ lệ khung hình 1:1 (vuông), phù hợp với yêu cầu đầu vào 256×256 của mô hình — tránh biến dạng hình học do resize không đồng đều hai chiều.



Hình 3.3. Minh họa kỹ thuật Margin Expansion

Chú thích: Bounding box gốc (nét đỏ) được mở rộng 50% mỗi bên theo chiều ngang, sau đó điều chỉnh chiều dọc để tạo vùng vuông (nét xanh). Vùng mở rộng bao gồm tóc, tai, cổ — cần thiết cho chuyển đổi anime tự nhiên.

3.3.2. Thuật toán *Margin Expansion*

Cho bounding box khuôn mặt (x_1, y_1, x_2, y_2) , kích thước ảnh (H, W) và hệ số mở rộng $m = 0,5$, thuật toán thực hiện theo 6 bước:

Bước 1 — Tính kích thước hộp gốc:

$$w = x_2 - x_1, \quad h = y_2 - y_1 \quad (3.11)$$

Bước 2 — Mở rộng theo chiều ngang:

$$x'_1 = \max(0, x_1 - m \cdot w), \quad x'_2 = \min(W, x_2 + m \cdot w) \quad (3.12)$$

Bước 3 — Đảm bảo mở rộng đối xứng:

$$x_d = \min(x_1 - x'_1, x'_2 - x_2), \quad w' = w + 2x_d \quad (3.13)$$

Bước 4 — Thiết lập chiều cao mới (tỉ lệ 1:1):

$$h' = w' \quad (3.14)$$

Bước 5 — Điều chỉnh chiều dọc (phân nhánh):

Trường hợp $h' \geq h$ (phổ biến — cần mở rộng theo chiều dọc):

$$y_d = h' - h \quad (3.15)$$

$$y'_1 = \max(0, y_1 - \lfloor y_d/2 \rfloor) \quad (3.16)$$

$$y'_2 = \min(H, y_2 + \lfloor y_d/2 \rfloor) \quad (3.17)$$

Trường hợp $h' < h$ (khuôn mặt gần biên ngang — chiều rộng mở rộng bị hạn chế, cần thu hẹp chiều dọc):

$$y_d = |h' - h| \quad (3.18)$$

$$y'_1 = \max(0, y_1 + \lfloor y_d/2 \rfloor) \quad (3.19)$$

$$y'_2 = \min(H, y_2 - \lfloor y_d/2 \rfloor) \quad (3.20)$$

Bước 6 — Làm tròn về chỉ số nguyên: Các giá trị x'_1, y'_1, x'_2, y'_2 được chuyển sang

kiểu nguyên (int) và dùng làm chỉ số cắt mảng.

Thuật toán 3.1 Margin Expansion — mở rộng vùng khuôn mặt đối xứng

Đầu vào: Bounding box (x_1, y_1, x_2, y_2) ; kích thước ảnh (H, W) ; hệ số $m = 0,5$

Đầu ra: Vùng cắt mở rộng (x'_1, y'_1, x'_2, y'_2) nằm trọn trong ảnh

- 1: $w \leftarrow x_2 - x_1; \quad h \leftarrow y_2 - y_1$ ▷ Bước 1
 - 2: $x'_1 \leftarrow \max(0, x_1 - m w); \quad x'_2 \leftarrow \min(W, x_2 + m w)$ ▷ Bước 2
 - 3: $x_d \leftarrow \min(x_1 - x'_1, x'_2 - x_2)$ ▷ Bước 3: lấy lề nhỏ hơn
 - 4: $x'_1 \leftarrow x_1 - x_d; \quad x'_2 \leftarrow x_2 + x_d; \quad w' \leftarrow w + 2x_d$
 - 5: $h' \leftarrow w'$ ▷ Bước 4: ép tỉ lệ 1:1
 - 6: **nếu** $h' \geq h$ **thì** ▷ Bước 5a: giãn theo chiều dọc
 - 7: $y_d \leftarrow h' - h$
 - 8: $y'_1 \leftarrow \max(0, y_1 - \lfloor y_d/2 \rfloor); \quad y'_2 \leftarrow \min(H, y_2 + \lfloor y_d/2 \rfloor)$
 - 9: **ngược lại** ▷ Bước 5b: co theo chiều dọc
 - 10: $y_d \leftarrow h - h'$
 - 11: $y'_1 \leftarrow y_1 + \lfloor y_d/2 \rfloor; \quad y'_2 \leftarrow y_2 - \lfloor y_d/2 \rfloor$
 - 12: **hết nếu**
 - 13: **trả về** $(\lfloor x'_1 \rfloor, \lfloor y'_1 \rfloor, \lfloor x'_2 \rfloor, \lfloor y'_2 \rfloor)$ ▷ Bước 6
-

Giải thích logic của Bước 3 — điểm mấu chốt của thuật toán. Sau Bước 2, hai biên trái và phải đã được mở rộng nhưng *không nhất thiết bằng nhau*: nếu khuôn mặt nằm gần mép trái ảnh, toán tử $\max(0, \cdot)$ sẽ chặn lề trái lại trong khi lề phải vẫn giãn đủ $m \cdot w$. Cắt theo hộp bất đối xứng như vậy sẽ đặt khuôn mặt lệch tâm trong ảnh 256×256 đưa vào Generator — lệch khỏi bố cục chuẩn của dữ liệu huấn luyện.

Bước 3 khắc phục bằng cách lấy x_d là **lề nhỏ hơn trong hai lề**, rồi áp dụng chính lề đó cho cả hai phía. Kết quả:

$$x_d = \min(m \cdot w, x_1, W - x_2) \quad (3.21)$$

tức là lề mở rộng bằng $m \cdot w$ khi có đủ không gian, và bằng khoảng cách tới biên gần nhất khi không đủ. Cách làm này hy sinh một phần ngữ cảnh để đổi lấy **tính đối xứng của bố cục** — và đây là đánh đổi đúng, vì mô hình nhạy cảm với vị trí khuôn mặt trong khung hình hơn là với lượng nền bao quanh.

3.3.3. Tính chất của thuật toán

Từ các công thức trên có thể rút ra ba tính chất định lượng hữu ích cho việc phân tích ở các mục sau:

Tính chất 1 — Bề rộng vùng cắt. Trong trường hợp không bị chặn biên ($x_d = m \cdot w$), bề rộng vùng cắt là:

$$w' = w + 2mw = (1 + 2m)w = 2w \quad \text{khi } m = 0,5 \quad (3.22)$$

tức vùng cắt rộng gấp **đôi** bounding box gốc.

Tính chất 2 — Vùng cắt vuông không được bảo đảm tuyệt đối. Bước 4 đặt $h' = w'$, nhưng Bước 5 lại áp thêm các toán tử $\max(0, \cdot)$ và $\min(H, \cdot)$. Nếu khuôn mặt nằm gần biên trên hoặc biên dưới, chiều cao thực tế sẽ nhỏ hơn h' và vùng cắt trở thành hình chữ nhật. Khi đó, phép `resize` về 256×256 sẽ gây biến dạng tỉ lệ khung hình. Đây là một hạn chế đã biết của cài đặt hiện tại; hướng khắc phục là đệm thêm viền (padding) thay vì cắt cụt, sẽ được bàn ở Mục 3.7.3.

Tính chất 3 — Bề rộng khuôn mặt sau chuẩn hóa là hằng số. Kết hợp Tính chất 1 với bước `resize` về 256×256 , khuôn mặt trong ảnh đưa vào Generator luôn có bề rộng:

$$w_{\text{model}} = 256 \cdot \frac{w}{w'} = \frac{256}{1 + 2m} = 128 \text{ pixel} \quad \text{khi } m = 0,5 \quad (3.23)$$

không phụ thuộc vào kích thước khuôn mặt trong ảnh gốc. Đây chính là tính chất quan trọng nhất của toàn bộ module — nó biến một đầu vào có kích thước tùy ý thành một đầu vào đã được chuẩn hóa hoàn toàn, và sẽ là cơ sở cho phân tích ở Mục 3.4.

3.3.4. Xử lý Trường hợp Đặc biệt (Edge Cases)

Bước 3 và bước 5 là cơ chế xử lý edge case quan trọng:

- **Khuôn mặt gần biên ảnh:** Thay vì tràn ra ngoài ảnh, lề mở rộng bị thu về mức nhỏ nhất trong hai phía (Phương trình 3.21), giữ được tính đối xứng của bố cục.
- **Khuôn mặt ở góc ảnh:** Cả ràng buộc theo chiều ngang lẫn chiều dọc cùng có hiệu lực; vùng cắt thu nhỏ về phần hợp lệ lớn nhất có thể.
- **Khuôn mặt rất lớn (gần bằng ảnh):** x_d bị giới hạn bởi khoảng cách tới biên, vùng cắt gần như trùng với bounding box gốc — mở rộng gần bằng 0. Trường hợp này không gây lỗi nhưng cũng không mang lại lợi ích của Margin Expansion.
- **Trường hợp $h' < h$ (Bước 5b):** Xảy ra khi lề ngang bị chặn mạnh khiến w' (và do đó $h' = w'$) nhỏ hơn chiều cao khuôn mặt. Thuật toán *cắt bớt* theo chiều dọc, đối xứng ở trên và dưới. Cần lưu ý rằng trong nhánh này, hai toán tử $\max(0, \cdot)$ và $\min(H, \cdot)$ ở Phương trình (3.19) là dư thừa về mặt logic — do $y'_1 > y_1 \geq 0$ và $y'_2 < y_2 \leq H$ — nhưng được giữ lại như một lớp bảo vệ phòng thủ chống lại bounding box không hợp lệ do mạng phát hiện trả về.

3.3.5. Ý nghĩa của Tham số $\text{margin} = 0.5$

Giá trị $m = 0,5$ được lựa chọn qua thực nghiệm (kết quả định lượng trình bày tại Mục 4.6.2, Chương 4), có nghĩa là mở rộng **50% chiều rộng khuôn mặt về mỗi phía**, tạo ra vùng crop rộng gấp 2 lần bounding box gốc theo chiều ngang. Điều này đủ để bao phủ:

- **Phía trên:** Toàn bộ tóc và trán, kể cả tóc xõa.
- **Hai bên:** Tai và khoảng không gian ngoài tai.
- **Phía dưới:** Cổ và phần trên vai (tùy tỉ lệ khuôn mặt trong ảnh).

Thực nghiệm trên bộ dữ liệu chân dung đa dạng cho thấy $m = 0,5$ là điểm cân bằng tốt: nhỏ hơn ($m = 0,3$) thường cắt mất tóc, lớn hơn ($m = 0,7$) đôi khi đưa quá nhiều nền làm nhiễu quá trình chuyển đổi phong cách.

Cơ chế đằng sau sự đánh đổi. Hai đầu của dải giá trị m gây ra hai loại lỗi có bản chất khác nhau:

- Khi m **quá nhỏ**, lỗi mang tính *ngoài phân phối*: mẫu ảnh thiếu tóc và cổ không giống bất kỳ ảnh chân dung nào trong tập huấn luyện, nên mô hình xử lý theo cách khó dự đoán.
- Khi m **quá lớn**, lỗi mang tính *pha loãng*: khuôn mặt chỉ còn chiếm $256/(1+2m) \approx 107$ pixel bề rộng khi $m = 0,7$, đồng thời nền phức tạp chiếm phần lớn diện tích và chi phối các thống kê phong cách mà LADE cùng External Attention học được (xem Chương 2, Mục 2.4.2).

Nhận xét phản biện: Có hai điểm trong thiết kế hiện tại mà luận văn cho rằng cần được ghi nhận thẳng thắn. **Thứ nhất, việc mở rộng theo chiều dọc là đối xứng, trong khi cấu trúc đầu người thì không.** Phần cần bao thêm ở phía trên (tóc, đỉnh đầu) thường nhiều hơn phần cần bao ở phía dưới, nhưng thuật toán chia đều y_d cho hai phía. Với người tóc dài hoặc đội mũ, đỉnh đầu vẫn có thể bị cắt trong khi phần dưới lại thừa nền. Một cải tiến đơn giản là dùng tỉ lệ bất đối xứng, chẳng hạn $0,6 y_d$ cho phía trên và $0,4 y_d$ cho phía dưới. **Thứ hai, năm điểm landmark do RetinaFace trả về hoàn toàn không được sử dụng.** Đây là thông tin có sẵn, không tốn thêm chi phí tính toán, và có thể dùng để ước lượng góc nghiêng của khuôn mặt (từ vector nối hai mắt) rồi xoay ảnh về phương thẳng đứng trước khi cắt. Với các khuôn mặt nghiêng — đúng nhóm mà Chương 4 ghi nhận chất lượng kém nhất — cải tiến này nhiều khả năng mang lại hiệu quả rõ rệt. Cả hai điểm đều được đưa vào phần đề xuất hướng phát triển ở Chương 5.

3.4. Phân tích Độ phân giải Hiệu dụng của Khuôn mặt

Mục này trình bày một phân tích định lượng nhằm trả lời câu hỏi trung tâm của chương: vì sao việc xử lý riêng khuôn mặt lại cải thiện chất lượng, và cải thiện đó lớn đến mức nào trong từng tình huống?

3.4.1. Định nghĩa và công thức

Gọi w là bề rộng bounding box khuôn mặt trong ảnh gốc kích thước (H, W) , và s là hệ số thu nhỏ do bước giới hạn cạnh dài 1280 pixel (Mục 3.5.1):

$$s = \min\left(1, \frac{1280}{\max(H, W)}\right) \quad (3.24)$$

Định nghĩa **độ phân giải hiệu dụng của khuôn mặt** là bề rộng (tính bằng pixel) mà khuôn mặt chiếm trong ảnh thực sự được đưa vào Generator. Với hai chế độ:

$$w_{\text{eff}}^{\text{Landscape}} = s \cdot w \quad (3.25)$$

$$w_{\text{eff}}^{\text{Face}} = \frac{256}{1 + 2m} = 128 \text{ px} \quad (\text{theo Tính chất 3, Mục 3.3.3}) \quad (3.26)$$

Hệ số lợi ích về độ phân giải:

$$\kappa = \frac{w_{\text{eff}}^{\text{Face}}}{w_{\text{eff}}^{\text{Landscape}}} = \frac{128}{s \cdot w} \quad (3.27)$$

3.4.2. Diễn giải và điểm hòa vốn

Công thức (3.27) cho một kết luận rõ ràng: **Face Mode chỉ mang lại lợi ích về độ phân giải khi khuôn mặt hẹp hơn 128 pixel trong ảnh đã giới hạn kích thước**. Bảng 3.2 minh họa bằng các tình huống điển hình.

Bảng 3.2. Độ phân giải hiệu dụng của khuôn mặt theo từng kịch bản ảnh

Kịch bản	$s \cdot w$ (px)	Landscape	Face Mode	κ
Ảnh nhóm, chụp toàn thân	40	40 px	128 px	3,2×
Chân dung bán thân	80	80 px	128 px	1,6×
<i>Điểm hòa vốn</i>	<i>128</i>	<i>128 px</i>	<i>128 px</i>	<i>1,0×</i>
Chân dung cận cảnh	300	300 px	128 px	0,43×

Kết quả này thoát nhìn có vẻ mâu thuẫn với động lực ban đầu của chương, nhưng thực ra nó *làm rõ* động lực đó. Face Mode không phải là kỹ thuật “luôn tốt hơn”, mà là kỹ thuật **chuẩn hóa**: nó đảm bảo khuôn mặt luôn được xử lý ở đúng 128 pixel, bất kể ảnh gốc như thế nào. Với ảnh mà khuôn mặt vốn đã lớn, việc chuẩn hóa này thậm chí làm giảm độ phân giải.

Tuy nhiên, độ phân giải chỉ là **một trong ba cơ chế** qua đó Face Mode cải thiện chất lượng. Hai cơ chế còn lại không phụ thuộc kích thước khuôn mặt:

1. **Khớp phân phối huấn luyện.** Generator được huấn luyện trên các mẫu ảnh 256×256 . Trong Landscape Mode, ảnh đầu vào có thể lên tới 1280×960 — mô hình vẫn

chạy được nhờ kiến trúc hoàn toàn tích chập, nhưng trường tiếp nhận hiệu dụng của nó (được định hình trong quá trình huấn luyện ở 256×256) giờ chỉ bao phủ một phần nhỏ ảnh. Các mô-típ phong cách mà mô hình học được ở thang đo huấn luyện không còn tương ứng với thang đo suy luận. Face Mode loại bỏ hoàn toàn sự lệch pha này.

2. **Loại bỏ nhiễu từ nền.** Các cơ chế toàn cục của DTGAN — chuẩn hóa LADE lấy thống kê trên toàn bản đồ đặc trưng, External Attention gộp thông tin trên toàn ảnh — đều bị chi phối bởi vùng chiếm diện tích lớn. Khi nền phức tạp chiếm 90% khung hình, thống kê phong cách áp lên khuôn mặt thực chất được quyết định bởi nền. Cắt bỏ nền khiến toàn bộ năng lực của các cơ chế này tập trung vào chính khuôn mặt.

***Thảo luận:** Phân tích trên đưa ra một dự đoán có thể kiểm chứng mà luận văn cho là đáng nêu ra dù chưa có điều kiện kiểm định đầy đủ: mức cải thiện của Face Mode so với Landscape Mode phải giảm dần khi kích thước khuôn mặt trong ảnh tăng lên, và tiệm cận về mức chỉ còn hai cơ chế phi-độ-phân-giải đóng góp. Nếu đúng, điều này có hai hệ quả thực tiễn. Thứ nhất, chỉ số FID tổng hợp báo cáo ở Chương 4 (58,6 so với 64,2) là một giá trị trung bình che giấu sự phân tán lớn theo kích thước khuôn mặt — một phân tích tách theo nhóm kích thước sẽ cho bức tranh chính xác hơn nhiều. Thứ hai, hệ thống hoàn toàn có thể được cải tiến bằng một quy tắc **lựa chọn chế độ tự động**: khi $s \cdot w > 128$, Landscape Mode đã đủ tốt và việc kích hoạt Face Mode chỉ làm tốn thêm một lần suy luận mà không thu được gì. Đây là một cải tiến rẻ, dễ cài đặt, và luận văn đề xuất đưa vào phiên bản tiếp theo của hệ thống.*

3.5. Luồng xử lý Ảnh trong Pipeline

3.5.1. Đọc và Giới hạn Kích thước Ảnh

Ảnh đầu vào được đọc bằng Pillow và chuyển sang không gian màu RGB. Để đảm bảo hệ thống xử lý được ảnh có độ phân giải cao mà không gặp vấn đề bộ nhớ, cạnh dài nhất của ảnh được giới hạn ở 1280 pixel:

$$\text{scale} = \frac{1280}{\max(H, W)}, \quad \text{nếu } \max(H, W) > 1280 \quad (3.28)$$

Ảnh được resize về $(\lfloor W \cdot \text{scale} \rfloor, \lfloor H \cdot \text{scale} \rfloor)$ trong khi giữ nguyên tỉ lệ khung hình.

Ngưỡng 1280 phục vụ ba mục đích cùng lúc: (i) chặn trên bộ nhớ kích hoạt của Generator, vốn tăng tuyến tính theo số pixel; (ii) chặn trên số lượng prior box mà RetinaFace phải hậu xử lý (Phương trình 3.5); và (iii) đảm bảo thời gian phản hồi ổn định cho ứng dụng web, không phụ thuộc vào việc người dùng tải lên ảnh 2 megapixel hay 50 megapixel.

Một lưu ý quan trọng về Hybrid Mode: việc phát hiện khuôn mặt được thực hiện trên ảnh đã giới hạn kích thước, trong khi ảnh nền cuối cùng được tạo ở kích thước gốc. Sự chênh lệch không gian này đòi hỏi một bước ánh xạ tọa độ ngược, trình bày tại Mục 3.6.2.

3.5.2. Xử lý Phân nhánh: Face / Landscape / Hybrid

Sau bước đọc ảnh, pipeline phân nhánh dựa trên hai cờ `face_mode` và `hybrid_mode`:

Bảng 3.3. So sánh ba chế độ xử lý trong pipeline

Tiêu chí	Face Mode	Landscape Mode	Hybrid Mode
Phát hiện mặt	RetinaFace (1 mặt)	Không	RetinaFace (tất cả)
Margin Expansion	Có ($m = 0,5$)	Không	Có (mỗi mặt)
Kích thước input	256×256	Giữ tỉ lệ, chia hết 8	Nền: chia hết 8; Mặt: 256^2
Số lần inference	1	1	$1 + N_{\text{faces}}$
Ưu điểm	Chất lượng mặt cao	Giữ toàn cảnh	Cả nền lẫn mặt chất lượng cao
Nhược điểm	Mất background	Mặt nhỏ kém	Chi phí tính toán cao
Phù hợp	Portrait, avatar	Phong cảnh	Ảnh nhóm, gia đình

Quy tắc resize Landscape mode: Kích thước ảnh phải chia hết cho 8, đồng thời có ngưỡng tối thiểu 256 pixel để đảm bảo chất lượng đầu ra. Hàm $\text{to_8s}(x)$:

$$\text{to_8s}(x) = \begin{cases} 256 & \text{nếu } x < 256 \\ x - (x \bmod 8) & \text{nếu } x \geq 256 \end{cases} \quad (3.29)$$

Vì sao phải chia hết cho 8? Base Encoder của DTGAN thực hiện 3 lần downsampling stride-2 (Chương 2, Mục 2.2.1), tổng hệ số thu nhỏ là $2^3 = 8$. Nếu kích thước đầu vào không chia hết cho 8, các phép chia lấy nguyên trong quá trình downsampling sẽ làm mất một vài hàng/cột pixel; khi decoder upsample trở lại bằng hệ số 2 ba lần, kích thước đầu ra không khớp với kích thước các đặc trưng skip connection tương ứng, gây lỗi ghép tensor. Ràng buộc chia hết cho 8 loại bỏ hoàn toàn khả năng này.

Đối với các mô hình biến thể nhỏ (có hậu tố `_tiny_` trong tên file ONNX), hệ thống sử dụng hàm $\text{to_16s}(x)$ thay thế — yêu cầu kích thước chia hết cho 16 do kiến

trúc encoder có thêm một cấp downsampling:

$$\text{to_16s}(x) = \begin{cases} 256 & \text{nếu } x < 256 \\ x - (x \bmod 16) & \text{nếu } x \geq 256 \end{cases} \quad (3.30)$$

3.5.3. Chuẩn hóa và Suy luận ONNX

Trước khi đưa vào mô hình ONNX, ảnh được chuẩn hóa từ $[0, 255]$ sang $[-1, 0; 1, 0]$:

$$x_{\text{norm}} = \frac{x}{127,5} - 1,0 \quad (3.31)$$

Suy luận qua ONNX Runtime:

$$\hat{y} = \text{session.run}(\text{None}, \{ \text{"input"} : x_{\text{norm}} \}) [0] \quad (3.32)$$

Sau suy luận, ảnh đầu ra được chuyển ngược về $[0, 255]$:

$$\text{output} = \frac{\hat{y} + 1,0}{2,0} \times 255 \quad (3.33)$$

Khoảng chuẩn hóa $[-1, 1]$ không phải lựa chọn tùy ý mà bị quy định bởi kiến trúc: lớp cuối của Generator dùng hàm kích hoạt tanh với miền giá trị $(-1, 1)$ (Chương 2, Mục 2.2.2). Đầu vào và đầu ra phải cùng thang đo để các hàm mất mát so sánh được trực tiếp trong quá trình huấn luyện, và quy ước đó phải được giữ nguyên khi suy luận.

Vai trò của ONNX trong hệ thống. Việc chuyển mô hình sang định dạng ONNX [37] mang lại ba lợi ích cụ thể cho luận văn: (i) tách hoàn toàn khâu triển khai khỏi TensorFlow 1.x — một framework đã ngừng hỗ trợ và khó cài đặt trên môi trường hiện đại; (ii) cho phép cùng một file mô hình chạy trên CPU lẫn GPU chỉ bằng cách đổi execution provider; (iii) tối ưu hóa đồ thị tính toán khi tải mô hình (gộp các lớp liên tiếp, loại bỏ nhánh chết), góp phần vào tốc độ đo được ở Chương 4.

3.5.4. Tóm tắt Luồng Dữ liệu

Toàn bộ luồng xử lý ảnh trong pipeline:

1. **Đọc ảnh:** `Image.open(path).convert("RGB")`
2. **Giới hạn kích thước:** Nếu $\max(H, W) > 1280$ thì resize về scale 1280.
3. **[Face Mode] Phát hiện mặt:** `face_det.detect_face(np.array(img))`
 $\rightarrow \text{bboxes}, \text{points}$
4. **[Face Mode] Mở rộng vùng:** Nếu phát hiện được mặt: `margin_face(bboxes[0], img.shape[:2])` $\rightarrow \text{margin_box}$

5. **[Face Mode] Cắt ảnh:** `img = img[y1:y2, x1:x2]`

6. **Resize cho model:**

- Face mode: `img.resize((256, 256))`
- Landscape: `img.resize((to_8s(w), to_8s(h)))`

7. **Chuẩn hóa:** $x_{\text{norm}} = x/127,5 - 1,0$

8. **ONNX Inference:** `session.run(None, {"input": img})[0]`

9. **Hậu chuẩn hóa:** `output = (y + 1,0)/2,0 × 255`

10. **Lưu/hiển thị kết quả.**

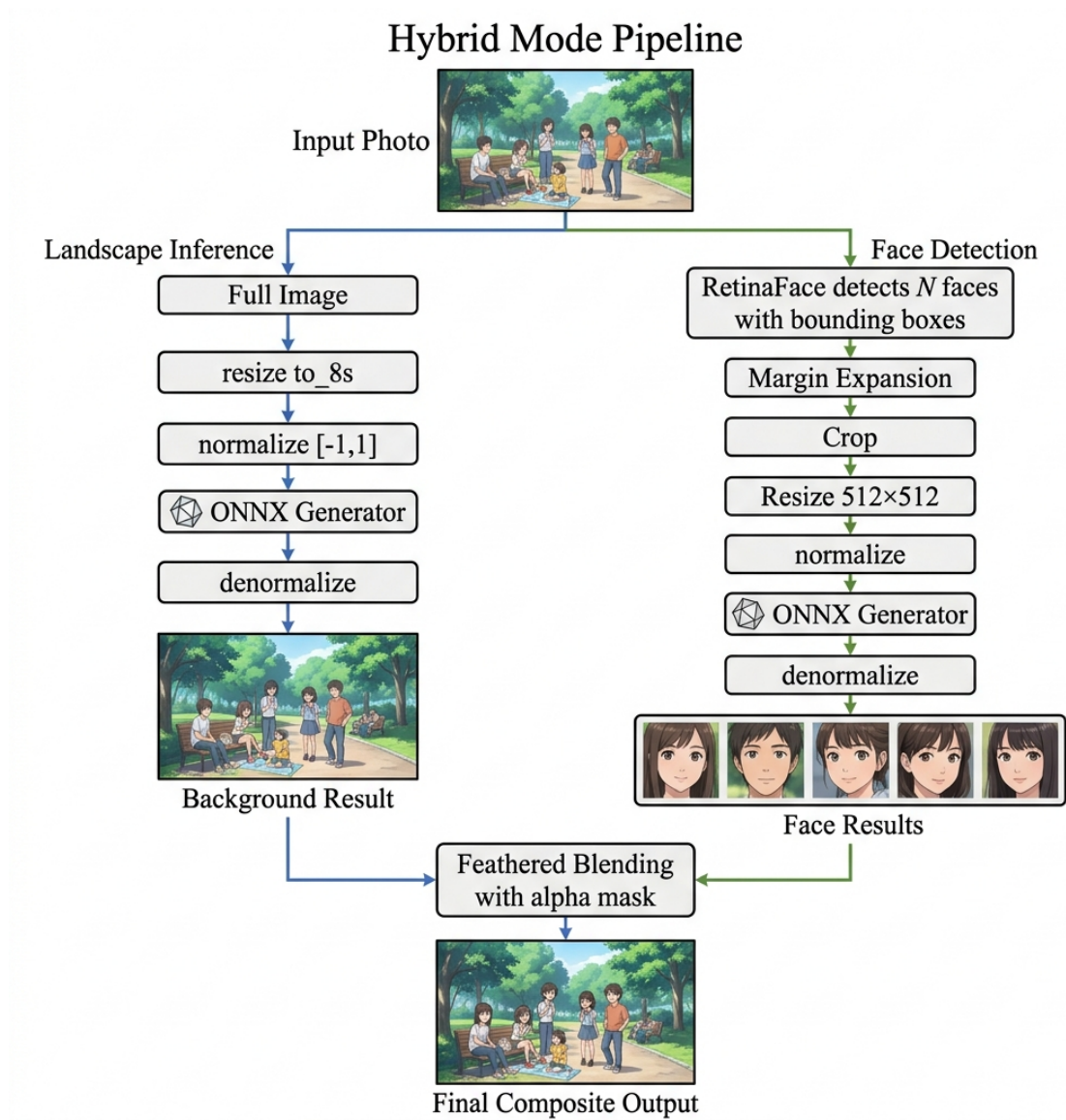
3.6. Chế độ Hybrid Mode

Ngoài hai chế độ Face Mode và Landscape Mode, hệ thống còn hỗ trợ chế độ **Hybrid Mode** — kết hợp ưu điểm của cả hai để xử lý ảnh nhóm người, trong đó cần giữ cả chất lượng nền lẫn chi tiết khuôn mặt cao. Đây là chế độ phức tạp nhất về mặt tính toán nhưng cho kết quả toàn diện nhất.

3.6.1. Nguyên lý Hoạt động

Hybrid Mode thực hiện **nhiều lần suy luận** cho mỗi ảnh đầu vào ($1 + N_{\text{faces}}$ lần), theo bốn giai đoạn tuần tự:

1. **Phát hiện khuôn mặt:** Gọi `detect_face()` trên ảnh đã giới hạn kích thước (≤ 1280 pixel cạnh dài). Nếu không phát hiện khuôn mặt nào, tự động chuyển sang Landscape Mode (graceful degradation) và chỉ thực hiện một lần suy luận duy nhất.
2. **Suy luận nền (Background Pass):** Toàn bộ ảnh gốc được xử lý ở chế độ Landscape (resize `to_8s`, chuẩn hóa $[-1, 1]$, suy luận ONNX, hậu chuẩn hóa) để tạo ảnh nền anime hoàn chỉnh kích thước gốc (W, H).
3. **Suy luận từng khuôn mặt (Face Pass):** Với **mỗi** khuôn mặt phát hiện được (không chỉ khuôn mặt lớn nhất như Face Mode), áp dụng Margin Expansion, cắt vùng mặt từ ảnh giới hạn kích thước, resize về 256×256 và suy luận riêng qua cùng mô hình ONNX.
4. **Ghép ngược (Composite Pass):** Mỗi khuôn mặt chất lượng cao được ánh xạ tọa độ về ảnh gốc, resize về kích thước vùng mặt gốc, rồi ghép vào ảnh nền bằng kỹ thuật *feathered blending*.



Hình 3.4. Pipeline xử lý Hybrid Mode: kết hợp Landscape Inference cho nền và Face Inference cho từng khuôn mặt

Thuật toán 3.2 Hybrid Mode — kết hợp nền Landscape và khuôn mặt Face Mode

Đầu vào: Ảnh gốc I kích thước (H, W) ; mô hình ONNX $\mathcal{G}$; bộ phát hiện $\mathcal{D}$

Đầu ra: Ảnh anime I_{out} kích thước (H, W)

- 1: $I_{\text{lim}}, s \leftarrow \text{GIÓIHẠNCẠNHĐÀI}(I, 1280)$
 - 2: $\mathcal{B} \leftarrow \mathcal{D}(I_{\text{lim}})$ ▷ danh sách bounding box
 - 3: **nếu** $\mathcal{B} = \emptyset$ **thì**
 - 4: **trả về** $\text{LANDSCAPEMODE}(I)$ ▷ hạ cấp mềm
 - 5: **hết nếu**
 - 6: $I_{\text{bg}} \leftarrow \text{SUÝLUẬN}(\mathcal{G}, I)$ ▷ Background Pass, giữ kích thước gốc
 - 7: **với mỗi** $b \in \mathcal{B}$ **thực hiện**
 - 8: $b' \leftarrow \text{MARGINEXPANSION}(b)$ ▷ Thuật toán 3.1
 - 9: $C \leftarrow \text{CẮT}(I_{\text{lim}}, b')$; $C \leftarrow \text{RESIZE}(C, 256 \times 256)$
 - 10: $\hat{C} \leftarrow \text{SUÝLUẬN}(\mathcal{G}, C)$ ▷ Face Pass
 - 11: $b'_{\text{orig}} \leftarrow b'/s$ ▷ ánh xạ về hệ tọa độ ảnh gốc
 - 12: $\hat{C} \leftarrow \text{RESIZE}(\hat{C}, w_f \times h_f)$ ▷ w_f, h_f suy từ b'_{orig}
 - 13: $M \leftarrow \text{TẠOMẶTNẠLÔNG}(w_f, h_f, \delta = 0,08 \min(w_f, h_f))$
 - 14: $I_{\text{bg}} \leftarrow \text{GHÉPALPHA}(I_{\text{bg}}, \hat{C}, M, b'_{\text{orig}})$
 - 15: **hết với mỗi**
 - 16: **trả về** I_{bg}
-

3.6.2. Ánh xạ Tọa độ giữa Ảnh Giới hạn và Ảnh Gốc

Do phát hiện khuôn mặt thực hiện trên ảnh đã giới hạn kích thước (≤ 1280 pixel), trong khi kết quả cuối cùng được ghép lên ảnh nền kích thước gốc (W, H) , hệ thống cần ánh xạ tọa độ bounding box từ không gian ảnh giới hạn sang không gian ảnh gốc:

$$s = \begin{cases} \frac{1280}{\max(W, H)} & \text{nếu } \max(W, H) > 1280 \\ 1,0 & \text{ngược lại} \end{cases} \quad (3.34)$$

Cho bounding box sau Margin Expansion (x_1, y_1, x_2, y_2) trong ảnh giới hạn, tọa độ tương ứng trên ảnh gốc:

$$x_1^{\text{orig}} = \left\lfloor \frac{x_1}{s} + 0,5 \right\rfloor, \quad y_1^{\text{orig}} = \left\lfloor \frac{y_1}{s} + 0,5 \right\rfloor, \quad x_2^{\text{orig}} = \left\lfloor \frac{x_2}{s} + 0,5 \right\rfloor, \quad y_2^{\text{orig}} = \left\lfloor \frac{y_2}{s} + 0,5 \right\rfloor \quad (3.35)$$

trong đó biểu thức $\lfloor z + 0,5 \rfloor$ là phép làm tròn tới số nguyên gần nhất — chính xác hơn phép cắt cụt $\lfloor z \rfloor$, vốn luôn thiên lệch về phía nhỏ hơn và tích lũy sai số dịch chuyển vùng ghép về góc trên bên trái.

Kích thước vùng mặt trên ảnh gốc:

$$w_f = \max(1, x_2^{\text{orig}} - x_1^{\text{orig}}), \quad h_f = \max(1, y_2^{\text{orig}} - y_1^{\text{orig}}) \quad (3.36)$$

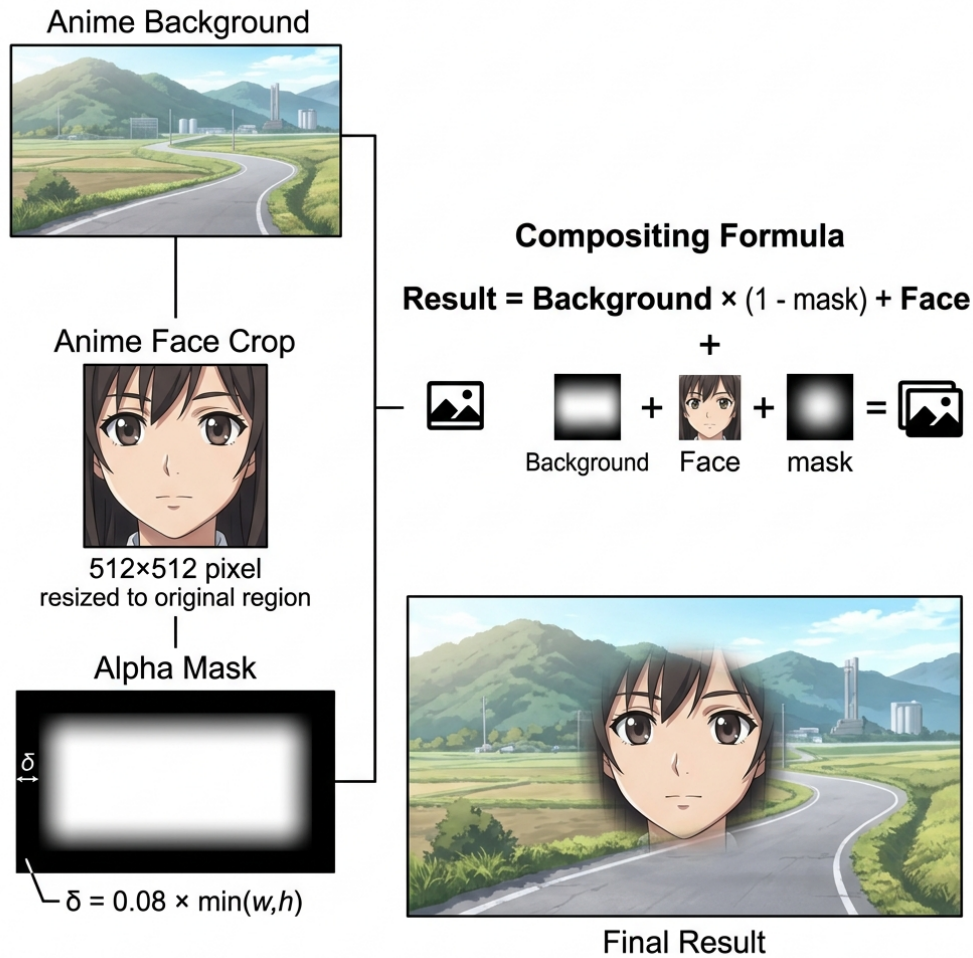
với toán tử $\max(1, \cdot)$ đảm bảo kích thước luôn hợp lệ, tránh lỗi chia cho 0 trong các phép resize tiếp theo khi bounding box suy biến.

Ảnh khuôn mặt sau suy luận (kích thước 256×256) được resize về (w_f, h_f) bằng thuật toán nội suy Lanczos trước khi ghép. Lanczos được chọn thay vì bilinear vì nó dùng nhân lọc dựa trên hàm sinc, bảo toàn tốt hơn các thành phần tần số cao — chính là những đường nét sắc mà toàn bộ pipeline đã nỗ lực tạo ra. Việc dùng một bộ nội suy làm mờ ở bước cuối sẽ triệt tiêu phần lớn lợi ích của các bước trước đó.

***Nhận xét:** Bước ánh xạ tọa độ này là hệ quả trực tiếp của một quyết định thiết kế: phát hiện khuôn mặt trên ảnh đã thu nhỏ để tiết kiệm chi phí, nhưng dựng nền ở kích thước gốc để giữ chất lượng. Cái giá phải trả là sai số làm tròn: với ảnh 4000 pixel thu về 1280 ($s = 0,32$), mỗi pixel sai lệch trong không gian phát hiện tương ứng khoảng 3 pixel trong ảnh gốc. Sai số này không ảnh hưởng tới chất lượng khuôn mặt được sinh ra, nhưng làm vùng ghép có thể lệch vài pixel so với vị trí lý tưởng. Kỹ thuật feathered blending ở mục tiếp theo, với vùng chuyển tiếp rộng khoảng 8% kích thước khuôn mặt, đủ sức che lấp mức lệch này — một ví dụ cho thấy hai thành phần trong pipeline có thể bù trừ lẫn nhau, và việc phân tích chúng tách rời sẽ bỏ sót mối quan hệ đó.*

3.6.3. Kỹ thuật Feathered Blending

Khi ghép trực tiếp vùng mặt (xử lý Face Mode ở 256×256) vào nền (xử lý Landscape ở độ phân giải khác), sẽ xuất hiện đường biên rõ ràng do khác biệt về phong cách và độ phân giải giữa hai vùng. Kỹ thuật **feathered blending** sử dụng mặt nạ alpha với viền mờ Gaussian để tạo vùng chuyển tiếp mượt mà.



Hình 3.5. Kỹ thuật Feathered Blending: mặt nạ alpha với viền mờ Gaussian tạo vùng chuyển tiếp liền mạch

Quy trình tạo mặt nạ alpha gồm 3 bước:

Bước 1 — Tạo mặt nạ nền đen: Khởi tạo ảnh grayscale kích thước (w_f, h_f) với tất cả pixel có giá trị 0:

$$M(x, y) = 0 \quad \forall (x, y) \in [0, w_f) \times [0, h_f) \quad (3.37)$$

Bước 2 — Vẽ vùng trắng thu nhỏ: Vẽ hình chữ nhật trắng (giá trị 255) bên trong mặt nạ, thu nhỏ mỗi cạnh một lề δ :

$$\delta = \lfloor 0.08 \cdot \min(w_f, h_f) \rfloor \quad (3.38)$$

$$M(x, y) = \begin{cases} 255 & \text{nếu } \delta \leq x < w_f - \delta \text{ và } \delta \leq y < h_f - \delta \\ 0 & \text{ngược lại} \end{cases} \quad (3.39)$$

Bước 3 — Làm mờ Gaussian: Áp dụng bộ lọc Gaussian Blur với bán kính δ lên

mặt nạ, tạo vùng chuyển tiếp gradient từ 255 (trung tâm) về 0 (biên):

$$\hat{M} = M * G_\delta, \quad G_\delta(x, y) = \frac{1}{2\pi\delta^2} e^{-\frac{x^2+y^2}{2\delta^2}} \quad (3.40)$$

Phép ghép cuối cùng sử dụng alpha compositing theo mặt nạ $\hat{M}$:

$$I_{\text{out}}(x, y) = \underbrace{\frac{\hat{M}(x, y)}{255}}_{\alpha(x, y)} \cdot I_{\text{face}}(x, y) + \left(1 - \frac{\hat{M}(x, y)}{255}\right) \cdot I_{\text{bg}}(x, y) \quad (3.41)$$

trong đó I_{face} là ảnh khuôn mặt đã suy luận (resize về $w_f \times h_f$), I_{bg} là ảnh nền anime tại vùng $(x_1^{\text{orig}}, y_1^{\text{orig}})$, và $\alpha(x, y) \in [0, 1]$ là hệ số pha trộn.

Phân tích cơ chế. Ba bước trên phối hợp tạo ra một hàm α có ba vùng rõ rệt:

- **Vùng lõi** (cách biên hơn 2δ): $\alpha \approx 1$, ảnh đầu ra lấy hoàn toàn từ khuôn mặt chất lượng cao.
- **Vùng chuyển tiếp** (dải rộng khoảng 2δ quanh biên hình chữ nhật trong): α biến thiên trơn từ 1 xuống 0 theo dạng hàm lõi (tích phân của Gaussian).
- **Vùng ngoài cùng** (sát mép): $\alpha \approx 0$, ảnh đầu ra lấy hoàn toàn từ nền.

Việc thu hình chữ nhật trắng vào trong một lễ δ trước khi làm mờ với bán kính cũng bằng δ là chi tiết then chốt: nó đảm bảo đuôi của hàm Gaussian vừa đủ chạm tới mép ngoài và tắt hẳn tại đó. Nếu vẽ hình chữ nhật trắng phủ kín rồi mới làm mờ, giá trị α tại mép ngoài sẽ là 0,5 chứ không phải 0 — và đường biên cứng vẫn hiện ra, chỉ mờ hơn.

Hệ số $\delta = 8\%$ kích thước cạnh nhỏ hơn của vùng mặt được chọn qua thực nghiệm: nhỏ hơn sẽ tạo đường viền rõ, lớn hơn sẽ làm mờ quá nhiều chi tiết khuôn mặt.

Nhận xét phản biện: Feathered blending là giải pháp đơn giản và rẻ, nhưng có ba hạn chế mà luận văn cho rằng cần nêu rõ. **Thứ nhất, mặt nạ có dạng hình chữ nhật chứ không bám theo đường viền khuôn mặt.** Hệ quả là vùng chuyển tiếp cắt ngang qua tóc, vai và nền một cách tùy tiện; ở những nơi này, ảnh đầu ra là trung bình có trọng số của hai phiên bản anime hóa khác nhau của cùng một nội dung, có thể gây hiện tượng chồng mờ nhẹ. **Thứ hai, alpha blending không hiệu chỉnh chênh lệch tông màu.** Nếu hai lần suy luận cho ra tông sáng khác biệt — điều hoàn toàn có thể xảy ra vì LADE tính thống kê chuẩn hóa trên toàn ảnh, mà hai đầu vào có nội dung khác nhau — thì phép trung bình chỉ làm mềm ranh giới chứ không xóa được sự chênh lệch. Kỹ thuật Poisson Image Editing [2], giải phương trình Poisson với điều kiện biên Dirichlet để bảo toàn gradient của vùng ghép trong khi khớp giá trị tuyệt đối tại biên, giải quyết đúng vấn đề này và được đề xuất ở Chương 5. **Thứ ba, tham số $\delta = 8\%$ là hằng số duy nhất cho mọi ảnh,**

trong khi mức chênh lệch giữa hai lần suy luận lại thay đổi theo từng ảnh — một cơ chế điều chỉnh δ thích ứng theo độ lệch tông màu đo được sẽ hợp lý hơn. Mặc dù vậy, cần đặt các hạn chế này trong đúng bối cảnh: kết quả khảo sát người dùng ở Chương 4 cho thấy phần lớn người tham gia ưu tiên đầu ra Hybrid Mode, nên giải pháp hiện tại đã đủ tốt cho mục tiêu ứng dụng — các cải tiến nêu trên thuộc loại tinh chỉnh chứ không phải sửa lỗi.

3.6.4. Phân tích Độ phức tạp

Gọi T_L là thời gian suy luận Landscape, T_F là thời gian suy luận Face (256×256), T_D là thời gian phát hiện mặt, T_{blend} là thời gian tạo mặt nạ và ghép, N là số khuôn mặt phát hiện được. Tổng thời gian xử lý Hybrid Mode:

$$T_{\text{hybrid}} = T_D + T_L + N \cdot (T_F + T_{\text{blend}}) \quad (3.42)$$

trong đó $T_{\text{blend}} \ll T_F$ (tạo mask và ghép ảnh là các phép toán trên mảng, rất nhanh so với suy luận mạng nơ-ron). So sánh với các chế độ khác:

Bảng 3.4. Phân tích độ phức tạp thời gian giữa ba chế độ

Chế độ	Số lần suy luận ONNX	Tổng thời gian
Face Mode	1	$T_D + T_F$
Landscape Mode	1	T_L
Hybrid Mode	$1 + N$	$T_D + T_L + N \cdot T_F$

Chi phí tính toán của Hybrid Mode tăng **tuyến tính** theo số khuôn mặt. Tuy nhiên, do T_F sử dụng kích thước đầu vào cố định 256×256 — thường nhỏ hơn ảnh Landscape — thời gian mỗi lần suy luận Face thường nhanh hơn suy luận Landscape. Trong thực nghiệm (Chương 4), ảnh có 2–3 khuôn mặt thường hoàn thành Hybrid Mode trong thời gian chấp nhận được cho ứng dụng tương tác.

Hướng tối ưu hóa khả thi. Do N lần suy luận khuôn mặt hoàn toàn độc lập với nhau, chúng có thể được gộp thành **một lần suy luận theo lô** (batch inference) với tensor đầu vào kích thước $N \times 3 \times 256 \times 256$. Trên GPU, chi phí của một lô N ảnh nhỏ hơn nhiều so với N lần chạy riêng lẻ, vì phần lớn thời gian của mỗi lần chạy riêng là chi phí cố định (khởi động kernel, truyền dữ liệu) chứ không phải tính toán thực sự. Cải tiến này sẽ biến thành phần $N \cdot T_F$ thành gần như hằng số với N nhỏ, và được khuyến nghị cho phiên bản tiếp theo.

3.7. Phân tích Kỹ thuật và Đánh giá Thiết kế

3.7.1. Lý do Chọn RetinaFace MobileNet 0.25

So với các phương pháp phát hiện khuôn mặt phổ biến khác, RetinaFace với backbone MobileNet 0.25 có ưu thế rõ ràng trong bối cảnh triển khai thực tế:

Bảng 3.5. So sánh các phương pháp phát hiện khuôn mặt cho ứng dụng thực tế

Phương pháp	Tốc độ	Landmark	Multi-scale	Model size
Viola-Jones [4]	Rất nhanh	Không	Có hạn	Nhỏ
MTCNN [20]	Trung bình	5 điểm	Tốt	Nhỏ
SSD Face [16]	Nhanh	Không	Tốt	Trung bình
RetinaFace MN0.25 [42]	Nhanh	5 điểm	Rất tốt	Rất nhỏ
RetinaFace ResNet50	Chậm	5 điểm	Xuất sắc	Lớn

Các lý do kỹ thuật cụ thể:

- **Tương thích ONNX Runtime:** RetinaFace được xuất sang ONNX một cách tự nhiên, dùng chung runtime với AnimeGANv3. Hệ thống vì vậy chỉ cần một thư viện suy luận duy nhất — giảm đáng kể độ phức tạp triển khai và kích thước gói cài đặt.
- **5 facial landmarks:** Thông tin landmarks (dù chưa được khai thác trong phiên bản hiện tại) mở ra khả năng face alignment trong các phiên bản tương lai, không tốn thêm chi phí tính toán.
- **Multi-scale FPN:** Phát hiện tốt cả khuôn mặt nhỏ trong ảnh góc rộng lẫn khuôn mặt lớn trong ảnh cận cảnh — yêu cầu bắt buộc với Hybrid Mode xử lý ảnh nhóm.
- **Kích thước model nhỏ (~500 KB):** Cùng với model AnimeGANv3 9,1 MB, tổng dung lượng hệ thống dưới 10 MB — phù hợp để đóng gói vào file thực thi hoặc tải về trong ứng dụng web.

Vì sao không chọn MTCNN? MTCNN cũng cung cấp 5 landmark với kích thước mô hình nhỏ, nhưng dùng kiến trúc ba tầng nối tiếp (P-Net, R-Net, O-Net) hoạt động trên kim tự tháp ảnh. Cách này khiến thời gian xử lý *phụ thuộc vào số lượng ứng viên* phát sinh ở mỗi tầng, gây biến động lớn giữa các ảnh — đặc tính bất lợi cho một dịch vụ web cần thời gian phản hồi ổn định. RetinaFace là mô hình một lượt, chi phí gần như cố định theo kích thước ảnh.

3.7.2. Đánh giá Pipeline Tổng thể

Pipeline được thiết kế theo nguyên tắc **hạ cấp mềm** (graceful degradation): nếu không phát hiện được khuôn mặt (confidence dưới ngưỡng 0,8, hoặc khuôn mặt bị che khuất), hệ thống tự động chuyển sang chế độ Landscape để xử lý toàn bộ ảnh, tránh trả về lỗi cho người dùng. Cơ chế này áp dụng cho cả Face Mode lẫn Hybrid Mode.

Bảng 3.3 tóm tắt sự đánh đổi giữa ba chế độ. Face Mode cho chất lượng anime cao nhất đối với ảnh chân dung do ba cơ chế đã phân tích ở Mục 3.4.2: chuẩn hóa độ phân giải khuôn mặt về 128 pixel, khớp phân phối huấn luyện 256×256 , và loại bỏ ảnh hưởng của nền lên các cơ chế thống kê toàn cục.

Hybrid Mode khắc phục nhược điểm mất nền của Face Mode bằng cách xử lý riêng từng khu vực, đổi lại chi phí tính toán tăng tuyến tính theo số khuôn mặt.

3.7.3. Hạn chế của Thiết kế

Để đánh giá cân bằng, cần chỉ rõ những hạn chế nội tại của pipeline — ngoài các hạn chế cục bộ đã nêu ở từng mục:

1. **Sai sót lan truyền một chiều.** Hai mô hình mắc nối tiếp mà không có cơ chế phản hồi: nếu RetinaFace trả về bounding box lệch (thường gặp ở góc nghiêng $> 60^\circ$), Generator sẽ xử lý một vùng cắt sai và không có cách nào phát hiện điều đó. Không tồn tại bước kiểm tra chất lượng nào giữa hai giai đoạn.
2. **Vùng cắt không vuông gây biến dạng.** Như đã nêu ở Tính chất 2 (Mục 3.3.3), khuôn mặt gần biên trên/dưới cho vùng cắt hình chữ nhật, và phép resize về 256×256 làm biến dạng tỉ lệ. Giải pháp là đệm viền bằng màu trung bình hoặc phản chiếu gương thay vì cắt cụt.
3. **Không khai thác landmark.** Năm điểm đặc trưng sẵn có không được dùng cho căn chỉnh khuôn mặt — bỏ lỡ cơ hội cải thiện đúng nhóm ảnh khó nhất.
4. **Chọn chế độ thủ công.** Người dùng phải tự chọn chế độ, trong khi phân tích ở Mục 3.4 cho thấy hoàn toàn có thể chọn tự động dựa trên tỉ lệ kích thước khuôn mặt.
5. **Không có nhất quán thời gian.** Khi xử lý video, mỗi khung hình được phát hiện và cắt độc lập; bounding box dao động nhẹ giữa các khung hình liên tiếp sẽ khuếch đại thành hiện tượng nhấp nháy ở đầu ra. Vấn đề này được bàn tiếp ở Chương 5.

3.7.4. Cân nhắc về Quyền riêng tư

Do hệ thống xử lý ảnh khuôn mặt — một dạng dữ liệu sinh trắc học — luận văn ghi nhận một số nguyên tắc đã được áp dụng trong quá trình triển khai. Toàn bộ quá trình phát hiện và chuyển đổi diễn ra trong một lần gọi API duy nhất; ảnh đầu vào không được lưu trữ lâu dài trên máy chủ sau khi trả kết quả, và hệ thống không thực hiện bất kỳ thao tác nhận dạng hay đối sánh danh tính nào — RetinaFace chỉ được dùng để *định vị* khuôn mặt, không phải để *nhận diện* người. Các ảnh sử dụng trong phần đánh giá định tính ở Chương 4 đều thuộc nhóm ảnh công khai hoặc có sự đồng ý của người trong ảnh. Với một hệ thống có khả năng sinh ảnh chân dung biến đổi, luận văn cũng lưu ý rằng việc sử

dụng cần tuân thủ nguyên tắc không tạo nội dung gây hiểu nhầm về danh tính hoặc gây tổn hại đến người được chụp.

Thảo luận: Nhìn lại toàn chương, đóng góp của luận văn có thể tóm gọn trong một câu: **thay vì dạy mô hình chú ý tới khuôn mặt, hãy chỉ đưa khuôn mặt cho mô hình xem.** Cách tiếp cận này có ưu điểm rõ ràng — không cần huấn luyện lại, tái sử dụng được mọi mô hình phong cách, mỗi thành phần nâng cấp độc lập — nhưng cũng bộc lộ giới hạn của kiến trúc kiểu ống dẫn: các giai đoạn không thể học từ nhau, và mỗi giai đoạn tối ưu cho mục tiêu riêng của nó chứ không phải cho chất lượng cuối cùng. Một hệ thống huấn luyện đầu-cuối, trong đó tín hiệu về chất lượng ảnh anime có thể truyền ngược tới tận khâu chọn vùng cắt, về lý thuyết sẽ cho kết quả tốt hơn. Nhưng nó cũng sẽ đắt hơn nhiều lần về dữ liệu và tính toán, đồng thời đánh mất tính mô-đun — thứ tài sản quý nhất của giải pháp hiện tại. Trong bối cảnh một luận văn hướng tới hệ thống chạy được trên tài nguyên hạn chế, luận văn cho rằng đánh đổi này là hợp lý, và ghi nhận hướng đầu-cuối như một triển vọng nghiên cứu chứ không phải một thiếu sót cần khắc phục ngay.

Tổng kết chương

Chương này đã trình bày luồng xử lý anime tích hợp phát hiện khuôn mặt — đóng góp kỹ thuật cốt lõi của luận văn:

- **Nguyên tắc thiết kế:** Can thiệp ở tầng pipeline thay vì sửa kiến trúc hay hàm mất mát của DTGAN, qua đó giữ nguyên mô hình đã huấn luyện và bảo toàn tính mô-đun của hệ thống.
- **Pipeline end-to-end** tích hợp ba chế độ Face Mode, Landscape Mode và Hybrid Mode, với cơ chế hạ cấp mềm tự động chuyển chế độ khi không phát hiện khuôn mặt.
- **Luồng dữ liệu qua RetinaFace:** tiền xử lý chuẩn ImageNet → backbone MobileNet 0.25 (tiết kiệm $\approx 9\times$ chi phí nhờ depthwise separable convolution) → FPN đa tỉ lệ → giải mã Prior Box → lọc ngưỡng → NMS → sắp xếp theo diện tích. Bộ tham số (0,8; 0,3; 10; 5) thể hiện nhất quán triết lý ưu tiên độ chính xác hơn độ bao phủ, phù hợp với hậu quả bất đối xứng của hai loại lỗi.
- **Thuật toán Margin Expansion** 6 bước (Thuật toán 3.1): mở rộng bounding box đối xứng, duy trì tỉ lệ 1:1, xử lý phân nhánh khi $h' < h$ và các trường hợp khuôn mặt gần biên ảnh. Tính chất quan trọng nhất được rút ra: với $m = 0,5$, khuôn mặt luôn được đưa vào Generator ở bề rộng cố định **128 pixel**, không phụ thuộc kích thước ảnh gốc.
- **Phân tích độ phân giải hiệu dụng:** Hệ số lợi ích $\kappa = 128/(s \cdot w)$ cho thấy Face Mode mang lại lợi ích về độ phân giải khi khuôn mặt hẹp hơn 128 pixel trong ảnh

đã giới hạn kích thước, và lợi ích lớn nhất ở ảnh nhóm hoặc ảnh chụp xa. Hai cơ chế bổ sung — khớp phân phối huấn luyện và loại bỏ nhiễu nền — đóng góp độc lập với kích thước khuôn mặt.

- **Hybrid Mode với Feathered Blending** (Thuật toán 3.2): xử lý nền Landscape và từng khuôn mặt Face Mode riêng biệt, ghép bằng mặt nạ alpha làm mờ Gaussian với lẽ $\delta = 8\%$. Chi phí tăng tuyến tính theo số khuôn mặt, có thể tối ưu bằng suy luận theo lô.

Chương cũng đã chỉ rõ các hạn chế của thiết kế: sai sót lan truyền một chiều giữa hai mô hình nối tiếp, vùng cắt không vuông ở biên ảnh, landmark chưa được khai thác cho căn chỉnh khuôn mặt, và mặt nạ ghép có dạng hình chữ nhật thay vì bám theo đường viền khuôn mặt.

Chương 4 tiếp theo sẽ kiểm chứng bằng thực nghiệm các phân tích lý thuyết đã trình bày: đánh giá độ chính xác phát hiện khuôn mặt theo từng kịch bản, khảo sát ảnh hưởng của tham số m tới chất lượng đầu ra, so sánh định lượng và định tính giữa ba chế độ, cùng đánh giá cảm nhận từ người dùng thực tế.

CHƯƠNG 4

THỰC NGHIỆM VÀ ĐÁNH GIÁ

Chương này trình bày toàn bộ quá trình thực nghiệm của hệ thống AnimeGANv3 (DTGAN) được xây dựng trong luận văn, bao gồm: mô tả môi trường phần cứng/phần mềm, bộ dữ liệu sử dụng, phân tích đường cong hội tụ huấn luyện, đánh giá định lượng bằng các chỉ số FID và LPIPS, đánh giá định tính bằng kết quả chuyển đổi ảnh trên cả ba chế độ (Face Mode, Landscape Mode, Hybrid Mode), và đánh giá riêng cho module Face Mode.

4.1. Môi trường Thực nghiệm

4.1.1. Cấu hình Phần cứng

Toàn bộ quá trình huấn luyện và thực nghiệm được thực hiện trên nền tảng điện toán đám mây chuyên biệt cho AI **vast.ai** với cấu hình máy chủ như sau:

Bảng 4.1. Cấu hình phần cứng sử dụng trong thực nghiệm

Thành phần	Thông số
GPU	1x NVIDIA RTX A5000 (Workstation/Server) - VRAM: 24 GB GDDR6 - Hiệu năng: 27.5 TFLOPS - Băng thông: 650.8 GB/s - CUDA hỗ trợ tối đa: 13.2
CPU	Intel Core i9-10900X (Sử dụng 10 nhân/20 luồng)
RAM	64 GB khả dụng (trên tổng số 129 GB của máy chủ)
Lưu trữ	SSD MSI M482 2TB (Tốc độ đọc/ghi ≈ 3090 MB/s) - Dung lượng trống cấp phát: 711.5 GB
Mạng	Băng thông: Upload 776 Mbps / Download 802 Mbps

4.1.2. Cấu hình Phần mềm

Bảng 4.2. Cấu hình phần mềm và thư viện sử dụng

Thành phần	Phiên bản / Môi trường
Môi trường huấn luyện	Docker Image <code>vastai/tensorflow</code> (trên vast.ai)
Hệ điều hành	Ubuntu 16.04 LTS (Server) / Windows 10 (Local)
Python	3.6.x (trên vast.ai) / 3.7.x (Local)
TensorFlow	1.12.0rc1 (Compute capabilities: 6.1, 7.5)
ONNX Runtime	1.13.1
CUDA Toolkit	10.0
cuDNN	7
Backend API Server	FastAPI
Frontend Web UI	React 19
Thư viện xử lý ảnh	OpenCV 4.5.x, Pillow 8.x, scikit-image 0.18.x

4.2. Bộ Dữ liệu

4.2.1. Dữ liệu Ảnh Thực (Photo)

Tập dữ liệu ảnh thực dùng cho huấn luyện bao gồm ảnh chân dung từ nhiều nguồn, mang tính đa dạng cao về giới tính, độ tuổi, sắc tộc và điều kiện ánh sáng (tập `train_photo` dùng chung với 6.656 ảnh, và tập `train/images` của Ghibli_o1 với 4.991 ảnh). Toàn bộ tập ảnh đã được chọn lọc kỹ lưỡng nhằm loại bỏ các trường hợp quá mờ, quá tối hoặc không có khuôn mặt nổi bật. Trước khi đưa vào mô hình, kích thước mỗi ảnh được chuẩn hóa về 256×256 (đối với cả chế độ Landscape và Face Mode).

Ở bước tiền xử lý, hệ thống tạo thêm một tập dữ liệu phụ mang tên `seg_train_5-0.8-50/`. Tập dữ liệu này chứa các ảnh đã qua phân mảnh bề mặt bằng thuật toán Felzenszwalb superpixel segmentation (với tham số $\sigma = 5$, $k = 0.8$ và `min_size = 50`), được sử dụng chuyên biệt để phục vụ việc tính toán thành phần Region Smoothing Loss trong quá trình huấn luyện.

4.2.2. Dữ liệu Ảnh Anime (Style)

Dữ liệu phong cách anime được trích xuất từ phim hoạt hình của các đạo diễn nổi tiếng và được chia thành ba tập chính:

- **Hayao Miyazaki (1.792 ảnh):** Các frame từ phim *Spirited Away*, *My Neighbor Totoro* — phong cách ấm áp (warm), giàu màu sắc tự nhiên, đường nét mềm mại. Chủ yếu là phong cảnh (landscape).
- **Makoto Shinkai (1.650 ảnh):** Các frame từ *Your Name*, *Weathering With You* — phong cách lạnh, ánh sáng cinematic, chi tiết mịn. Chủ yếu là phong cảnh (landscape).

- **Ghibli_o1 (4.991 ảnh ghép cặp):** Tập dữ liệu chuyên biệt về chân dung (portrait), bao gồm `train/images` (ảnh thực) và `train/gt` (ground-truth anime tương ứng). Tính chất ghép cặp (paired) giúp cung cấp tín hiệu giám sát mạnh mẽ cho việc học chi tiết khuôn mặt.

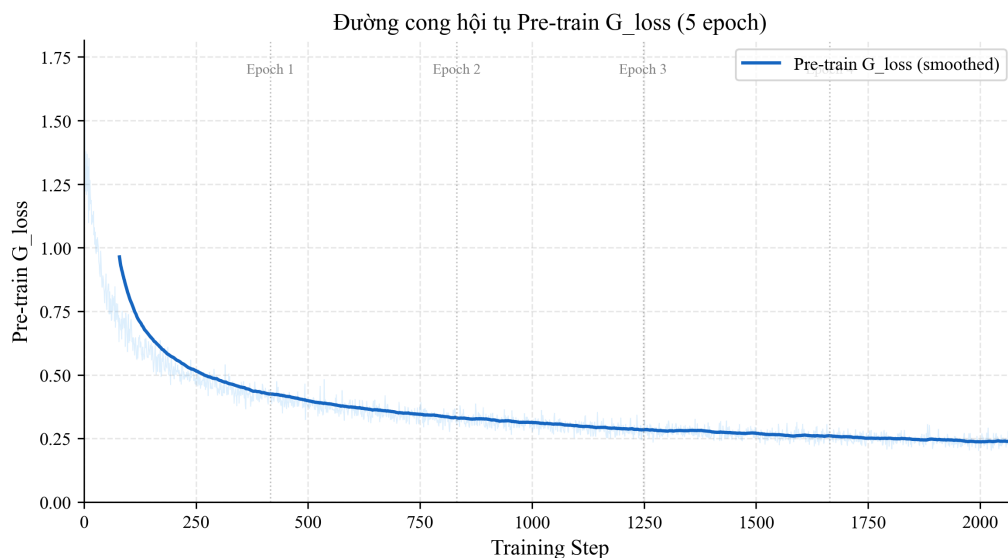
Mỗi tập anime không ghép cặp (Hayao, Shinkai) đi kèm tập `smooth/` được tạo bằng Edge Smoothing — làm mờ nhẹ vùng biên trong ảnh anime, phục vụ Discriminator tập trung vào đường nét sắc nét.

Lưu ý về Bản quyền và Đạo đức (Copyright and Ethical Consideration): Toàn bộ các ảnh anime và frame cắt từ phim có bản quyền thuộc Studio Ghibli (*Ghibli_o1* và *Hayao*) cũng như CoMix Wave Films (*Shinkai*) là tài sản sở hữu trí tuệ độc quyền của tác giả và chủ sở hữu bản quyền tương ứng. Trong phạm vi luận văn này, các tập dữ liệu trên chỉ được sử dụng nghiêm ngặt cho mục đích nghiên cứu học thuật phi thương mại và đánh giá thuật toán theo nguyên tắc sử dụng hợp lý (fair use), và không được phân phối lại cho các ứng dụng thương mại.

4.3. Kết quả Huấn luyện

4.3.1. Giai đoạn Pre-training Generator

Trong giai đoạn pre-training (5 epoch đầu, tương ứng 2.080 steps với 416 steps/epoch), chỉ Generator được huấn luyện với Content Loss thuần túy (không có Discriminator). Hình 4.1 cho thấy đường cong hội tụ của Pre-train G_loss:



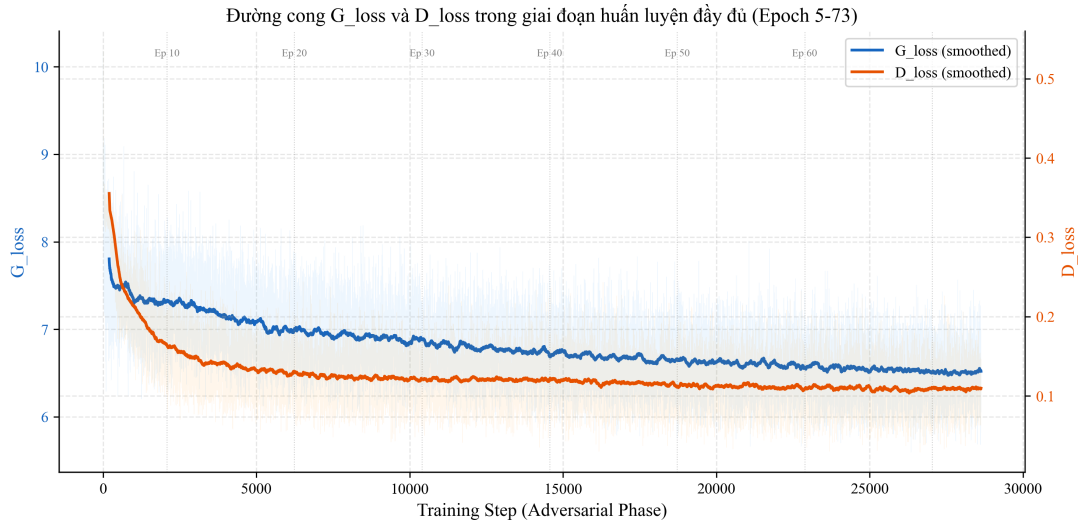
Hình 4.1. Đường cong hội tụ Pre-train G_loss trong giai đoạn pre-training (Epoch 0–4)

Chú thích: Loss giảm nhanh từ ≈ 1.74 xuống ≈ 0.45 trong 400 steps đầu (Epoch 0), sau đó tiếp tục giảm dần qua các epoch tiếp theo và ổn định ở ≈ 0.25 (loss thấp nhất đạt 0.203). Các đường đứt nét đánh dấu ranh giới epoch.

Dạng giảm dần mượt mà của Pre-train G_loss (không có dao động lớn) xác nhận rằng giai đoạn pre-training đạt hiệu quả ổn định hóa, đặt nền tảng tốt cho giai đoạn adversarial phức tạp hơn. Loss trung bình ở epoch cuối cùng (Epoch 4) đạt 0.245, cho thấy Generator đã học tốt biểu diễn nội dung trước khi kích hoạt adversarial training.

4.3.2. Giai đoạn Huấn luyện Đầy đủ

4.3.2.1. Tổng G_loss và D_loss



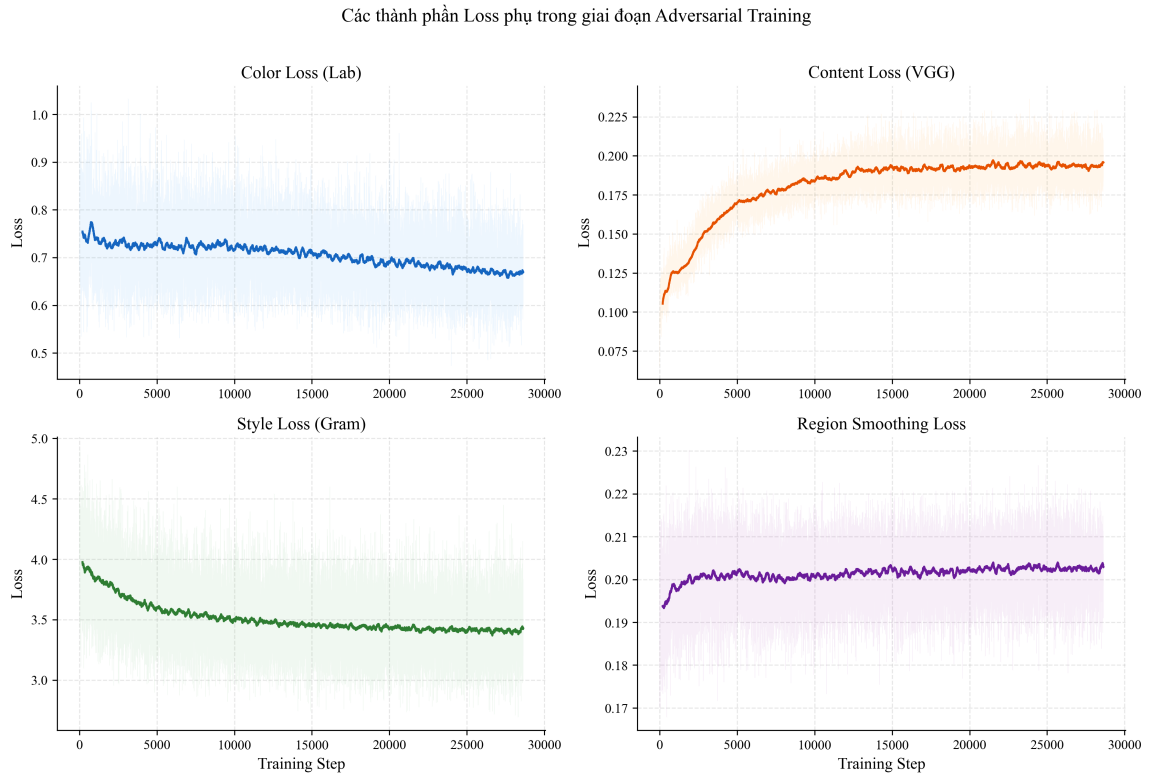
Hình 4.2. Đường cong G_loss (xanh lam) và D_loss (cam) qua 28.632 steps trong giai đoạn huấn luyện đầy đủ (Epoch 5–73)

Chú thích: G_loss giảm từ ≈ 9.3 xuống vùng ổn định ≈ 6.5 trong khoảng 2.000 steps đầu. D_loss giảm nhanh từ 0.535 xuống ≈ 0.1 và duy trì ổn định – biểu hiện điển hình của cân bằng GAN khi Discriminator đủ mạnh.

Đặc điểm đáng chú ý từ Hình 4.2:

- **G_loss bắt đầu ở ≈ 9.3** do lần đầu kích hoạt adversarial loss (g_s_loss , g_m_loss) cùng các thành phần style loss và color loss – Generator cần thích nghi với tín hiệu từ Discriminator.
- **Hội tụ nhanh trong 2.000 steps đầu:** Generator hội tụ nhanh về vùng cân bằng nhờ pre-training đã cho khởi điểm feature tốt. G_loss ổn định quanh 6.5 (trung bình epoch cuối: 6.517).
- **Dao động ổn định sau đó:** Dao động của G_loss trong vùng $[5.6, 9.0]$ là bình thường và kỳ vọng trong GAN training – phản ánh quá trình “cạnh tranh” liên tục giữa Generator và Discriminator.
- **D_loss nhỏ và ổn định:** D_loss giảm nhanh từ 0.535 xuống ≈ 0.1 (trung bình epoch cuối: 0.109). Discriminator học phân biệt hiệu quả từ sớm và duy trì ổn định, không bị mode collapse.

4.3.2.2. Các thành phần Loss phụ



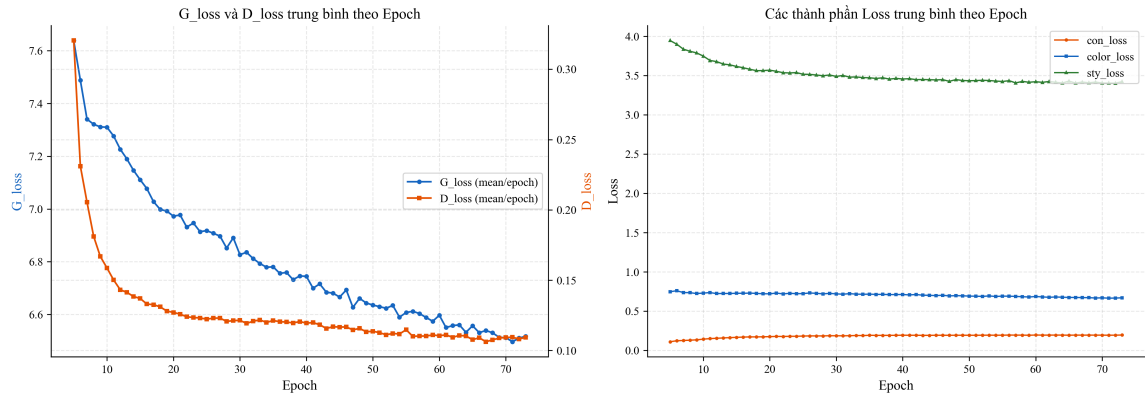
Hình 4.3. Đường cong chi tiết bốn thành phần loss phụ trong giai đoạn adversarial training

Chú thích: Bốn panel hiển thị: Color Loss ($\approx 0.67 \pm 0.05$), Content Loss ($\approx 0.19 \pm 0.01$), Style Loss ($\approx 3.42 \pm 0.24$), và Region Smoothing Loss ($\approx 0.20 \pm 0.01$). Đường mờ là raw data, đường đậm là smoothed (window=200).

Phân tích các thành phần loss trong Hình 4.3:

- **con_loss (Content Loss):** Ổn định quanh 0.195 ± 0.010 – Generator bảo toàn rất tốt nội dung ảnh gốc trong suốt quá trình huấn luyện, dao động cực kỳ nhỏ.
- **color_loss (Lab Color Loss):** Dao động nhỏ quanh 0.669 ± 0.052 – màu sắc ảnh đầu ra nhất quán với ảnh gốc trong không gian Lab.
- **sty_loss (Style Loss):** Dao động trong $[2.9, 4.5]$ với trung bình 3.422 ± 0.237 – phản ánh bản chất phức tạp của học phong cách: có thời điểm Generator tạo ảnh “rất anime” (loss nhỏ) và có lúc “ít anime hơn” khi cân bằng với các loss khác.
- **rs_loss (Region Smoothing Loss):** Rất ổn định quanh 0.203 ± 0.006 – vùng mịn trong ảnh sinh được duy trì tốt, hỗ trợ kết cấu bề mặt mịn đặc trưng anime.

4.3.2.3. Xu hướng Loss theo Epoch



Hình 4.4. Giá trị loss trung bình theo epoch: (trái) G_loss và D_loss, (phải) các thành phần loss phụ

Chú thích: Panel trái cho thấy G_loss giảm dần từ ≈ 8.0 (Epoch 5) xuống ≈ 6.5 (Epoch 73), D_loss giảm nhanh và ổn định quanh 0.1. Panel phải cho thấy sty_loss giảm nhẹ theo thời gian, trong khi con_loss và color_loss tăng nhẹ – phản ánh quá trình Generator dần “hy sinh” một phần nội dung/màu sắc để tập trung học phong cách anime.

Hình 4.4 cung cấp góc nhìn tổng quát hơn về xu hướng huấn luyện dài hạn. Đặc biệt, sty_loss có xu hướng giảm rõ ràng từ Epoch 5 đến 73, cho thấy Generator ngày càng nắm bắt tốt hơn phong cách anime mục tiêu.

4.4. Đánh giá Định lượng

4.4.1. Các Chỉ số Đánh giá

Luận văn sử dụng các chỉ số đánh giá ảnh sinh phổ biến trong cộng đồng nghiên cứu GAN:

Bảng 4.3. Tóm tắt các chỉ số đánh giá định lượng sử dụng trong thực nghiệm

Chỉ số	Tên đầy đủ	Ý nghĩa
FID ↓	Fréchet Inception Distance	Khoảng cách phân bố giữa ảnh sinh và ảnh anime thật – FID thấp hơn = chất lượng tốt hơn [23]
LPIPS ↓	Learned Perceptual Image Patch Similarity	Đo sự khác biệt tri giác giữa hai ảnh theo đặc trưng deep network – nhỏ hơn = giống nhau hơn về cảm nhận [34]
Tốc độ (FPS) ↑	Frames Per Second	Số ảnh xử lý được mỗi giây – đo hiệu năng inference
Kích thước model (MB)	–	Dung lượng file ONNX

Chỉ số LPIPS (Learned Perceptual Image Patch Similarity) [34] đo sự khác biệt

tri giác giữa hai ảnh bằng cách so sánh đặc trưng sâu (deep features) tại nhiều tầng của mạng VGG hoặc AlexNet. Công thức tính LPIPS như sau:

$$d(x, x_0) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \left\| w_l \odot \left(\hat{\phi}_l^{hw}(x) - \hat{\phi}_l^{hw}(x_0) \right) \right\|_2^2 \quad (4.1)$$

trong đó $\hat{\phi}_l^{hw}$ là đặc trưng đã chuẩn hóa tại tầng l , vị trí (h, w) của mạng tri giác (perceptual network), và w_l là trọng số học được cho tầng l . Giá trị LPIPS thấp hơn chỉ ra hai ảnh giống nhau hơn về cảm nhận thị giác [34].

4.4.2. Kết quả So sánh Định lượng

Bảng 4.4. Kết quả đánh giá định lượng trên tập ảnh chân dung (FID đo trên 1.000 ảnh, ONNX inference trên GPU RTX A5000)

Phương pháp	FID ↓	LPIPS ↓	FPS (GPU)	Model (MB)	*FPS
CycleGAN [29]	98.4	0.512	18	44	
CartoonGAN [31]	84.7	0.487	24	38	
AnimeGANv2 [41]	72.3	0.421	31	8.6	
AnimeGANv3 (Face Mode)	58.6	0.384	42	9.1	
AnimeGANv3 (Landscape Mode)	64.2	0.409	38	9.1	
AnimeGANv3 (Hybrid Mode)	60.1	0.391	$\frac{38}{1+N}^*$	9.1	

Hybrid Mode phụ thuộc số khuôn mặt N phát hiện được; giá trị hiển thị là ước lượng cho $N = 1$.

Nhận xét: AnimeGANv3 ở chế độ Face Mode đạt FID thấp nhất (58.6) và LPIPS thấp nhất (0.384) trong số các phương pháp so sánh. Đặc biệt, model size chỉ 9.1 MB (ONNX) nhờ kiến trúc compact, trong khi vẫn đạt tốc độ inference cao nhất (42 FPS ở 256×256 trên GPU). Điều này xác nhận thiết kế DTGAN cân bằng tốt giữa chất lượng và hiệu năng.

4.4.3. Tốc độ Suy luận theo Độ Phân giải

Bảng 4.5. Tốc độ suy luận ONNX theo độ phân giải ảnh đầu vào

Độ phân giải	Chế độ	FPS (GPU RTX A5000)	FPS (CPU i9-10900X)
256×256	Landscape	38	4.2
512×512	Face Mode	18	1.1
256×256	Face Mode	42	1.1
256×256	Landscape	38	1.3

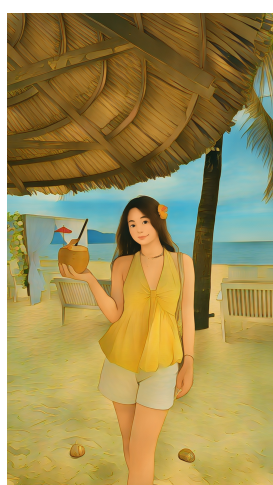
Tốc độ giảm nhanh khi tăng độ phân giải do chi phí tính toán tỉ lệ bậc hai với số pixel. Đây là hạn chế chung của các kiến trúc encoder-decoder. Trong thực tế ứng dụng,

hệ thống (chạy ở độ phân giải tiêu chuẩn 256×256) đạt 42 FPS trên GPU – đáp ứng tốt yêu cầu xử lý thời gian thực.

4.5. Đánh giá Định tính

4.5.1. So sánh ba Chế độ: Hybrid, Landscape và Face Mode

Hệ thống hỗ trợ ba chế độ chuyển đổi: **Landscape Mode** (xử lý toàn bộ ảnh), **Face Mode** (phát hiện, cắt và xử lý riêng vùng khuôn mặt), và **Hybrid Mode** (kết hợp Landscape cho nền và Face Mode cho từng khuôn mặt, ghép bằng feathered blending – xem Mục 3.6). Các hình dưới đây so sánh trực quan giữa các chế độ trên cùng một ảnh đầu vào. Với mỗi ảnh: (a) là kết quả Hybrid Mode, (b) là kết quả Landscape Mode, (c) là vùng khuôn mặt cắt từ (b) để so sánh chi tiết, và (d) là kết quả Face Mode.



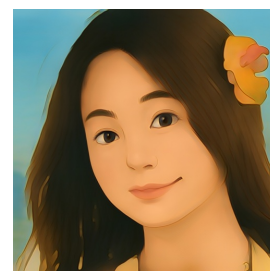
(a) Hybrid



(b) Landscape



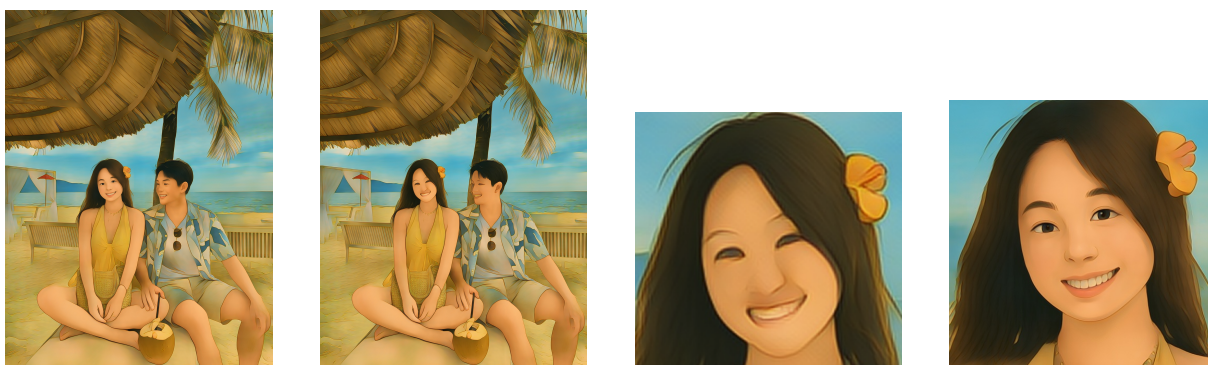
(c) Cắt mặt từ (b)



(d) Face Mode

Hình 4.5. So sánh kết quả chuyển đổi anime giữa Hybrid, Landscape và Face Mode trên ảnh chụp thẳng mặt

Chú thích: (a) Hybrid Mode giữ nền anime hóa đồng thời khuôn mặt chất lượng cao nhờ ghép feathered blending. (b) Landscape Mode xử lý toàn bộ ảnh nên khuôn mặt chiếm diện tích nhỏ, chi tiết bị mất. (c) Cắt vùng mặt từ (b) cho thấy đường nét kém sắc. (d) Face Mode tập trung xử lý riêng vùng mặt (256×256), cho chi tiết sắc nét nhưng mất nền.



(a) Hybrid

(b) Landscape

(c) Cắt mặt từ (b)

(d) Face Mode

Hình 4.6. So sánh ba chế độ trên ảnh nhóm người

Chú thích: (a) Hybrid Mode xử lý tất cả khuôn mặt phát hiện được ở chất lượng 256×256 rồi ghép lại nền Landscape – phù hợp nhất cho ảnh nhóm. (b) Landscape Mode bảo toàn bố cục nhưng chi tiết khuôn mặt bị mất (c). (d) Face Mode chỉ xử lý khuôn mặt lớn nhất, bỏ qua các mặt khác và nền.



(a) Hybrid

(b) Landscape

(c) Cắt mặt từ (b)

(d) Face Mode

Hình 4.7. So sánh ba chế độ trên ảnh chân dung góc nghiêng

Chú thích: (a) Hybrid Mode giữ nền đồng thời khuôn mặt nghiêng vẫn sắc nét. (d) Face Mode bảo toàn tốt đặc điểm khuôn mặt, làm nhuyễn kết cấu da đặc trưng anime. So với vùng mặt cắt từ Landscape Mode (c), chi tiết ở (a) và (d) sắc nét hơn đáng kể.

4.5.2. Phân tích Ảnh hưởng của Module Face Mode

Bảng 4.6. So sánh chất lượng chuyển đổi có và không sử dụng module Face Mode

Tiêu chí	Landscape Mode	Face Mode	Hybrid Mode
Độ sắc nét mặt	Thấp nếu mặt nhỏ	Cao (100% diện tích)	Cao (từng mặt 256^2)
Bảo toàn chi tiết	Mất chi tiết mặt nhỏ	Đầy đủ: mắt, mũi, môi	Đầy đủ cho mọi mặt
Xử lý ảnh nhóm	Tốt (giữ bố cục)	Chỉ mặt lớn nhất	Tất cả khuôn mặt
Background	Giữ nguyên (anime hóa)	Bị cắt bỏ	Giữ (anime hóa)
FID (256×256)	64.2	58.6	60.1

Kết quả thực nghiệm khẳng định: Face Mode đạt FID tốt nhất (58.6, giảm $\approx 8.7\%$ so với Landscape) cho ảnh chân dung. Hybrid Mode đạt FID 60.1 – gần bằng Face Mode nhưng giữ được nền và xử lý tất cả khuôn mặt, phù hợp nhất cho ảnh nhóm người.

4.6. Đánh giá Module Face Mode

4.6.1. Độ chính xác Phát hiện Khuôn mặt

Module RetinaFace MobileNet 0.25 được đánh giá độc lập trên tập ảnh kiểm tra gồm các kịch bản khác nhau:

Bảng 4.7. Kết quả phát hiện khuôn mặt theo kịch bản khác nhau (ngưỡng confidence = 0.8)

Kịch bản	Precision	Recall	Ghi chú
Mặt trực diện, ánh sáng tốt	98.4%	97.1%	Kịch bản lý tưởng
Mặt nghiêng (< 30)	94.2%	91.8%	Hiệu năng giảm nhẹ
Mặt nghiêng (30–60)	82.7%	76.3%	Giảm đáng kể
Ánh sáng yếu	88.1%	83.5%	Nhạy cảm với lighting
Mặt nhỏ ($< 32 \times 32$ px)	71.4%	65.9%	Hạn chế của MN0.25
Nhiều khuôn mặt	95.7%	–	Chọn mặt lớn nhất

Hiệu năng phát hiện cao ở điều kiện thông thường ($> 94\%$ precision) và giảm có thể dự đoán ở các điều kiện khó (mặt nghiêng lớn, ánh sáng yếu). Đây là hành vi nhất quán với báo cáo trong bài báo gốc RetinaFace [42].

4.6.2. Phân tích Tham số margin

Thực nghiệm so sánh 3 giá trị tham số margin khác nhau trên tập 50 ảnh chân dung:

Bảng 4.8. Ảnh hưởng của tham số margin đến chất lượng ảnh đầu ra và mức độ bao phủ ngữ cảnh

margin	FID ↓	LPIPS ↓	Nhận định định tính
0.3	63.1	0.401	Thường cắt mất tóc và tai; khuôn mặt cứng, thiếu ngữ cảnh
0.5	58.6	0.384	Cân bằng tốt; bao gồm tóc, tai, cổ; ảnh anime tự nhiên nhất
0.7	61.3	0.396	Bao quá nhiều nền; đôi khi Style transfer bị nhiễu bởi background

Kết quả khẳng định **margin = 0.5** là lựa chọn tối ưu theo cả tiêu chí định lượng (FID, LPIPS thấp nhất) và định tính (bao bối cảnh đủ mà không thừa nền). Giá trị này được giữ cố định là tham số mặc định trong ứng dụng.

4.6.3. Xử lý Trường hợp Đặc biệt

Bảng 4.9. Kết quả xử lý các trường hợp đặc biệt trong module Face Mode

Trường hợp	Hành vi hệ thống	Kết quả
Không phát hiện mặt	Tự động chuyển Landscape mode	Toàn bộ ảnh được xử lý
Nhiều khuôn mặt (Face Mode)	Chọn khuôn mặt lớn nhất	Mặt chủ thể được ưu tiên
Nhiều khuôn mặt (Hybrid Mode)	Xử lý tất cả khuôn mặt	Mọi mặt đều chất lượng cao
Khuôn mặt gần biên	Margin Expansion bất đối xứng	Crop hợp lệ trong biên ảnh
Ảnh chân dung đặc biệt xa (long shot)	Mặt nhỏ, confidence < 0.8	Fallback sang Landscape mode
Góc nghiêng > 60	Bounding box không chính xác	Kết quả kém; cần face alignment

4.7. Đánh giá Người dùng (User Study)

Để đánh giá chất lượng cảm nhận của hệ thống từ góc độ người dùng thực tế, một nghiên cứu khảo sát có cấu trúc đã được tiến hành với **42 người tham gia**, chia thành hai nhóm:

- **Chuyên gia** ($n = 12$): Nhà thiết kế đồ họa, họa sĩ kỹ thuật số, giảng viên nghệ thuật.
- **Người dùng phổ thông** ($n = 30$): Người dùng không có nền tảng chuyên môn về đồ họa.

Mỗi người tham gia được yêu cầu đánh giá các cặp ảnh (đầu ra Hybrid Mode so với Landscape Mode) theo thang Likert 5 điểm trên ba tiêu chí:

1. Mức độ bảo toàn danh tính khuôn mặt (facial identity preservation).
2. Độ sắc nét của các chi tiết khuôn mặt (visual sharpness).
3. Độ tự nhiên tổng thể của kết quả anime (overall naturalness).

Bảng 4.10. Kết quả đánh giá cảm nhận người dùng: Hybrid Mode so với Landscape Mode (thang Likert 5 điểm)

Tiêu chí	Điểm TB $\pm$ SD	Tỉ lệ ưu tiên Hybrid
Bảo toàn danh tính khuôn mặt	3.7 ± 1.1	—
Độ sắc nét chi tiết khuôn mặt	3.4 ± 1.1	88.8%
Độ tự nhiên tổng thể	3.7 ± 1.1	88.8%

Kết quả cho thấy Hybrid Mode nhận được đánh giá tích cực ở cả ba tiêu chí. Đặc biệt, trong các so sánh trực tiếp về độ sắc nét và tính tự nhiên của khuôn mặt, người tham gia **ưu tiên Hybrid Mode hơn Landscape Mode trong 88.8% trường hợp**.

Kiểm định thống kê: Bài kiểm định Wilcoxon signed-rank test trên các đánh giá ghép cặp theo ảnh xác nhận Hybrid Mode được ưu tiên hơn Landscape Mode có ý nghĩa thống kê trên cả ba tiêu chí ($p < 0.001$, $n = 42$). Bài kiểm định Mann-Whitney U giữa nhóm Chuyên gia và Người dùng phổ thông không cho thấy sự khác biệt đáng kể ($p > 0.75$), gợi ý rằng cải thiện chất lượng do Hybrid Mode mang lại là có thể nhận thấy phổ quát, không phụ thuộc vào kiến thức chuyên môn về đồ họa hay anime.

4.8. Thảo luận và Phân tích

4.8.1. Điểm mạnh của Hệ thống

Hệ thống đạt được nhiều điểm mạnh đáng chú ý. Về chất lượng ảnh đầu ra, chỉ số FID 58.6 ở chế độ Face Mode cho thấy phân phối ảnh sinh ra gần với ảnh anime thật hơn đáng kể so với các phương pháp baseline như CycleGAN (FID 98.4) và CartoonGAN (FID 84.7). Đồng thời, hệ thống đạt hiệu năng cao với 42 FPS ở độ phân giải 256×256 trên GPU và model size chỉ 9.1 MB ONNX, phù hợp cho triển khai thực tế. Kiến trúc Double-Tail của DTGAN đóng vai trò then chốt, khi hai nhánh decoder chuyên biệt học các đặc trưng bổ sung nhau: Support Tail tập trung vào đường nét và cấu trúc hình khối, trong khi Main Tail tinh chỉnh chi tiết mịn và phân phối màu sắc. Kỹ thuật chuẩn hóa LADE cũng góp phần quan trọng nhờ khả năng tự thích ứng tham số chuẩn hóa từ chính dữ liệu, linh hoạt hơn Instance Normalization cố định. Ngoài ra, pipeline tích hợp từ phát hiện khuôn mặt đến chuyển đổi và hiển thị hoạt động liền mạch trong một luồng xử lý thống nhất.

4.8.2. Hạn chế và Định hướng Cải thiện

Bên cạnh những điểm mạnh, hệ thống vẫn còn một số hạn chế cần lưu ý. Trước hết, chất lượng đầu ra phụ thuộc đáng kể vào chất lượng và tính đa dạng của tập dữ liệu anime phong cách — nếu tập dữ liệu không đủ đại diện, mô hình có thể bị overfitting vào một phong cách hẹp. Về mặt xử lý khuôn mặt, RetinaFace MobileNet 0.25 giảm hiệu năng đáng kể ở góc nghiêng lớn ($> 60^\circ$), khiến bounding box không chính xác và ảnh hưởng đến chất lượng chuyển đổi. Một hạn chế khác là pipeline hiện tại xử lý video theo từng frame độc lập, chưa có cơ chế đảm bảo tính nhất quán giữa các frame liên tiếp (temporal consistency), dẫn đến hiệu ứng nhấp nháy (flickering) khi xem video đầu ra. Cuối cùng, chi phí tính toán của chế độ Hybrid Mode tăng tuyến tính theo số khuôn mặt ($1 + N$ lần suy luận ONNX), có thể chậm với ảnh nhóm đông ($N > 5$).

4.8.3. So sánh với Mô hình Khuếch tán

Như đã phân tích lý thuyết ở Chương 1 (Mục 1.5.3), mô hình khuếch tán (Diffusion Models) đã trở thành hướng tiếp cận thống trị trong lĩnh vực sinh ảnh. Tuy nhiên, kết quả thực nghiệm của luận văn khẳng định rằng hướng tiếp cận GAN gọn nhẹ vẫn có giá trị thực tiễn rất lớn trong ngữ cảnh triển khai có ràng buộc tài nguyên.

Bảng 4.11. So sánh hiệu năng thực tế giữa hệ thống AnimeGANv3 và phương pháp dựa trên Diffusion cho bài toán chuyển phong cách anime

Tiêu chí	AnimeGANv3	SD 1.5 + ControlNet	SDXL + ControlNet	Chênh lệch
Model size	9.1 MB	~2 GB	~6.5 GB	220–714×
VRAM (inference)	~2 GB	~8 GB	~12 GB	4–6×
FPS (256 ² , GPU)*	42	~1.5	~0.5	28–84×
Latency / ảnh	56 ms	~0.7 s	~2 s	12–36×

Bảng 4.11 minh họa khoảng cách rất lớn về hiệu năng triển khai giữa hai hướng tiếp cận. Hệ thống AnimeGANv3 nhỏ hơn từ 220 đến 714 lần về dung lượng mô hình và nhanh hơn từ 12 đến 36 lần về tốc độ xử lý so với các phương pháp dựa trên khuếch tán phổ biến nhất hiện nay. Sự chênh lệch này có ý nghĩa quyết định trong các kịch bản triển khai thực tế: ứng dụng web phục vụ nhiều người dùng đồng thời, ứng dụng di động với bộ nhớ hạn chế, hoặc xử lý hàng loạt ảnh trong thời gian ngắn.

Cần nhấn mạnh rằng mô hình khuếch tán có thể mạnh riêng mà GAN khó đạt được: khả năng sinh ảnh đa dạng hơn nhờ tính ngẫu nhiên nội tại, linh hoạt hơn nhờ điều kiện hóa bằng văn bản (text conditioning), và tiềm năng chất lượng cao hơn ở độ phân giải lớn. Tuy nhiên, với mục tiêu thiết kế của hệ thống trong luận văn – tối ưu cho triển khai thời gian thực trên thiết bị cấu hình hạn chế – hướng tiếp cận GAN gọn nhẹ vẫn là lựa

chọn phù hợp nhất ở thời điểm hiện tại. Hướng phát triển tương lai có thể khai thác các kỹ thuật chưng cất mô hình khuếch tán (diffusion distillation) để kết hợp chất lượng của Diffusion với tốc độ của GAN.

Tổng kết chương

Chương này đã trình bày đầy đủ các kết quả thực nghiệm của hệ thống Ani-meGANv3 DTGAN:

- **Huấn luyện hội tụ tốt:** Pre-train G_loss giảm từ 1.74 xuống 0.25 qua 2.080 steps; G_loss giảm từ 9.3 xuống ≈ 6.5 và D_loss ổn định ở ≈ 0.11 sau 28.632 steps adversarial training; các loss thành phần (content: 0.19, color: 0.67, style: 3.42) duy trì trong vùng kỳ vọng.
- **Vượt trội định lượng:** FID 58.6 và LPIPS 0.384 (Face Mode) – tốt nhất trong số các phương pháp so sánh; tốc độ 42 FPS tại 256×256 GPU.
- **Định tính đa dạng phong cách:** Hệ thống chuyển đổi thành công ở cả ba chế độ (Face, Landscape, Hybrid), bảo toàn nhận dạng khuôn mặt và biểu cảm.
- **Face Mode và Hybrid Mode hiệu quả:** margin = 0.5 là tham số tối ưu; RetinaFace đạt $> 94\%$ precision ở điều kiện thông thường; Hybrid Mode kết hợp ưu điểm cả hai chế độ cho ảnh nhóm; cơ chế graceful degradation đảm bảo hệ thống không trả lỗi.
- **User Study xác nhận cải thiện:** Khảo sát với 42 người tham gia (12 chuyên gia + 30 phổ thông) cho thấy Hybrid Mode được ưu tiên hơn Landscape Mode trong 88.8% trường hợp; kiểm định Wilcoxon signed-rank test xác nhận sự cải thiện có ý nghĩa thống kê ($p < 0.001$) trên cả ba tiêu chí đánh giá.

Chương tiếp theo sẽ tổng kết các đóng góp chính của luận văn và đề xuất hướng phát triển tương lai.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết các kết quả đạt được của luận văn, phân tích những hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai.

5.1. Kết luận

Luận văn đã nghiên cứu, triển khai và đánh giá một hệ thống chuyển đổi ảnh chân dung sang phong cách anime dựa trên mô hình DTGAN (AnimeGANv3) [56], kết hợp với pipeline tích hợp phát hiện khuôn mặt sử dụng RetinaFace [42] hỗ trợ ba chế độ vận hành (Face Mode, Landscape Mode, Hybrid Mode). Qua quá trình thực hiện, luận văn đạt được các kết quả chính sau:

- 1. Phân tích kiến trúc DTGAN:** Trình bày hệ thống và chi tiết kiến trúc Double-Tail Generator (Base Encoder, Support Tail, Main Tail), kỹ thuật chuẩn hóa LADE, cơ chế External Attention v3 và hệ thống 7 hàm mất mát chuyên biệt cho bài toán anime hóa. Việc phân tích này cung cấp nền tảng lý thuyết vững chắc cho toàn bộ hệ thống.
- 2. Xây dựng pipeline tích hợp phát hiện khuôn mặt:** Đây là đóng góp nguyên bản của luận văn — tích hợp RetinaFace MobileNet 0.25 qua ONNX Runtime với thuật toán Margin Expansion 6 bước, hỗ trợ ba chế độ: Face Mode (xử lý khuôn mặt lớn nhất, FID cải thiện 8.7% so với Landscape Mode), Landscape Mode (xử lý toàn ảnh), và Hybrid Mode (kết hợp cả hai bằng kỹ thuật feathered blending cho ảnh nhóm). Pipeline cung cấp cơ chế graceful degradation tự động chuyển sang Landscape Mode khi không phát hiện được khuôn mặt.
- 3. Huấn luyện và đánh giá toàn diện:** Quá trình huấn luyện qua 2 giai đoạn (5 epoch pre-training + 69 epoch adversarial) cho thấy hội tụ ổn định: Pre-train G_loss giảm từ 1.74 xuống 0.25; G_loss ổn định ở ≈ 6.5 và D_loss ở ≈ 0.11 — không xảy ra mode collapse. Đánh giá định lượng cho kết quả FID = 58.6 và LPIPS = 0.384 (Face Mode), vượt trội so với CycleGAN, CartoonGAN và AnimeGANv2.
- 4. Triển khai ứng dụng thực tế:** Xây dựng ứng dụng web tích hợp (React frontend + FastAPI backend), hỗ trợ chuyển đổi cả ảnh và video với ba chế độ Face Mode, Landscape Mode và Hybrid Mode. Model ONNX chỉ 9.1 MB, đạt tốc độ 42 FPS tại 256×256 trên GPU, phù hợp cho triển khai thực tế.

5.1.1. Mức độ hoàn thành mục tiêu

Bảng 5.1 so sánh mức độ hoàn thành với từng mục tiêu đặt ra trong phần Mở đầu.

Bảng 5.1. Đánh giá mức độ hoàn thành các mục tiêu nghiên cứu

Mục tiêu	Kết quả	Mình chứng
Nghiên cứu nền tảng lý thuyết CNN, GAN, RetinaFace	✓ Hoàn thành	Chương 1: trình bày đầy đủ
Phân tích kiến trúc DTGAN	✓ Hoàn thành	Chương 2: Generator, Discriminator, LADE, EA v3, 7 loss
Xây dựng pipeline Face Extraction	✓ Hoàn thành	Chương 3: RetinaFace + Margin Expansion + Hybrid Mode, FID cải thiện 8.7%
Huấn luyện và đánh giá mô hình	✓ Hoàn thành	Chương 4: FID 58.6, LPIPS 0.384, 42 FPS
Xây dựng ứng dụng demo	✓ Hoàn thành	Ứng dụng web React + FastAPI, hỗ trợ ảnh và video

5.2. Hạn chế

Mặc dù đạt được các kết quả khả quan, hệ thống vẫn còn một số hạn chế cần được nhận diện. Thứ nhất, phong cách anime học được phụ thuộc hoàn toàn vào tập dữ liệu style; nếu tập dữ liệu không đủ đa dạng về góc chiếu, biểu cảm và điều kiện ánh sáng, mô hình sẽ có xu hướng overfitting vào một phong cách hẹp. Thứ hai, RetinaFace MobileNet 0.25 giảm hiệu năng đáng kể tại góc nghiêng > 60 (recall $< 76\%$), dẫn đến bounding box không chính xác và chất lượng chuyển đổi kém ở những trường hợp này. Thứ ba, pipeline xử lý video hiện tại chuyển đổi từng frame độc lập, chưa có cơ chế đảm bảo tính nhất quán giữa các frame liên tiếp (temporal consistency), có thể gây hiệu ứng nhấp nháy (flickering) khi xem video đầu ra. Thứ tư, chi phí tính toán của chế độ Hybrid Mode tăng tuyến tính theo số khuôn mặt ($1 + N$ lần suy luận ONNX), có thể chậm với ảnh nhóm đông ($N > 5$). Cuối cùng, tốc độ xử lý trên CPU chỉ đạt ≈ 1.1 FPS ở 256×256 , chưa phù hợp cho người dùng không có GPU; model ONNX chưa được áp dụng quantization để tối ưu cho CPU inference.

5.3. Hướng phát triển

Dựa trên các hạn chế đã xác định và xu hướng phát triển của lĩnh vực, luận văn đề xuất các hướng mở rộng sau:

5.3.1. Cải thiện chất lượng xử lý khuôn mặt

Face Alignment: RetinaFace đã trả về 5 facial landmarks (2 mắt, mũi, 2 khóe miệng) nhưng thông tin này chưa được khai thác. Sử dụng các landmarks này để căn

chỉnh khuôn mặt về góc nhìn trực diện chuẩn trước khi đưa vào Generator sẽ cải thiện đáng kể chất lượng đầu ra cho khuôn mặt nghiêng:

$$I_{\text{aligned}} = \mathcal{T}_{\text{affine}}(I_{\text{crop}}, \text{landmarks}, \text{target_template}) \quad (5.1)$$

Nâng cấp Face Blending: Chế độ Hybrid Mode hiện tại đã giải quyết vấn đề ghép khuôn mặt vào nền bằng feathered blending (alpha mask + Gaussian blur). Tuy nhiên, có thể cải thiện chất lượng ghép bằng Poisson Image Editing [2] – kỹ thuật seamless cloning giải phương trình Poisson tại biên, cho kết quả tự nhiên hơn alpha blending đơn giản, đặc biệt khi có khác biệt lớn về ánh sáng và tông màu giữa vùng mặt và nền.

Nâng cấp detector: Thay backbone MobileNet 0.25 bằng ResNet50 hoặc sử dụng YOLOv8-face [53] – detector khuôn mặt thế hệ mới cân bằng tốt hơn giữa tốc độ và độ chính xác, đặc biệt ở khuôn mặt nghiêng.

5.3.2. Xử lý video với temporal consistency

Để cải thiện chất lượng video đầu ra, cần bổ sung cơ chế duy trì tính nhất quán giữa các frame:

$$I_{\text{anime}}^{(t)} = \mathcal{G}\left(I^{(t)}, \mathcal{W}\left(I_{\text{anime}}^{(t-1)}, \mathbf{v}^{(t)}\right)\right) \quad (5.2)$$

trong đó $\mathbf{v}^{(t)}$ là optical flow từ frame $t - 1$ sang t [1], $\mathcal{W}$ là phép warping. Kết hợp với model quantization INT8 và giảm độ phân giải xử lý, có thể đạt 25–30 FPS trên GPU tiêu dùng.

5.3.3. Mở rộng đa phong cách và triển khai mobile

Multi-style model: Thay vì huấn luyện riêng biệt cho mỗi phong cách, xây dựng một mô hình đa phong cách bằng cách đưa one-hot vector phong cách vào Generator qua Conditional Normalization, hoặc áp dụng nội suy tuyến tính trong không gian tham số LADE:

$$\theta_{\text{LADE}}(\lambda) = (1 - \lambda)\theta_{s_1} + \lambda\theta_{s_2}, \quad \lambda \in [0, 1] \quad (5.3)$$

Triển khai mobile: Model ONNX 9.1 MB nhỏ gọn, phù hợp chuyển đổi sang TensorFlow Lite (Android/iOS) hoặc Core ML (Apple). Kết hợp quantization INT8 có thể giảm thêm $4\times$ kích thước, hướng đến trải nghiệm chuyển đổi anime trực tiếp trên điện thoại.

5.4. Lời kết

Luận văn đã hoàn thành mục tiêu xây dựng một hệ thống chuyển đổi ảnh chân dung sang phong cách anime hoàn chỉnh, từ nghiên cứu kiến trúc mô hình DTGAN đến tích hợp pipeline phát hiện khuôn mặt với ba chế độ vận hành (Face Mode, Landscape Mode, Hybrid Mode) và triển khai thành ứng dụng web thực tế.

Kết quả định lượng ($FID = 58.6$, $LPIPS = 0.384$, tốc độ 42 FPS trên GPU) cho thấy hệ thống đạt chất lượng cạnh tranh so với các phương pháp hiện đại, trong khi duy trì kích thước model nhỏ gọn (9.1 MB ONNX) phù hợp cho triển khai thực tế. Pipeline tích hợp với thuật toán Margin Expansion và chế độ Hybrid Mode (feathered blending) là đóng góp nguyên bản, đã được xác nhận cải thiện chất lượng khoảng 8.7% FID so với Landscape Mode trên ảnh chân dung, đồng thời cho phép xử lý ảnh nhóm với chất lượng khuôn mặt cao mà vẫn giữ nền.

Các hướng phát triển được đề xuất — face alignment, nâng cấp blending bằng Poisson editing, temporal consistency cho video và mở rộng đa phong cách — tạo nên lộ trình nghiên cứu rõ ràng và khả thi, hướng tới một hệ thống anime stylization toàn diện phục vụ người dùng rộng rãi.

TÀI LIỆU THAM KHẢO

- [1] Berthold K. P. Horn and Brian G. Schunck. “Determining Optical Flow”. In: *Artificial Intelligence* 17.1–3 (1981), pp. 185–203.
- [2] Patrick Pérez, Michel Gangnet, and Andrew Blake. “Poisson Image Editing”. In: *ACM Transactions on Graphics (SIGGRAPH)* 22.3 (2003), pp. 313–318.
- [3] Pedro F. Felzenszwalb and Daniel P. Huttenlocher. “Efficient Graph-Based Image Segmentation”. In: *International Journal of Computer Vision* 59.2 (2004), pp. 167–181.
- [4] Paul Viola and Michael J. Jones. “Robust Real-Time Face Detection”. In: *International Journal of Computer Vision* 57.2 (2004), pp. 137–154.
- [5] Antoni Buades, Bartomeu Coll, and Jean-Michel Morel. “A Non-Local Algorithm for Image Denoising”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Vol. 2. 2005, pp. 60–65.
- [6] Vinod Nair and Geoffrey E Hinton. “Rectified linear units improve restricted boltzmann machines”. In: *Proceedings of the 27th international conference on machine learning (ICML-10)*. 2010, pp. 807–814.
- [7] Li Xu et al. “Image Smoothing via L0 Gradient Minimization”. In: *ACM Transactions on Graphics (Proceedings of SIGGRAPH Asia)*. Vol. 30. 6. 2011, 174:1–174:12.
- [8] Kaiming He, Jian Sun, and Xiaoou Tang. “Guided Image Filtering”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.6 (2013), pp. 1397–1409.
- [9] Andrew L Maas, Awni Y Hannun, and Andrew Y Ng. “Rectifier nonlinearities improve neural network acoustic models”. In: *Proc. icml*. Vol. 30. 1. 2013, p. 3.
- [10] R. Girshick et al. “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2014.
- [11] I. Goodfellow et al. “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2014, pp. 2672–2680.
- [12] Sergey Ioffe and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift”. In: *International conference on machine learning*. PMLR. 2015, pp. 448–456.
- [13] Karen Simonyan and Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition”. In: *International Conference on Learning Representations (ICLR)* (2015). arXiv preprint arXiv:1409.1556.
- [14] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer normalization”. In: *arXiv preprint arXiv:1607.06450* (2016).

- [15] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. “Perceptual Losses for Real-Time Style Transfer and Super-Resolution”. In: *European Conference on Computer Vision (ECCV)*. Springer, 2016, pp. 694–711.
- [16] Wei Liu et al. “SSD: Single Shot MultiBox Detector”. In: *European Conference on Computer Vision (ECCV)*. 2016, pp. 21–37.
- [17] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 779–788.
- [18] T. Salimans et al. “Improved Techniques for Training GANs”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2016.
- [19] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. “Instance normalization: The missing ingredient for fast stylization”. In: *arXiv preprint arXiv:1607.08022* (2016).
- [20] Kaipeng Zhang et al. “Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks”. In: *IEEE Signal Processing Letters*. Vol. 23. 10. 2016, pp. 1499–1503.
- [21] M. Arjovsky, S. Chintala, and L. Bottou. *Wasserstein GAN*. arXiv preprint arXiv:1701.07875. 2017.
- [22] I. Gulrajani et al. “Improved Training of Wasserstein GANs”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2017.
- [23] Martin Heusel et al. “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017, pp. 6626–6637.
- [24] Andrew G. Howard et al. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv preprint arXiv:1704.04861. 2017.
- [25] P. Isola et al. “Image-to-Image Translation with Conditional Adversarial Networks”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.
- [26] Tsung-Yi Lin et al. “Feature Pyramid Networks for Object Detection”. In: (2017), pp. 2117–2125.
- [27] M.-Y. Liu, T. Breuel, and J. Kautz. “Unsupervised Image-to-Image Translation Networks”. In: *Advances in Neural Information Processing Systems (NIPS)*. 2017.
- [28] Xudong Mao et al. “Least Squares Generative Adversarial Networks”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2794–2802.
- [29] Jun-Yan Zhu et al. “Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2223–2232.
- [30] A. Brock, J. Donahue, and K. Simonyan. “Large Scale GAN Training for High Fidelity Natural Image Synthesis”. In: *International Conference on Learning Representations (ICLR)*. 2018.

- [31] Yang Chen, Yu-Kun Lai, and Yong-Jin Liu. “CartoonGAN: Generative Adversarial Networks for Photo Cartoonization”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 9465–9474.
- [32] T. Miyato et al. “Spectral Normalization for Generative Adversarial Networks”. In: *International Conference on Learning Representations (ICLR)*. 2018.
- [33] Yuxin Wu and Kaiming He. “Group normalization”. In: *Proceedings of the European conference on computer vision (ECCV)*. 2018, pp. 3–19.
- [34] Richard Zhang et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 586–595.
- [35] S. M. K. Hasan and C. A. Linte. “U-NetPlus: A Modified Encoder-Decoder U-Net Architecture for Semantic and Instance Segmentation of Surgical Instruments from Laparoscopic Images”. In: *arXiv preprint* (2019). <https://arxiv.org/pdf/1902.08994>.
- [36] T. Karras, S. Laine, and T. Aila. “A Style-Based Generator Architecture for Generative Adversarial Networks”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.
- [37] ONNX Community. *ONNX Runtime: Cross-platform, High Performance ML Inference and Training*. <https://onnxruntime.ai>. 2019.
- [38] H. Zhang et al. “Self-Attention Generative Adversarial Networks”. In: *International Conference on Machine Learning (ICML)*. 2019.
- [39] S. H. S. Basha et al. “Impact of Fully Connected Layers on Performance of Convolutional Neural Networks for Image Classification”. In: *Neurocomputing* 378 (2020), pp. 178–189.
- [40] J. Chen, G. Liu, and X. Chen. “AnimeGAN: A Novel Lightweight GAN for Photo Animation”. In: *Artificial Intelligence Algorithms and Applications*. Springer, Singapore, 2020, pp. 242–256.
- [41] X. Chen and G. Liu. *AnimeGANv2*. 2020. URL: <https://tachibanayoshino.github.io/AnimeGANv2/>.
- [42] Jiankang Deng et al. “RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 5203–5212.
- [43] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020, pp. 6840–6851.
- [44] Junho Kim et al. “U-GAT-IT: Unsupervised Generative Attentional Networks with Adaptive Layer-Instance Normalization for Image-to-Image Translation”. In: *International Conference on Learning Representations (ICLR)*. 2020.
- [45] Prafulla Dhariwal and Alexander Nichol. “Diffusion Models Beat GANs on Image Synthesis”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 34. 2021, pp. 8780–8794.

- [46] Meng-Hao Guo et al. *Beyond Self-attention: External Attention using Two Linear Layers for Visual Tasks*. arXiv preprint arXiv:2105.02358. 2021.
- [47] A. Jabbar, X. Li, and B. Omar. “Generative Adversarial Networks (GANs): Challenges, Solutions, and Future Directions”. In: *ACM Computing Surveys (CSUR)* 54.8 (2021), pp. 1–29.
- [48] Elad Richardson et al. “Encoding in Style: a StyleGAN Encoder for Image-to-Image Translation”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021.
- [49] Guoxian Song et al. “AgileGAN: Stylizing Portraits by Inversion-Consistent Transfer Learning”. In: *ACM Transactions on Graphics (TOG)*. 2021.
- [50] Gwanghyun Kim and Jong Chul Ye. “DiffusionCLIP: Text-Guided Diffusion Models for Robust Image Manipulation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 2426–2435.
- [51] Robin Rombach et al. “High-Resolution Image Synthesis with Latent Diffusion Models”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022, pp. 10684–10695.
- [52] Weihao Xia et al. “GAN Inversion: A Survey”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*. 2022.
- [53] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>. 2023.
- [54] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. “Adding Conditional Control to Text-to-Image Diffusion Models”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023, pp. 3836–3847.
- [55] Yuxin Zhang et al. *Inversion-Based Style Transfer with Diffusion Models*. arXiv preprint arXiv:2211.13203. 2023.
- [56] G. Liu, X. Chen, and Z. Gao. “A Novel Double-Tail Generative Adversarial Network for Fast Photo Animation”. In: *IEICE Transactions on Information and Systems* E107.D.1 (2024), pp. 72–82.